

KRISHI VAIDYA: AN AI-ASSISTED MOBILE SYSTEM FOR CROP DISEASE DIAGNOSIS AND ADVISORY



**A PROJECT WORK SUBMITTED TO THE
DEPARTMENT OF COMPUTER SCIENCE
EXAMPLE CAMPUS
INSTITUTE OF SCIENCE AND TECHNOLOGY
TRIBHUVAN UNIVERSITY
NEPAL
FOR THE AWARD OF
BACHELOR OF SCIENCE (B.Sc.) IN COMPUTER SCIENCE**

**BY
FULL NAME OF THE STUDENT
SYMBOL No. 0000000
T.U. REGISTRATION No. 000-0-00-00000-0000**

MAY 2026

RECOMMENDATION

This is to recommend that **Full Name of the Student**, (Symbol No. 0000000, T.U. Registration No. 000-0-00-00000-0000), has carried out project work entitled “**Krishi Vaidya: An AI-Assisted Mobile System for Crop Disease Diagnosis and Advisory**” for the requirement to the project work in Bachelor of Science (B.Sc.) degree in Computer Science under my/our supervision in the Department of Computer Science, Example Campus, Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal.

To my/our knowledge, this work has not been submitted for any other degree.

He/She has fulfilled all the requirements laid down by the Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal for the submission of the project work for the partial fulfillment of Bachelor of Science (B.Sc.) degree.

Supervisor Name

Supervisor

Department: Department of Computer Science

Campus/Institute: Example

University: Tribhuvan University

Co-Supervisor Name

Co-Supervisor

Department: Department of Computer Science

Campus/Institute: Example

University: Tribhuvan University

[26 May 2026]

DECLARATION

This project work entitled “**Krishi Vaidya: An AI-Assisted Mobile System for Crop Disease Diagnosis and Advisory**” is being submitted to the Department of Computer Science, Example Campus, Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal for the partial fulfillment of the requirement to the project work in Bachelor of Science (B.Sc.) degree in Computer Science. This project work is carried out by me under the supervision of Supervisor Name and co-supervision of Co-Supervisor Name in the Department of Computer Science, Example Campus, Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal.

This work is original and has not been submitted earlier in part or full in this or any other form to any university or institute, here or elsewhere, for the award of any degree.

Signature

Full Name of the Student

Symbol No. 0000000

T.U. Registration No. 000-0-00-00000-0000

[26 May 2026]

LETTER OF FORWARD

[Date: 26/05/2026]

On the recommendation of **Supervisor Name** and **Co-Supervisor Name**, this project work is submitted by **Full Name of the Student**, Symbol No. 0000000, T.U. Registration No. 000-0-00-00000-0000, entitled “**Krishi Vaidya: An AI-Assisted Mobile System for Crop Disease Diagnosis and Advisory**” is forwarded by the Department of Computer Science, Example Campus, for the approval to the Evaluation Committee, Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal

He/She has fulfilled all the requirements laid down by the Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal for the project work.

[Name of HOD]

Head of Department

Department of Computer Science

Example Campus

Tribhuvan University

BOARD OF EXAMINATION AND CERTIFICATE OF APPROVAL

This project work (PRO-406) entitled “**Krishi Vaidya: An AI-Assisted Mobile System for Crop Disease Diagnosis and Advisory**” by Full Name of the Student (Symbol No. 0000000 and T.U. Registration No. 000-0-00-00000-0000) under the supervision of Supervisor Name and co-supervision of Co-Supervisor Name in the Department of Computer Science, Example Campus, Institute of Science and Technology (IoST), Tribhuvan University (T.U.), is hereby submitted for the partial fulfillment of the Bachelor of Science (B.Sc.) degree in Computer Science. This report has been accepted and forwarded to the Controller of Examination, Institute of Science and Technology, Tribhuvan University, Nepal for the legal procedure.

Supervisor Name

Supervisor

Department: Department of Computer
Science

Campus/Institute: Example

University: Tribhuvan University

Co-Supervisor Name

Co-Supervisor

Department: Department of Computer
Science

Campus/Institute: Example

University: Tribhuvan University

External Examiner

External Examiner

Department: Department of Computer
Science

Campus/Institute: Example Campus

University: Tribhuvan University

Internal Examiner

Internal Examiner

Department: Department of Computer
Science

Campus/Institute: Example Campus

University: Tribhuvan University

[Name of HOD]

Head of Department

Department of Computer Science

Example Campus

Tribhuvan University

[26 May 2026]

ACKNOWLEDGEMENTS

The author expresses sincere gratitude to Supervisor Name for continuous guidance, constructive feedback, and academic support throughout the project work. Appreciation is also extended to Co-Supervisor Name, the faculty members of Department of Computer Science, and the administration of Example Campus for the facilities and encouragement provided during the completion of this work.

The author is equally grateful to classmates, friends, and family members for their support, motivation, and patience during the study, experimentation, and writing phases of this project.

Full Name of the Student

Symbol No. 0000000

T.U. Registration No. 000-0-00-00000-0000

[May 2026]

ABSTRACT

Krishi Vaidya is an integrated crop-health diagnosis-support system developed as a final-year B.Sc. CSIT project. The project combines a YOLO-based object-detection model, a Qwen-based vision-language model, an Express/Mongoose backend API, and a React Native/Expo mobile application into a single diagnosis workflow. The system is designed for image-based crop analysis where a farmer captures or selects a crop image, submits it through the mobile app, and receives a structured diagnosis and advisory response through backend and model-service orchestration.

The YOLO component was trained for agricultural plant-part localization using a curated dataset of 30,981 images and 147,467 object annotations across eleven classes. The VLM component was prepared for disease-oriented interpretation using structured image-text data, LoRA-based fine-tuning, quantized training, and prediction parsing. The backend implements authentication, domain models, repositories, scan handling, diagnosis orchestration, validation middleware, and advisory response support. The mobile app implements scan workflow, crop records, local SQLite persistence, synchronization structure, localization, and farmer-facing result display.

The completed work demonstrates a functional integrated prototype and provides manuscript-ready evidence through model metrics, validation outputs, generated architecture diagrams, source-backed tables, and documented test coverage. The strongest evidence is available for the YOLO model, VLM data/training pipeline, backend implementation, mobile architecture, and integration design. The main limitations are the current VLM prediction reliability, missing live end-to-end runtime evidence, incomplete mobile device validation, and the need for agricultural expert review before real-world advisory use.

Keywords: Krishi Vaidya, crop disease diagnosis, YOLO, vision-language model, mobile agriculture, back-end API

शोधसार

कृषि वैद्य B.Sc. CSIT अन्तिम वर्षको परियोजनाका रूपमा विकास गरिएको एकीकृत बाली स्वास्थ्य निदान तथा सल्लाह प्रणाली हो। यस परियोजनामा YOLO आधारित वस्तु पहिचान मोडेल, Qwen आधारित भिजन-ल्यान्ग्वेज मोडेल, Express/Mongoose ब्याकएण्ड API, र React Native/Expo मोबाइल अनुप्रयोगलाई एउटै निदान कार्यप्रवाहमा जोडिएको छ। किसानले बालीको तस्वीर मोबाइल अनुप्रयोगमार्फत पठाउँदा प्रणालीले ब्याकएण्ड र मोडेल सेवाको सहायताले संरचित निदान तथा सल्लाह उपलब्ध गराउने उद्देश्य राखेको छ।

YOLO भाग कृषि बोटबिरुवाका भागहरू पहिचान गर्न ३०,९८१ तस्वीर र १,४७,४६७ annotation भएको dataset प्रयोग गरी तयार गरिएको हो। VLM भाग रोगसम्बन्धी व्याख्या उत्पादन गर्न structured image-text data, LoRA fine-tuning, quantized training, र prediction parsing प्रयोग गरी तयार गरिएको हो। Backend ले authentication, data model, repository, scan handling, diagnosis orchestration, validation middleware, र advisory response सम्हाल्छ। Mobile app ले scan workflow, crop record, local SQLite persistence, synchronization structure, localization, र farmer-facing result display समेट्छ।

परियोजनाले model metrics, validation outputs, architecture diagrams, source-backed tables, र documented tests मार्फत कार्यान्वयन भएको integrated prototype प्रस्तुत गर्छ। मुख्य सीमितताहरू VLM prediction reliability, live end-to-end runtime evidence को अभाव, mobile device validation को अपूर्णता, र real-world advisory प्रयोग अघि agricultural expert review आवश्यक हुनु हुन्।

Keywords: Krishi Vaidya, crop disease diagnosis, YOLO, vision-language model, mobile agriculture, back-end API

LIST OF ACRONYMS AND ABBREVIATIONS

API:	Application Programming Interface
CSIT:	Computer Science and Information Technology
FAO:	Food and Agriculture Organization of the United Nations
F1:	Harmonic mean of precision and recall
IoU:	Intersection over Union
IoST:	Institute of Science and Technology
JSON:	JavaScript Object Notation
LoRA:	Low-Rank Adaptation
mAP:	Mean Average Precision
NSCA:	National Sample Census of Agriculture
NSO:	National Statistics Office
ONNX:	Open Neural Network Exchange
PDF:	Portable Document Format
QLoRA:	Quantized Low-Rank Adaptation
REST:	Representational State Transfer
SQLite:	Lightweight relational database engine used for local mobile storage
TFLite:	TensorFlow Lite
T.U.:	Tribhuvan University
VLM:	Vision-Language Model
YOLO:	You Only Look Once

LIST OF SYMBOLS

P	Precision
R	Recall
F_1	F1 score
IoU	Intersection over Union between predicted and ground-truth bounding boxes
mAP_{50}	Mean average precision at IoU threshold 0.50
$mAP_{50:95}$	Mean average precision averaged across IoU thresholds from 0.50 to 0.95
TP	True positives
FP	False positives
FN	False negatives
n	Number of samples, records, or observations

TABLE OF CONTENTS

Cover Page	
Recommendation	i
Declaration	ii
Letter of Forward	iii
Board of Examination and Certificate of Approval	iv
Acknowledgements	vi
Abstract	vii
Shodhsar (Abstract in Nepali)	viii
List of Acronyms and Abbreviations	ix
List of Symbols	x
List of Tables	xvi
List of Figures	xviii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.1.1 Crop disease diagnosis challenge	1
1.1.2 Computer vision and mobile advisory context	2
1.1.3 Krishi Vaidya project context	2
1.2 Problem Statement	3
1.2.1 Visual detection problem	3
1.2.2 Disease interpretation problem	3
1.2.3 Mobile access and advisory workflow problem	3
1.3 Objectives	4
1.3.1 General objective	4
1.3.2 Specific objectives	4
1.4 Scope	4

1.5	Limitations	5
1.6	Methodology Overview	5
1.7	Report Organization	6
CHAPTER 2: LITERATURE REVIEW		7
2.1	Digital Agriculture and Crop Disease Diagnosis	7
2.1.1	Farmer-facing diagnosis systems	7
2.1.2	Dataset and field-variability challenges	8
2.2	Object Detection in Agricultural Vision	8
2.2.1	YOLO-family detectors	9
2.2.2	Detection metrics	9
2.2.3	Deployment and quantization	9
2.3	Vision-Language Models for Diagnosis	10
2.3.1	Multimodal disease reasoning	10
2.3.2	LoRA fine-tuning	11
2.3.3	Quantized fine-tuning	11
2.3.4	Structured prediction and parseability	11
2.4	Mobile and Backend Systems for Agricultural Advisory	12
2.4.1	Mobile scanning workflows	12
2.4.2	Offline-first and synchronization patterns	12
2.4.3	API-backed advisory systems	13
2.5	Research Gap	13
CHAPTER 3: MATERIALS AND METHODS		15
3.1	YOLO Object-Detection Dataset	15
3.1.1	Dataset Construction	16
3.1.2	Data Sources and Class Mapping	16
3.2	YOLO Object-Detection Method	17
3.2.1	Model and Training Setup	17
3.2.2	Evaluation Metrics	19
3.2.3	Confidence Threshold Analysis	20
3.2.4	Deployment and Quantization Evaluation	20
3.3	VLM Disease-Diagnosis Dataset	22
3.3.1	Dataset Pipeline	22
3.3.2	Taxonomy and Label Normalization	23
3.3.3	Qwen2.5-VL SFT Export	24
3.4	VLM Fine-Tuning Method	24
3.4.1	Base Model and Adapter Design	24
3.4.2	Quantized Training Configuration	24

3.4.3	Evaluation and Prediction Parsing	26
3.5	Backend System Architecture	27
3.5.1	Runtime Modes and Application Initialization	28
3.5.2	Dependency Container	29
3.6	Backend API Design	29
3.6.1	Route Groups	29
3.6.2	Controller and Validation Pattern	30
3.7	Backend Data Model	31
3.7.1	MongoDB Collections and Domain Entities	31
3.7.2	Repository Layer	31
3.8	Backend Diagnosis and Advisory Pipeline	32
3.8.1	Image Validation	32
3.8.2	VLM Inference Bridge	33
3.8.3	Google AI Advisory Bridge	34
3.9	Backend Security, Uploads, and Reliability Controls	35
3.9.1	Authentication and Session Handling	35
3.9.2	Upload Handling and Cloudinary Storage	36
3.9.3	Request Safety and Error Handling	36
3.10	Mobile Application Architecture	37
3.10.1	Purpose and Implementation Scope	37
3.10.2	Application Shell and Navigation Structure	38
3.10.3	Provider, Repository, and Dependency Structure	39
3.10.4	Local Persistence and Offline-Aware Design	40
3.10.5	Scan Workflow and Diagnosis Flow	41
3.10.6	On-Device Model Asset Support	42
3.10.7	Localization, Accessibility, and Farmer-Facing Output	43
3.11	Integrated Diagnosis Workflow	44
3.11.1	Integration Objective	44
3.11.2	End-to-End Control Flow	45
3.11.3	Backend Orchestration Boundary	46
3.11.4	VLM Inference Gateway Boundary	47
3.11.5	Prediction Parsing and Advisory Generation	47
3.11.6	Mobile Offline and Slow-Network Integration	48
3.11.7	Structured Response and Farmer-Facing Output	48
3.11.8	Integration Verification Boundary	49
3.12	Testing and Validation	49
3.12.1	Validation Strategy	49
3.12.2	YOLO Validation	49
3.12.3	VLM Validation	51

3.12.4	VLM Inference API Validation	52
3.12.5	Backend Validation	52
3.12.6	Mobile Validation	53
3.12.7	Integration Validation	53
CHAPTER 4: RESULTS AND DISCUSSION		55
4.1	YOLO Training Results	55
4.1.1	Training Dynamics	55
4.1.2	Best-Epoch Performance	56
4.2	YOLO Deployment and Threshold Results	57
4.2.1	Threshold Selection Behavior	57
4.2.2	Quantization Tradeoffs	58
4.2.3	Per-Class Behavior	59
4.3	YOLO Error Analysis	60
4.3.1	Confusion Patterns	60
4.3.2	Error Categories	61
4.3.3	Summary of YOLO Findings	62
4.4	VLM Training and Resource Results	63
4.4.1	Fine-Tuning Performance	63
4.4.2	Runtime and Memory Behavior	64
4.5	VLM Prediction Results	65
4.5.1	Dataset Split and Class Distribution	65
4.5.2	Prediction Summary	67
4.5.3	Per-Class Prediction Metrics	68
4.5.4	Confusion Matrix and Prediction Distribution	68
4.5.5	Summary of VLM Findings	70
4.6	Backend Implementation Results	71
4.6.1	Implemented API Surface	71
4.6.2	Layered Implementation Outcome	71
4.7	Backend Domain Discussion	72
4.7.1	Authentication and User Management	72
4.7.2	Scan Lifecycle	72
4.7.3	Crop, Task, Note, and Advisory Management	73
4.8	Backend Diagnosis Integration Discussion	74
4.8.1	Structured Diagnosis Response	74
4.8.2	Confidence and Out-of-Distribution Handling	74
4.9	Backend Verification and Limitations	74
4.9.1	Automated Test Coverage	74
4.9.2	Current Backend Limitations	75

4.10 Mobile App Implementation Results	76
4.10.1 Implemented Application Surface	76
4.10.2 Scan and Diagnosis Workflow Result	77
4.10.3 Local Data and Offline Result	78
4.10.4 Crop Management and Farmer Record-Keeping Result	78
4.10.5 Model Deployment and Mobile Integration Result	80
4.10.6 Mobile Verification and Current Limitations	81
4.11 Integrated System Discussion	82
4.11.1 End-to-End Integration Outcome	82
4.11.2 Data Contract Strengths	82
4.11.3 Offline and Degraded-Network Behavior	83
4.11.4 Reliability Controls Across the Integrated Path	84
4.11.5 Integration Limitations and Evidence Gaps	84
4.11.6 Overall Integrated System Interpretation	85
4.12 Cross-System Results Discussion and Limitations	85
4.12.1 Synthesis of Major Results	85
4.12.2 Model-Level Interpretation	86
4.12.3 System-Level Interpretation	87
4.12.4 Reliability and Safety Interpretation	87
4.12.5 Current Limitations	88
4.12.6 Implications for Final Manuscript Claims	88
 CHAPTER 5: CONCLUSION AND RECOMMENDATIONS	 90
5.1 Conclusion	90
5.2 Major Contributions	91
5.3 Recommendations	92
5.3.1 Dataset and model improvements	92
5.3.2 System deployment improvements	92
5.3.3 Mobile app and user evaluation improvements	93
5.4 Future Enhancements	93
 REFERENCE	 95
 APPENDIX	 98
Appendix A: Supplementary Model Outputs	98
Appendix B: Backend API Details	98
Appendix C: Mobile Screenshots	99
Appendix D: Additional Setup and Build Notes	99

LIST OF TABLES

Table 1:	YOLO dataset class distribution after curation	16
Table 2:	YOLO training configuration used for the Krishi Vaidya detector .	19
Table 3:	VLM fine-tuning configuration	26
Table 4:	Backend runtime modes and route availability	28
Table 5:	Implemented backend route groups	30
Table 6:	Implemented backend data models	31
Table 7:	Implemented mobile route and workflow groups	39
Table 8:	Implemented mobile local persistence groups	40
Table 9:	Integrated online diagnosis control flow	46
Table 10:	Integrated data contract across diagnosis boundaries	48
Table 11:	Testing and validation evidence summary	54
Table 12:	Summary of training dynamics and best-epoch performance for the YOLO detector	56
Table 13:	Sensitivity sweep of confidence thresholds on the YOLO baseline model	58
Table 14:	Deployment efficiency comparison across YOLO model variants	59
Table 15:	Per-class F1-score comparison across YOLO model variants . . .	60
Table 16:	Error analysis breakdown by YOLO class	62
Table 17:	Training summary for Qwen2.5-VL LoRA fine-tuning	64
Table 18:	Runtime and resource envelope for VLM fine-tuning	65
Table 19:	Dataset split validation summary for VLM records	66
Table 20:	Qwen record distribution by VLM class	66
Table 21:	Prediction summary on the VLM test split	67
Table 22:	Per-class prediction metrics for the VLM test split	68
Table 23:	Prediction distribution and median latency for the VLM test split .	70
Table 24:	Backend implementation coverage by domain	71
Table 25:	Backend implementation layers and their realized responsibilities	72
Table 26:	Implemented backend test focus	75
Table 27:	Mobile application implementation coverage	76
Table 28:	Mobile model deployment evidence and integration notes	81
Table 29:	Mobile app verification status and remaining evidence	81
Table 30:	Integrated subsystem responsibilities	82
Table 31:	Reliability controls in the integrated workflow	84
Table 32:	Integrated system evidence status	85

Table 33:	Cross-system result synthesis	86
Table 34:	Cross-system limitations and required future evidence	88

LIST OF FIGURES

Figure 1:	YOLO system architecture overview showing the data, training, validation, deployment, and export pipeline.	15
Figure 2:	YOLO data pipeline flow from raw annotations to canonical export, splitting, and training structure.	17
Figure 3:	YOLO training pipeline flow from dataset preparation through training configuration, optimization, and checkpoint selection. .	18
Figure 4:	YOLO quantization and export flow showing the transition from trained weights to deployment-ready artifacts.	21
Figure 5:	YOLO deployment architecture flow showing the relationship between the trained detector, export formats, and deployment targets.	21
Figure 6:	VLM system architecture overview separating dataset processing, training, inference, and serving infrastructure.	22
Figure 7:	VLM data pipeline flow from raw sources through curation, canonicalization, export, and training data assembly.	23
Figure 8:	VLM training pipeline flow from curated dataset export through quantized fine-tuning and checkpoint generation.	25
Figure 9:	VLM inference pipeline flow showing request preparation, generation, structured parsing, and fallback handling.	27
Figure 10:	Backend system architecture overview showing the request stack and external service boundaries.	28
Figure 11:	Backend API routing flow showing runtime mode selection, route-group registration, and request dispatch boundaries. . . .	30
Figure 12:	Backend data model and persistence flow showing the relationship between controllers, services, repositories, and MongoDB collections.	32
Figure 13:	VLM API gateway flow showing the backend request path, readiness check, authentication, inference call, and response parsing boundary.	33
Figure 14:	Backend diagnosis pipeline flow showing image validation, VLM inference bridging, advisory generation, and response assembly.	34
Figure 15:	Backend authentication and user-management flow showing registration, login, token refresh, OTP handling, and session control.	36

Figure 16:	Mobile application architecture overview showing presentation, state, native modules, and local storage boundaries.	38
Figure 17:	Mobile application navigation and routing flow showing onboarding, tab navigation, and feature-stack transitions.	39
Figure 18:	Mobile offline synchronization data flow showing local writes, queue management, connectivity checks, and retry behavior.	41
Figure 19:	Mobile scan pipeline flow showing capture, preview, editing, diagnosis submission, and retry-aware request handling.	42
Figure 20:	Mobile on-device ML inference flow showing model selection, load fallback, runtime precision switching, and benchmark support.	43
Figure 21:	Combined flow-diagram overview covering the subsystem work-flows used in the manuscript.	45
Figure 22:	YOLO validation and evaluation flow showing metric computation, threshold analysis, and validation outputs.	50
Figure 23:	VLM validation and evaluation flow showing dataset checks, prediction validation, and parse-error handling.	51
Figure 24:	Training dynamics of the Krishi Vaidya YOLO detector	55
Figure 25:	Confidence-threshold sensitivity of the YOLO detector	57
Figure 26:	Deployment comparison across baseline and quantized YOLO variants	59
Figure 27:	Confusion matrix of the YOLO detector	61
Figure 28:	Training dynamics of the Qwen2.5-VL LoRA fine-tuning run	63
Figure 29:	Runtime and memory behavior of the VLM training run	65
Figure 30:	Dataset composition for the VLM Qwen export	66
Figure 31:	Prediction performance of the VLM on the test split	67
Figure 32:	Confusion matrix for VLM prediction results	69
Figure 33:	Inference diagnostics for VLM predictions	69
Figure 34:	Representative first-run onboarding screenshots showing language selection, camera permission, crop selection, and onboarding completion.	76
Figure 35:	Representative dashboard and settings screenshots showing the home surface, crop summary, export controls, notifications, and accessibility-related configuration.	77
Figure 36:	Representative scan and diagnosis screenshots showing capture, preview, editor support, diagnosis completion, structured result display, and treatment guidance.	78
Figure 37:	Representative crop-management screenshots showing the crop dashboard and crop-registration flow.	79

Figure 38:	Representative note-creation screenshot showing crop-linked note entry and attachment flow.	80
-------------------	--	----

CHAPTER 1

1. INTRODUCTION

1.1 Background

Agriculture is not only an economic sector in Nepal; it is a daily livelihood system for a large part of the population. The Government of Nepal publishes national agricultural statistics to track crop production, livestock, land use, and the role of agriculture in the national economy (Ministry of Agriculture and Livestock Development, 2024). The National Statistics Office also conducted the National Sample Census of Agriculture 2021/22 across all districts and local levels to provide benchmark data on agricultural holdings, land use, crop areas, livestock, irrigation, machinery, labour, and agricultural inputs (National Statistics Office, 2024). These official efforts show that agriculture remains a planning, livelihood, and food-security priority for Nepal.

Crop disease diagnosis sits at the practical centre of this problem. A disease symptom often appears first as a visual change on a leaf, fruit, panicle, ear, or other plant structure. At that moment, the farmer needs timely guidance: whether the symptom is serious, whether it is likely to spread, what crop part is affected, and whether expert consultation is needed. If this decision is delayed or made incorrectly, the effect can move beyond one plant and affect crop yield, input cost, and household income. FAO reports that plant pests and diseases can cause up to 40 percent annual crop-production losses globally, which underlines the seriousness of plant-health problems even though the exact loss depends on crop, location, and season (Food and Agriculture Organization of the United Nations, 2026a).

1.1.1 Crop disease diagnosis challenge

Traditional disease diagnosis depends on farmer experience, local agro-vet advice, extension workers, or laboratory confirmation. These channels are important, but they are not always available at the exact time a farmer observes a symptom in the field. Nepal's geography, rural settlement patterns, variable connectivity, and uneven access to specialist services make this problem more difficult. A farmer may have a phone and a crop image, but still lack immediate access to a trained plant-health expert.

The diagnosis challenge is also technical. Field images are not controlled laboratory sam-

ples. They may include unclear lighting, cluttered background, multiple plant parts, partial symptoms, low image quality, or mixed disease stages. A reliable digital system must therefore do more than classify a clean image. It must support image capture, model inference, structured interpretation, advisory output, local record keeping, and uncertainty handling.

1.1.2 Computer vision and mobile advisory context

Computer vision can support agricultural diagnosis by extracting information from crop images. Object detection can identify relevant plant structures, while vision-language models can produce disease-oriented interpretation from visual input. Mobile technology is important because the smartphone is the most realistic interface for many field-level users. A diagnosis-support system therefore becomes useful only when the model layer is connected to a mobile workflow and a backend service that can store records, manage users, validate requests, and return structured advisories.

This direction is aligned with Nepal’s movement toward digital agriculture. FAO reports that Nepal’s digital agriculture strategy work emphasizes digital infrastructure, digital skills, public-private coordination, advisory services, and rural livelihood improvement (Food and Agriculture Organization of the United Nations, 2026b). Krishi Vaidya is positioned within this direction as a student-built integrated prototype for crop-health support.

1.1.3 Krishi Vaidya project context

Krishi Vaidya is an integrated final-year B.Sc. CSIT project composed of four major technical subsystems. The YOLO subsystem performs plant-part object detection. The VLM subsystem performs image-based disease interpretation and structured output generation. The backend subsystem provides authentication, records, scan handling, diagnosis orchestration, advisory generation, validation, and persistence. The mobile application provides the farmer-facing interface for image capture, scan submission, diagnosis viewing, crop records, local storage, and offline-aware workflow.

The project is therefore not a single machine-learning script. It is a full diagnosis-support prototype where trained model components are connected to application software. This manuscript documents the design, implementation, results, limitations, and remaining validation needs of that integrated system.

1.2 Problem Statement

Farmers need timely, understandable, and accessible support for crop disease diagnosis, but expert diagnosis is not always available at the field level. Existing digital approaches often focus on isolated image classification or model accuracy without fully addressing the workflow required by a practical advisory system. A useful crop-health support system must connect image capture, object localization, disease interpretation, backend validation, advisory generation, mobile display, local records, and failure handling.

The problem addressed by this project is therefore:

How can an integrated mobile, backend, object-detection, and vision-language system be designed and implemented to support image-based crop-health diagnosis and farmer-facing advisory output for the Nepalese agricultural context?

1.2.1 Visual detection problem

Crop images contain different plant structures, and disease-related reasoning often depends on which part of the plant is visible. A system that ignores plant-part localization may treat a complex field image as a simple classification input. Krishi Vaidya addresses this by training a YOLO object detector on agricultural plant-part classes such as leaves, panicles, grain clusters, fruits, ears, and vegetable heads.

1.2.2 Disease interpretation problem

Disease diagnosis requires more than detecting a plant object. The system must interpret visual evidence and produce structured information that can be converted into an advisory response. Krishi Vaidya addresses this through a VLM pipeline trained and evaluated for disease-oriented output. The project also records the current weakness of this component: prediction quality and parseability require further improvement before the VLM can be treated as a robust diagnostic authority.

1.2.3 Mobile access and advisory workflow problem

Even a useful model is incomplete if farmers cannot access it through a practical interface. The mobile app must support image capture, diagnosis requests, offline records, synchronization, crop records, and understandable result presentation. The backend must protect

the API, validate requests, call the diagnosis service, store records, and return consistent responses. Krishi Vaidya addresses this by implementing both the mobile and backend layers rather than limiting the project to model training.

1.3 Objectives

1.3.1 General objective

The general objective of Krishi Vaidya is to design and implement an integrated crop-health diagnosis-support system that uses object detection, vision-language modelling, backend services, and a mobile application to assist farmers with image-based crop disease interpretation and advisory output.

1.3.2 Specific objectives

1. To prepare and train a YOLO-based object-detection model for agricultural plant-part localization.
2. To prepare and fine-tune a VLM-based disease interpretation pipeline using structured image-text data.
3. To implement a backend API for user authentication, crop records, scan handling, diagnosis orchestration, advisory generation, validation, and persistence.
4. To implement a mobile application for image-based scanning, diagnosis display, local records, offline-aware storage, synchronization, and farmer-facing output.
5. To integrate the mobile app, backend, and model services into a documented diagnosis workflow.
6. To evaluate the implemented components using available model metrics, validation outputs, source-backed tests, generated diagrams, and documented evidence gaps.

1.4 Scope

The scope of this project includes the design, implementation, and manuscript documentation of the Krishi Vaidya prototype. The model scope includes YOLO object detection for plant-part localization and VLM-based disease interpretation. The backend scope includes the implemented Express/Mongoose service structure, scan and diagnosis workflow, advisory response handling, security middleware, validation, and tests available in

the repository. The mobile scope includes the implemented React Native/Expo app structure, scan workflow, local SQLite persistence, offline-aware behavior, synchronization structure, crop and task records, and farmer-facing diagnosis screens.

The scope does not include claiming a certified medical, agronomic, or government-approved diagnosis system. The project is a research and engineering prototype. It is suitable for demonstrating integrated software and model development, but it requires further expert review, field testing, and deployment hardening before real-world advisory use.

1.5 Limitations

The main limitations of the project are as follows:

1. The YOLO dataset is large and curated, but class distribution is imbalanced; some classes have much fewer annotations than others.
2. The VLM pipeline is implemented and validated, but the current prediction metrics and parse-error count show that structured diagnostic reliability must be improved.
3. The backend has scan-focused test evidence, but broader endpoint coverage for all domains is still required.
4. The mobile app implementation is source-verified, but final screenshots, physical-device testing, runtime latency, and offline synchronization demonstrations still need to be captured.
5. The integration workflow is source-verified, but live mobile-to-backend-to-VLM request-response evidence remains a final validation requirement.
6. The advisory output has not yet been validated by certified agricultural experts and should not be used as a replacement for expert consultation.

1.6 Methodology Overview

The project was implemented using an evidence-first workflow. First, the YOLO dataset was constructed, curated, trained, validated, and evaluated using detection metrics, threshold analysis, quantization comparison, and error analysis. Second, the VLM dataset was prepared using structured image-text records, then fine-tuned using LoRA and quantized training methods, with evaluation focused on dataset validity, training behavior, resource

use, prediction metrics, and parseability. Third, the backend was implemented as a layered API with routes, controllers, services, repositories, models, middleware, and tests. Fourth, the mobile app was implemented as a React Native/Expo application with routing, feature modules, local persistence, scan workflow, backend communication, and offline-aware behavior. Finally, the integration workflow was mapped across the mobile, backend, VLM, and advisory layers, and the testing/validation evidence was documented.

1.7 Report Organization

This report is organized into five chapters. Chapter 1 introduces the project context, problem, objectives, scope, limitations, and methodology overview. Chapter 2 reviews the literature and technical background related to digital agriculture, crop disease diagnosis, YOLO object detection, vision-language models, mobile systems, backend systems, and the research gap. Chapter 3 presents the materials and methods used to build the YOLO model, VLM pipeline, backend, mobile app, integration workflow, and validation structure. Chapter 4 presents results and discussion, including model metrics, implementation outcomes, integration interpretation, limitations, and evidence gaps. Chapter 5 concludes the work and provides recommendations for future improvement. The appendix provides supplementary implementation and evidence material.

CHAPTER 2

2. LITERATURE REVIEW

2.1 Digital Agriculture and Crop Disease Diagnosis

Agriculture remains a core national sector in Nepal and a direct livelihood concern for rural households. The Government of Nepal's agricultural statistics report records agriculture as a major component of national production, while the National Sample Census of Agriculture 2021/22 was designed to provide benchmark data on holdings, land use, crops, livestock, irrigation, machinery, labour, and agricultural inputs across all districts and local levels (Ministry of Agriculture and Livestock Development, 2024; National Statistics Office, 2024). This national context matters for Krishi Vaidya because crop disease diagnosis is not an isolated technical problem. It is a field-level decision-support problem that affects farmers who must decide whether a visible crop symptom requires treatment, monitoring, or expert consultation.

Plant disease and pest pressure is also a global food-security issue. FAO reports that up to 40 percent of global crop production can be lost annually due to plant pests and diseases (Food and Agriculture Organization of the United Nations, 2026a). The exact loss varies by crop, location, season, management practice, and pathogen pressure, but the underlying implication is consistent: delayed or incorrect diagnosis can convert a manageable crop-health problem into a production and income loss. For smallholder systems, this risk is amplified because advisory services, laboratory diagnosis, and trained plant-health experts are not always available at the time and place where the farmer first observes the symptom.

2.1.1 Farmer-facing diagnosis systems

Farmer-facing diagnosis systems must solve a different problem from laboratory plant pathology. They must accept imperfect field images, guide a non-specialist user, and return information in a form that is understandable and actionable. A useful system should therefore combine visual analysis with farmer-oriented communication. Image recognition alone may identify a visual class, but an advisory system must also explain the likely issue, severity, possible management steps, and the uncertainty of the result.

Krishi Vaidya follows this practical direction. The project does not treat model prediction

as the only output. Instead, the mobile app, backend, YOLO detector, VLM diagnosis pipeline, and advisory response work together so that image capture, analysis, record keeping, and farmer-facing output are connected within one system. This is consistent with the direction of digital agriculture in Nepal. FAO’s Nepal digital agriculture work highlights the need for stronger digital infrastructure, skilled users, coordinated strategy, and advisory services that can support rural livelihoods (Food and Agriculture Organization of the United Nations, 2026b).

2.1.2 Dataset and field-variability challenges

Dataset quality is a central challenge in crop disease diagnosis. Public plant-disease datasets make model development possible, but they do not automatically represent every field condition. The PlantVillage study demonstrated the value of image-based plant disease recognition using a large curated leaf-image dataset (Mohanty et al., 2016). However, controlled or semi-controlled leaf imagery can differ substantially from images captured by farmers in real fields. Field images may contain mixed lighting, partial occlusion, cluttered background, motion blur, multiple plant organs, and symptoms at different stages.

This limitation explains why Krishi Vaidya uses separate but connected model components. The YOLO component focuses on locating plant structures relevant to the image, while the VLM component produces structured disease-oriented interpretation. The project therefore separates object localization from disease reasoning rather than forcing a single classifier to perform every task. The literature supports this separation because plant disease analysis can involve classification, detection, segmentation, and diagnosis, depending on the visual target and the desired output (Liu & Wang, 2021).

2.2 Object Detection in Agricultural Vision

Object detection is appropriate when the system must identify not only what is present in an image but also where it appears. In agricultural images, this is useful because disease symptoms and plant structures may occupy only part of the image. A detector can localize leaves, panicles, fruits, ears, heads, and other crop parts that may then support downstream reasoning or user feedback. For Krishi Vaidya, this is the role of the YOLO model: it provides a fast object-detection layer for plant-part localization rather than replacing the disease-diagnosis logic.

2.2.1 YOLO-family detectors

YOLO-family detectors are widely used where real-time or near-real-time object detection is important. The Ultralytics YOLOv8 documentation describes YOLOv8 as a model family supporting tasks such as detection, segmentation, classification, pose estimation, and oriented bounding boxes, with multiple model sizes and deployment modes (Ultralytics, 2026a). The relevance to Krishi Vaidya is practical: a mobile agricultural system benefits from a detector that can be trained on custom crop classes, evaluated with standard detection metrics, and exported into deployment-oriented formats.

The YOLO section of this manuscript therefore uses YOLO terminology carefully. The model is not claimed to diagnose disease. Its manuscript role is object detection: training on the Krishi Vaidya bouncer dataset, validating localization performance, comparing thresholds, and evaluating deployment variants. This distinction is important because object detection and disease diagnosis measure different capabilities.

2.2.2 Detection metrics

Object detection performance is normally evaluated through metrics that combine localization and classification behavior. Precision measures how many predicted detections are correct. Recall measures how many ground-truth objects are found. F1 score summarizes the balance between precision and recall. Mean average precision at an intersection-over-union threshold evaluates detection quality across classes and confidence thresholds. These metrics are useful because an agricultural detector must avoid two opposite failures: false detections that confuse the downstream workflow and missed detections that hide relevant plant structures.

Krishi Vaidya uses these metrics in the YOLO validation and results sections. The project also uses threshold analysis because the default threshold may not be the best deployment threshold for the intended user experience. A higher threshold may reduce false positives but miss subtle or small objects; a lower threshold may increase recall but introduce noisy detections. The manuscript therefore discusses threshold behavior as an implementation decision, not only as a reported number.

2.2.3 Deployment and quantization

Deployment constraints are important in mobile and field-oriented agricultural systems. A model that performs well on a workstation may still be impractical if it is too large, too slow, or incompatible with the target runtime. Ultralytics documents model export

paths such as ONNX, TensorFlow Lite, and quantized formats, including INT8-oriented deployment options (Ultralytics, 2026b). This is directly relevant to Krishi Vaidya because the YOLO work includes deployment evaluation and because the mobile app contains TFLite model assets.

The literature and tooling therefore justify evaluating more than raw accuracy. The correct question is whether the trained detector can support an end-user workflow under realistic resource constraints. This manuscript treats quantization as a deployment tradeoff: compression and runtime feasibility must be evaluated together with the effect on prediction quality.

2.3 Vision-Language Models for Diagnosis

Vision-language models extend the disease-diagnosis workflow beyond fixed-label image classification. A VLM can consume an image and generate structured text, making it suitable for tasks where the output must include disease name, severity, explanation, and advisory content. In Krishi Vaidya, the VLM is used for disease-oriented reasoning and structured output, while the backend and mobile app transform that output into a farmer-facing response.

2.3.1 Multimodal disease reasoning

Qwen2-VL and Qwen2.5-VL provide relevant foundations for Krishi Vaidya because they are vision-language model families designed to process images and text jointly. Qwen2-VL introduced dynamic-resolution visual processing and multimodal positional encoding to improve perception across different visual inputs (Wang et al., 2024). Qwen2.5-VL further reports advances in visual recognition, object localization, document parsing, structured extraction, and multimodal reasoning (Bai et al., 2025). These capabilities align with the Krishi Vaidya requirement for image-based disease interpretation and structured outputs.

The project does not treat the VLM as an unrestricted chatbot. The VLM pipeline is constrained by prompt templates, class taxonomy, JSON-oriented records, prediction parsing, and validation metrics. This design is necessary because agricultural diagnosis must be reproducible enough for manuscript evaluation and safe enough for advisory presentation.

2.3.2 LoRA fine-tuning

Fine-tuning a full multimodal model can be computationally expensive. LoRA addresses this by freezing the base model weights and training low-rank adapter matrices, reducing the number of trainable parameters required for adaptation (Hu et al., 2021). This method is suitable for student and research environments where the goal is task adaptation without the cost of full model retraining.

Krishi Vaidya uses LoRA-based fine-tuning for the VLM component. The manuscript therefore reports the base model, adapter configuration, dataset format, and training behavior as method-level details. This is important because the final diagnosis behavior depends not only on the base VLM but also on how the adapter was trained and how predictions were parsed.

2.3.3 Quantized fine-tuning

Quantized fine-tuning reduces memory requirements further by combining low-bit model loading with adapter training. QLoRA demonstrates that gradients can be back-propagated through a frozen 4-bit quantized model into LoRA adapters, enabling efficient fine-tuning of large language models (Dettmers et al., 2023). Krishi Vaidya’s VLM section uses this line of work as the basis for discussing quantized training configuration and resource behavior.

The manuscript is careful not to overstate this result. Quantization improves feasibility, but it does not guarantee high diagnostic accuracy. The VLM results in Chapter 4 therefore distinguish between successful training execution, resource feasibility, dataset validity, and the current prediction-quality limitations observed in the validation outputs.

2.3.4 Structured prediction and parseability

For a farmer-facing advisory system, the model output must be parseable. A natural-language answer that cannot be reliably converted into application fields is difficult to store, validate, or show consistently in the mobile app. This is why Krishi Vaidya emphasizes structured output and prediction parsing. The VLM can generate diagnosis-oriented content, but the backend and mobile layers require stable fields such as crop, disease, confidence, severity, advice, and uncertainty.

The VLM results show that parseability is not a minor detail. A model can complete training while still producing a large number of outputs that fail the expected parser. The literature review therefore supports a central design claim of this manuscript: multimodal

diagnosis systems must evaluate both semantic correctness and output reliability.

2.4 Mobile and Backend Systems for Agricultural Advisory

The backend and mobile app convert model outputs into an operational system. Without these layers, the trained models would remain isolated experiments. Krishi Vaidya uses an API-centered backend for authentication, scan handling, data persistence, diagnosis orchestration, advisory generation, and user records. The mobile app provides the farmer-facing interface for image capture, scan submission, crop management, local storage, synchronization, and diagnosis display.

2.4.1 Mobile scanning workflows

React Native is an appropriate implementation basis for cross-platform mobile development because it enables native Android and iOS applications using React and platform-native capabilities (Meta Open Source, 2026). Expo adds a structured development workflow, routing support, and tooling for building and deploying React Native applications (Expo, 2026). In Krishi Vaidya, this technology choice supports rapid development of the scan workflow, crop records, local database, localization, and backend communication.

The agricultural requirement shapes the mobile design. A scan workflow should not assume perfect connectivity or expert users. It must guide image selection, preserve records, handle failures, and present output in a way that is understandable to the farmer. This is why the mobile implementation includes repository/provider structure, local persistence, sync status, and farmer-facing result screens rather than acting only as a thin image-upload client.

2.4.2 Offline-first and synchronization patterns

Rural and field environments often include unstable connectivity. Krishi Vaidya therefore treats offline behavior as a functional requirement. The mobile app stores records locally using SQLite and uses synchronization logic to reconcile local and remote state when connectivity permits. The literature on mobile systems commonly frames this as an offline-first problem: the app should remain usable when the network is unavailable, and synchronization should occur later without losing user data.

The current manuscript does not claim that all offline cases have been proven by device-level testing. It documents the implemented local storage and synchronization structure, then marks runtime evidence as a remaining validation requirement. This distinction is

necessary for a high-quality thesis: implementation evidence and field-readiness evidence are not the same.

2.4.3 API-backed advisory systems

The backend uses Express.js and Mongoose. Express documentation defines routing as the mechanism through which application endpoints respond to client requests, and middleware as the request processing stack that can modify requests, enforce checks, or end the response cycle (Express.js, 2026a, 2026b). Mongoose schemas define the shape of MongoDB documents and support validation, indexes, methods, middleware, and model construction (Mongoose, 2026). These ideas map directly to the backend architecture in Krishi Vaidya: routes expose the API surface, middleware handles security and validation, services implement business logic, repositories isolate persistence, and Mongoose models define stored records.

The VLM inference layer uses FastAPI. FastAPI is a Python framework for building APIs using standard Python type hints (FastAPI, 2026). This fits the VLM deployment boundary because the model pipeline is Python-based, while the main backend remains a Node.js/Express service. The result is a two-service architecture: the backend handles application state and orchestration, while the VLM service handles multimodal inference.

2.5 Research Gap

The reviewed literature and project context show three gaps that Krishi Vaidya addresses at a student-project scale.

First, many plant-disease systems stop at image classification. Such systems can be useful, but they do not fully address the workflow needed by a farmer who requires image capture, diagnosis, advice, history, and record keeping. Krishi Vaidya extends beyond isolated classification by combining object detection, VLM-based interpretation, backend orchestration, and a mobile app.

Second, model performance alone is not enough for deployment-oriented agricultural software. A usable system must consider model export, structured output, parseability, API contracts, offline behavior, security, and validation. Krishi Vaidya documents these engineering boundaries explicitly, including the limitations that remain unresolved.

Third, Nepal’s digital agriculture context requires practical, accessible, and mobile-oriented tools. Official sources identify agriculture as a nationally important sector and digital agriculture as an area requiring infrastructure, skills, coordination, and

advisory-service development (Food and Agriculture Organization of the United Nations, 2026b; Ministry of Agriculture and Livestock Development, 2024; National Statistics Office, 2024). Krishi Vaidya is positioned within this gap as an integrated prototype for crop-health support, not as a replacement for expert agronomists or laboratory diagnosis. Its contribution is the integration of trained model components with a backend and mobile workflow suitable for further validation, deployment hardening, and field evaluation.

CHAPTER 3

3. MATERIALS AND METHODS

3.1 YOLO Object-Detection Dataset

The YOLO component of Krishi Vaidya was prepared as a plant-part object-detection model. Its purpose is not to produce the final disease diagnosis by itself, but to identify visually relevant crop structures that can support the larger image-processing and diagnosis workflow. The detector therefore focuses on crop and plant-part classes that are useful for downstream agricultural analysis, including leaves, panicles, grain clusters, fruits, ears, and vegetable heads. The selection of YOLO for this role is consistent with its documented use as a real-time object-detection family with training, validation, prediction, and export workflows (Ultralytics, 2026a).

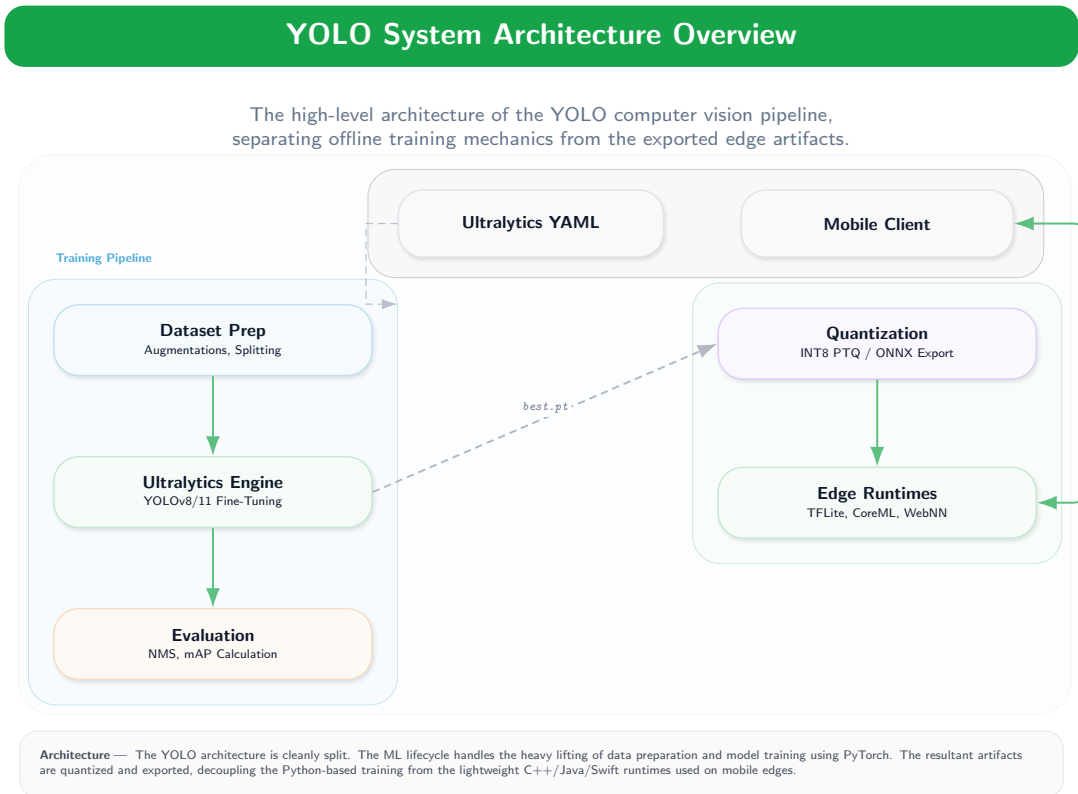


Figure 1: YOLO system architecture overview showing the data, training, validation, deployment, and export pipeline.

3.1.1 Dataset Construction

The object-detection dataset was assembled as the Krishi Vaidya bouncer dataset. The local dataset documentation records a WebDataset export with separate training and validation shards. Each sample is represented as an image and a JSON annotation record, where bounding boxes are stored in COCO-style pixel xywh format. The derived YOLO export remains the training format for the detector, while the canonical local dataset remains the source-of-truth artifact.

The final canonical dataset contains 30,981 images and 147,467 object annotations. The training split contains 27,454 images, and the validation split contains 3,527 images. The dataset covers eleven object classes: rice leaf, rice panicle, rice grain cluster, maize leaf, maize ear, potato leaf, tomato leaf, tomato fruit, brassica leaf, cauliflower head, and cabbage head.

Table 1: YOLO dataset class distribution after curation

Class	Object annotations
Rice leaf	1,657
Rice panicle	115,046
Rice grain cluster	473
Maize leaf	8,000
Maize ear	106
Potato leaf	1,199
Tomato leaf	8,792
Tomato fruit	6,808
Brassica leaf	3,834
Cauliflower head	525
Cabbage head	1,027

3.1.2 Data Sources and Class Mapping

The dataset was built from multiple Kaggle and Roboflow sources. The source list includes tomato leaf data from Kaggle, rice panicle data from Kaggle and Roboflow, rice leaf data from Kaggle, rice grain detection data from Roboflow, potato leaf data from Roboflow, maize leaf and maize ear data from Roboflow, tomato fruit data from Roboflow, brassica leaf and cauliflower data from Roboflow, and cabbage data from Roboflow. These sources were mapped into a single class taxonomy instead of being treated as unrelated datasets.

The materialization report shows that 34,992 candidate images were processed. During curation, 4,011 images and 13,218 objects were removed. The removed objects included annotations that were out of bounds, non-positive after coordinate correction, collapsed after clamping, too small in pixel area, too small in relative area, or truncated after clamping. This curation stage was necessary because the combined dataset came from independent public sources with different annotation conventions and quality levels.

The final class distribution is highly imbalanced. Rice panicle dominates the annotation count, while maize ear, rice grain cluster, cauliflower head, and cabbage head have far fewer examples. This imbalance is important for interpreting the validation results because class-level behavior cannot be understood only from aggregate precision, recall, or mAP values.

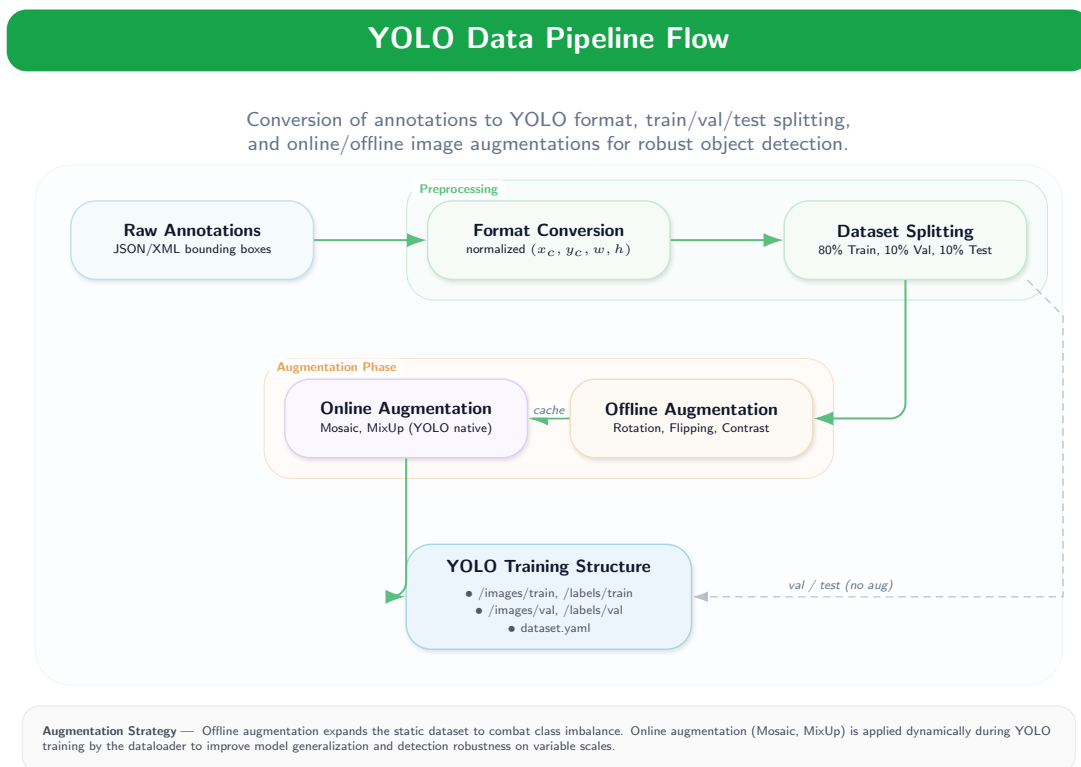


Figure 2: YOLO data pipeline flow from raw annotations to canonical export, splitting, and training structure.

3.2 YOLO Object-Detection Method

3.2.1 Model and Training Setup

The detector was trained using a YOLOv8 nano base model, identified in the training metadata as yolov8n.pt. The training task was object detection, and the input image

size was 640 pixels. The run was configured for 100 epochs with validation enabled. The metadata records the training device as GPU device 0, automatic optimizer selection, automatic mixed precision, deterministic execution enabled, and a random seed of 0.

The training configuration used standard detection losses: box regression loss, classification loss, and Distribution Focal Loss (DFL). The configured loss gains were 7.5 for box loss, 0.5 for classification loss, and 1.5 for DFL. The optimizer was selected automatically by the training framework, with an initial learning rate of 0.01, final learning-rate factor of 0.01, momentum of 0.937, and weight decay of 0.0005. Warmup was configured for 3 epochs.

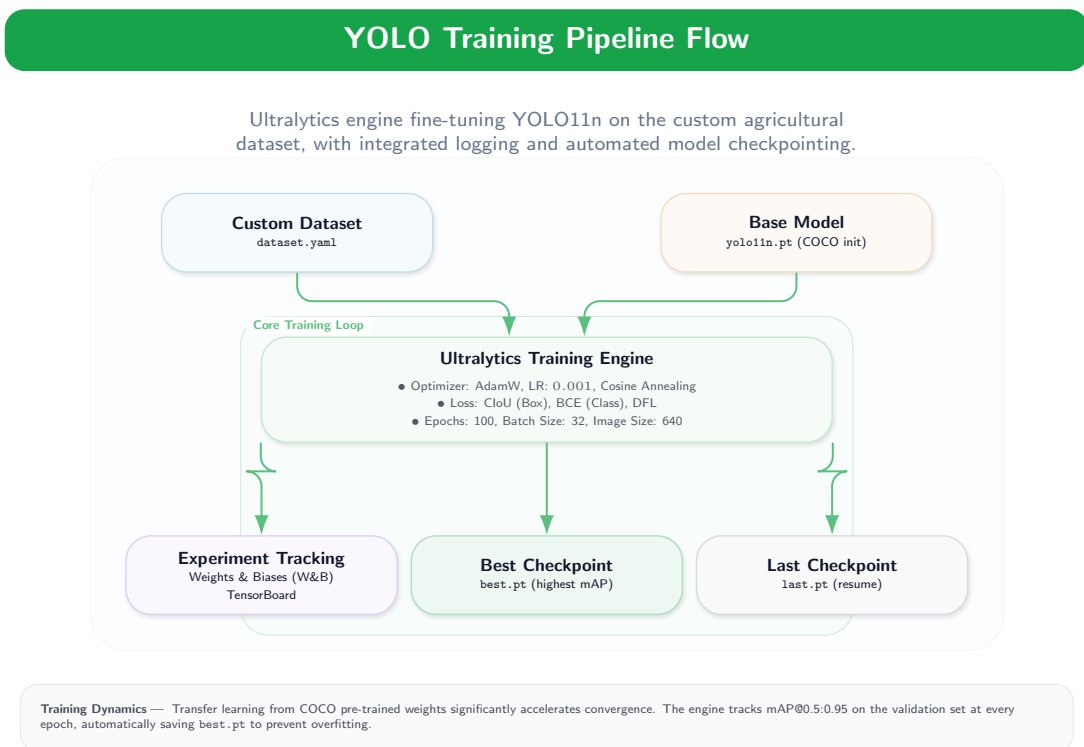


Figure 3: YOLO training pipeline flow from dataset preparation through training configuration, optimization, and checkpoint selection.

Image augmentation was enabled through common YOLO training transformations. The run used mosaic augmentation with probability 1.0, horizontal flipping with probability 0.5, HSV augmentation, translation, scaling, and automatic augmentation. Mixup and copy-paste augmentation were not used. Mosaic augmentation was closed during the final 10 epochs to reduce late-stage training noise.

Table 2: YOLO training configuration used for the Krishi Vaidya detector

Parameter	Value
Task	Object detection
Base model	yolo8n.pt
Epochs	100
Image size	640
Validation	Enabled
Optimizer	Auto-selected
Initial learning rate	0.01
Final learning-rate factor	0.01
Momentum	0.937
Weight decay	0.0005
Warmup epochs	3.0
Box loss gain	7.5
Classification loss gain	0.5
DFL gain	1.5
Mosaic augmentation	1.0
Horizontal flip probability	0.5
Close mosaic	Last 10 epochs

3.2.2 Evaluation Metrics

The detector was evaluated using precision, recall, F1-score, mAP@50, and mAP@50-95. Precision measures the fraction of predicted detections that are correct. Recall measures the fraction of ground-truth objects that are detected. F1-score summarizes the balance between precision and recall. mAP@50 evaluates mean average precision at an IoU threshold of 0.50, while mAP@50-95 uses the stricter COCO-style range of IoU thresholds from 0.50 to 0.95.

These metrics were used together because a single value is insufficient for evaluating this detector. For example, a high threshold can improve precision while reducing recall, and a deployment variant can preserve aggregate F1-score while changing per-class behavior. The final analysis therefore combines aggregate metrics, threshold sensitivity, deployment efficiency, per-class F1-score, and error analysis.

3.2.3 Confidence Threshold Analysis

A confidence-threshold sweep was performed to study how prediction filtering affects precision, recall, F1-score, mAP, and fitness. This step is important for deployment because the model is not used only as an offline benchmark; it is intended to support a mobile diagnosis workflow where excessive false detections and excessive missed detections both reduce practical usefulness.

The sweep evaluated thresholds from 0.05 to 0.95. Lower thresholds preserve more detections and usually increase recall, but they also admit weaker predictions. Higher thresholds remove weak predictions and can improve precision, but they reduce recall and can remove useful detections. The selected operating point must therefore be interpreted as a tradeoff rather than as a purely mathematical maximum of one metric.

3.2.4 Deployment and Quantization Evaluation

The deployment evaluation compared the baseline PyTorch model against TensorFlow Lite variants. The available manuscript-ready results include FP32, FP16, and INT8 TFLite variants. The analysis records model size, compression ratio, device, latency, FPS, precision, recall, and F1-score.

This comparison was included because Krishi Vaidya includes a mobile application, and the detector is intended to be compatible with lightweight deployment. However, the recorded latency table was measured on `cuda:0`; it should therefore be interpreted as a controlled model-variant comparison, not as a final Android-device benchmark. A final phone benchmark should be added only after direct mobile profiling is performed.

YOLO Quantization & Export Flow

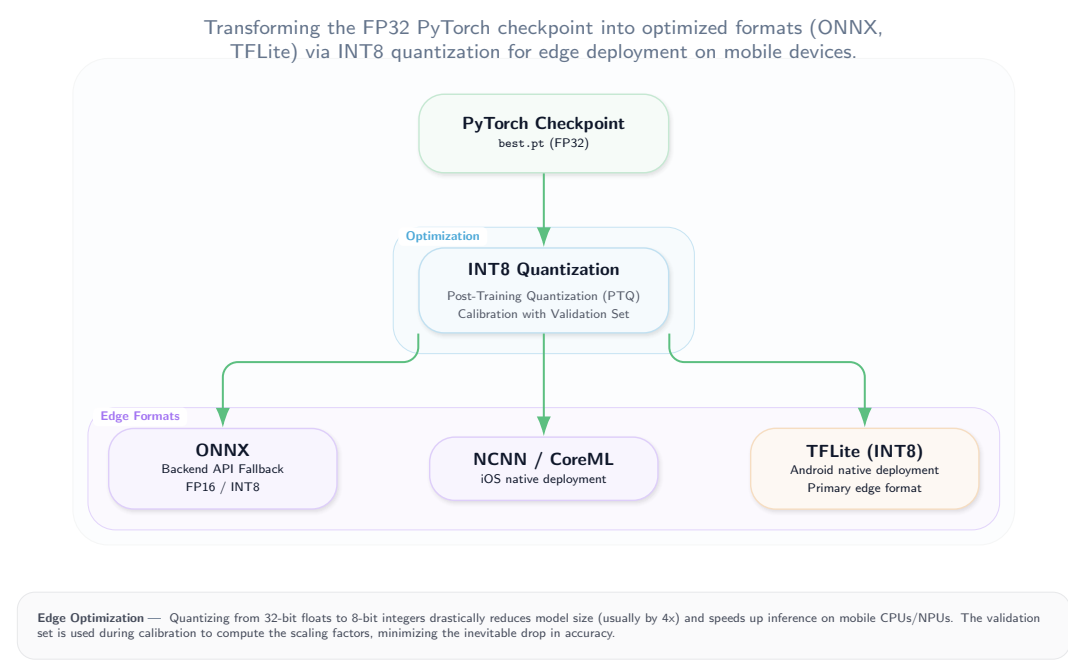


Figure 4: YOLO quantization and export flow showing the transition from trained weights to deployment-ready artifacts.

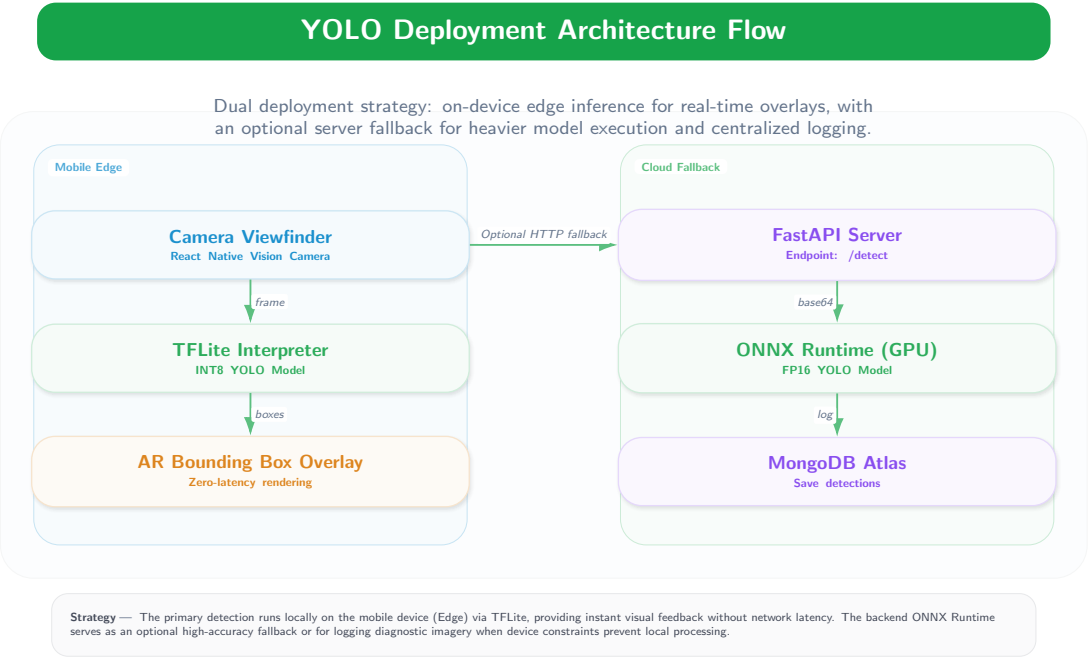


Figure 5: YOLO deployment architecture flow showing the relationship between the trained detector, export formats, and deployment targets.

3.3 VLM Disease-Diagnosis Dataset

The VLM component was prepared as the disease-classification and disease-interpretation component of Krishi Vaidya. Unlike the YOLO detector, which predicts bounding boxes for plant structures, the VLM receives a crop image and a natural-language instruction and is expected to return a canonical disease label or a compact structured answer. This design makes the VLM section distinct from the detector section: its output must be evaluated not only for classification correctness, but also for whether the generated text can be parsed into a usable downstream label.

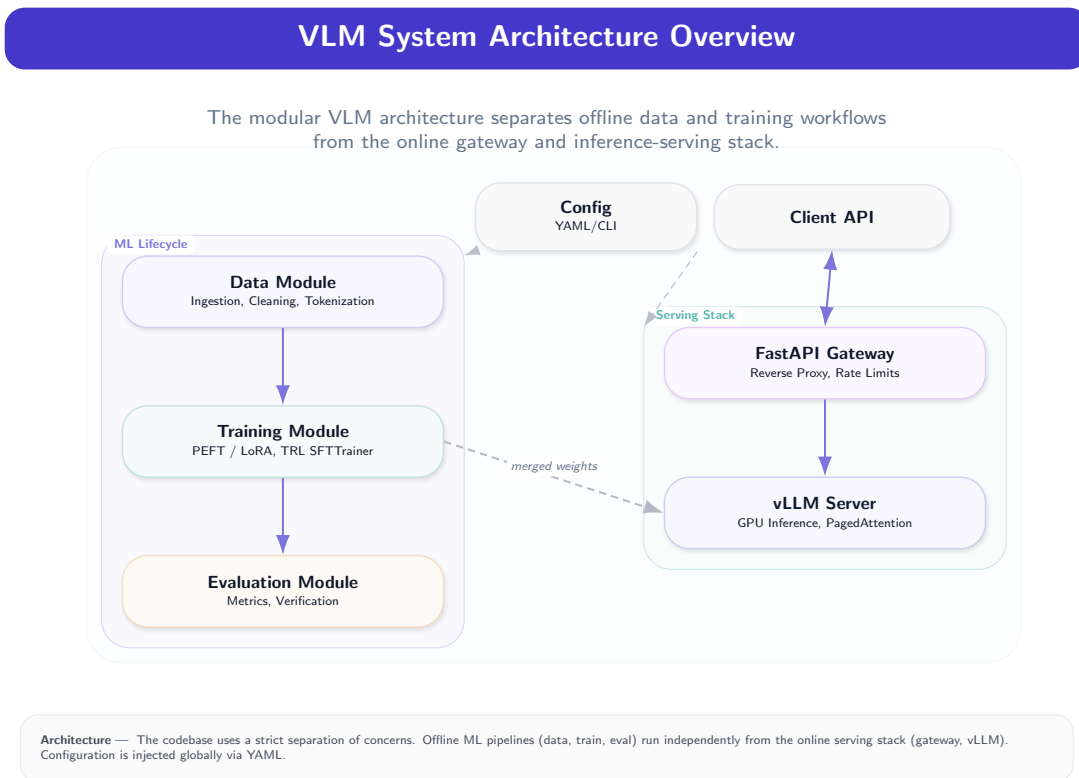


Figure 6: VLM system architecture overview separating dataset processing, training, inference, and serving infrastructure.

3.3.1 Dataset Pipeline

The VLM dataset pipeline is configured under `vlm_model/data/configs`. The project name is `krishivaidya`, and the dataset name is `kv-core5-ms`. The pipeline stages include download, scan, manifest construction, deduplication, split creation, validation, analysis, dataset-card generation, and packaging. The configured split policy reserves 15% of the data for a locked test set and 15% for validation. The curation configuration caps each selected class to avoid uncontrolled dominance by very large source categories, with per-crop caps of 800 records for rice, maize, potato, and tomato, and 600 records for

cauliflower and cabbage.

The active source configuration enables PlantVillage as a local-folder source. Additional sources are defined but disabled in the current configuration, including PlantSeg, DL-CPD25, VegNet, Irish potato, and rice multicrop sources. Because those sources are disabled, the manuscript should not claim that they were used in the reported VLM experiment. They are part of the pipeline design, but not part of the active dataset evidence used for the present results.

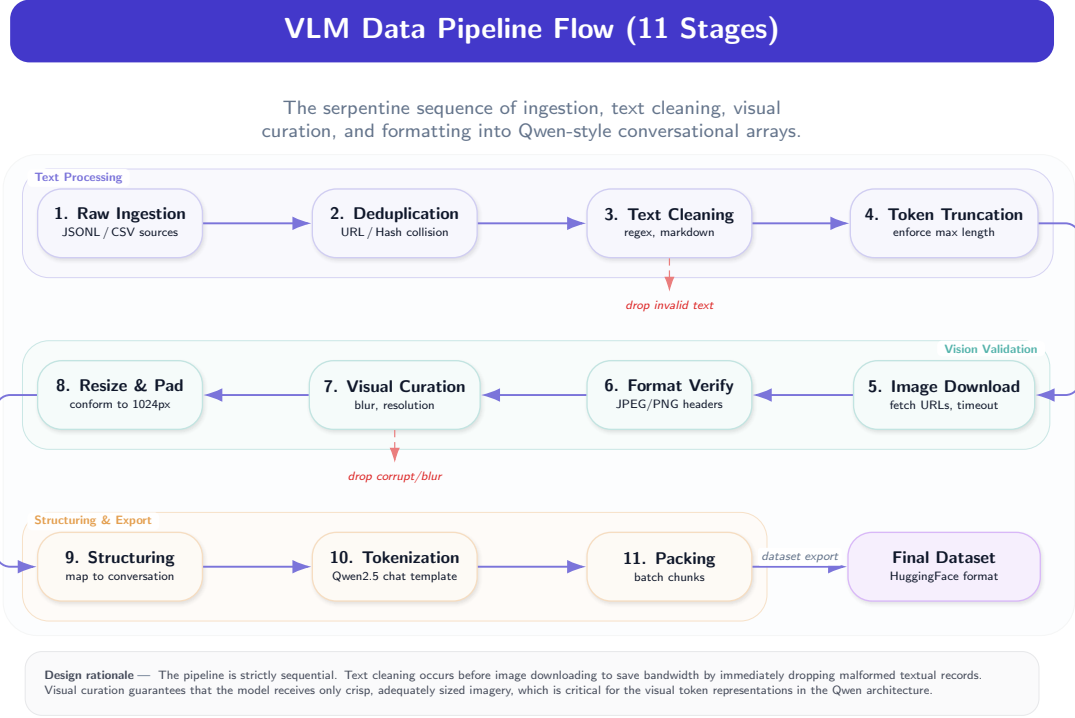


Figure 7: VLM data pipeline flow from raw sources through curation, canonicalization, export, and training data assembly.

3.3.2 Taxonomy and Label Normalization

The VLM taxonomy defines crop-specific disease labels and maps aliases into canonical labels. The configured taxonomy includes rice, maize, potato, tomato, cauliflower, and cabbage, but the reported VLM training and prediction outputs contain a subset of those taxonomy classes. The manuscript-ready class-distribution table reports eleven Qwen records classes: maize common rust, maize healthy, maize northern leaf blight, potato early blight, potato healthy, potato late blight, tomato early blight, tomato healthy, tomato late blight, tomato leaf mold, and tomato septoria leaf spot.

The active label representation uses canonical labels such as `maize__common_rust`, `potato__late_blight`, and `tomato__septoria_leaf_spot`. This canonical format

is important because the evaluation code compares predicted and expected labels using exact canonical-label matching.

3.3.3 Qwen2.5-VL SFT Export

The curated dataset is exported into Qwen2.5-VL supervised fine-tuning records. Each record contains an image path, a human message containing the image token and instruction, an assistant answer, and metadata fields such as split, crop name, disease name, canonical label, template id, and source name.

Prompt templates are selected deterministically from the sample identifier. Three templates are available. The `disease_json` template asks the model to return compact JSON with the keys `crop`, `disease`, `canonical_label`, and `confidence_label`. The `label_only` template asks for the canonical disease label and confidence label. The `diagnostic_short` template asks for a disease label, canonical label, confidence label, and brief visual diagnosis. This template mixture is useful for training the model on multiple answer forms, but it also increases the importance of robust output parsing during evaluation.

3.4 VLM Fine-Tuning Method

3.4.1 Base Model and Adapter Design

The VLM was fine-tuned from Qwen/Qwen2.5-VL-3B-Instruct, a member of the Qwen vision-language model family (Bai et al., 2025). The training recipe uses LoRA adapters instead of full model fine-tuning, following the parameter-efficient adaptation approach introduced by Hu et al. (2021). The LoRA rank is 16, the LoRA alpha is 32, and dropout is 0.05. The target modules include attention projection layers and feed-forward projection layers: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. The configuration excludes module names containing `visual`, `vision`, and `merger`, which keeps the adapter targeting focused away from the explicitly excluded visual modules.

3.4.2 Quantized Training Configuration

The training recipe uses 4-bit quantization with NF4 quantization type, double quantization, and bfloat16 compute dtype. The model is loaded with bfloat16 dtype, automatic device mapping, and flash-attention support in the resolved runtime. The maximum sequence length is 4096 tokens. The image processing limits are configured with minimum

pixels of 200,704 and maximum pixels of 451,584.

Training was configured for 3 epochs, per-device train batch size of 4, per-device evaluation batch size of 4, gradient accumulation of 4, learning rate of 2.0×10^{-5} , cosine learning-rate scheduling, weight decay of 0.01, maximum gradient norm of 1.0, and paged_adamw_8bit optimization. The resolved run used bfloat16 and TF32, enabled the Liger kernel, disabled torch.compile, and used an NVIDIA L4 GPU with 22.03 GB memory.

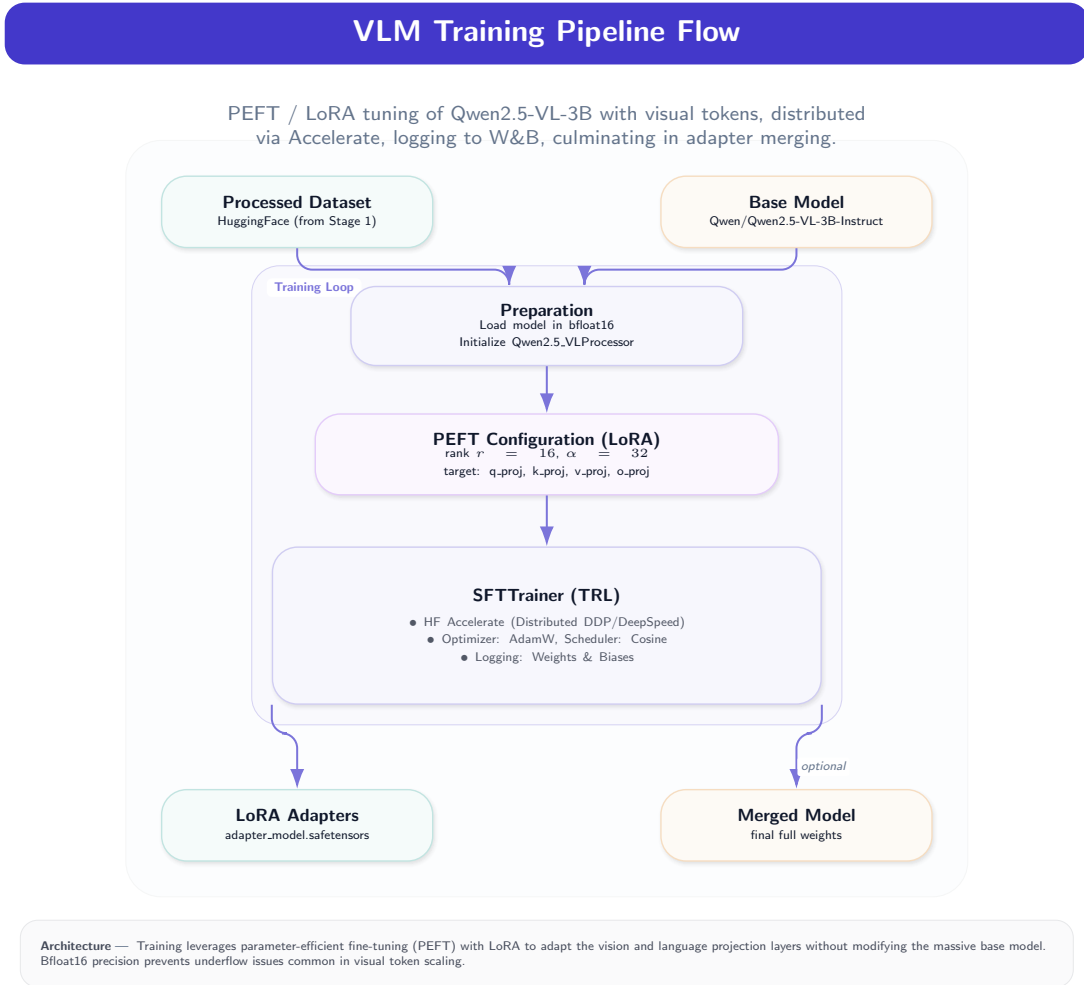


Figure 8: VLM training pipeline flow from curated dataset export through quantized fine-tuning and checkpoint generation.

Table 3: VLM fine-tuning configuration

Parameter	Value
Base model	Qwen/Qwen2.5-VL-3B-Instruct
Training method	LoRA adapter fine-tuning
Quantization	4-bit NF4 with double quantization
Compute dtype	bfloat16
LoRA rank / alpha	16 / 32
LoRA dropout	0.05
Epochs	3
Learning rate	2.0×10^{-5}
Train batch / accumulation	4 / 4
Effective batch	16 samples
Optimizer	paged_adamw_8bit
Scheduler	Cosine
Max sequence length	4096
Attention backend	flash_attention_2
GPU	NVIDIA L4

3.4.3 Evaluation and Prediction Parsing

The prediction evaluation uses the test split from the Qwen export. For each record, the inference request contains the image and the user prompt extracted from the first conversation message. The expected label is resolved from record metadata or from the assistant target answer. The generated prediction text is parsed as JSON, or as a JSON block embedded inside generated text. If a JSON object is found and contains `canonical_label`, that value becomes the predicted label. Otherwise, the prediction is assigned the special parse-error label `__parse_error__`.

This evaluation design is deliberately strict. In a backend or mobile workflow, a generated sentence that contains useful words but cannot be converted into a canonical label is not directly usable by downstream code. Therefore, parseability is treated as part of model performance rather than as a cosmetic formatting issue.

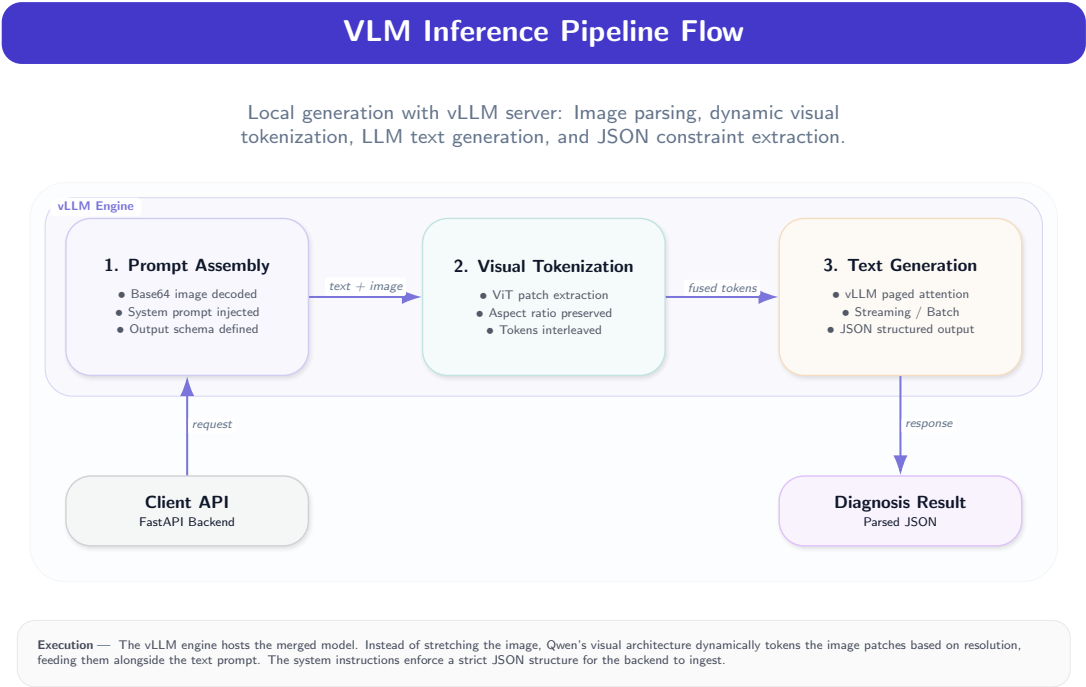


Figure 9: VLM inference pipeline flow showing request preparation, generation, structured parsing, and fallback handling.

3.5 Backend System Architecture

The backend of Krishi Vaidya was implemented as a TypeScript and Express API named `krishi-vaidya-api`. Its role is to connect the mobile application, persistent farm records, image storage, authentication, diagnosis inference, and advisory generation into one server-side interface. The backend is not only a request router; it contains explicit layers for configuration, middleware, routes, controllers, services, repositories, data models, and external integrations. Express routing and middleware concepts provide the basis for organizing endpoint handlers and request-processing controls (Express.js, 2026a, 2026b). This layered structure was used so that HTTP concerns, business rules, database access, and third-party service calls remain separated.

Figure 10 summarizes the verified backend architecture. The figure was generated from the implemented source structure rather than from a hypothetical design. The mobile application communicates with the Express API through JSON and multipart requests. The Express app applies common middleware before dispatching requests to route groups. Controllers validate request parameters and request bodies, services apply workflow-level rules, repositories isolate Mongoose queries, and models define the persisted MongoDB documents. External services are used for image storage, VLM inference, and advisory generation.

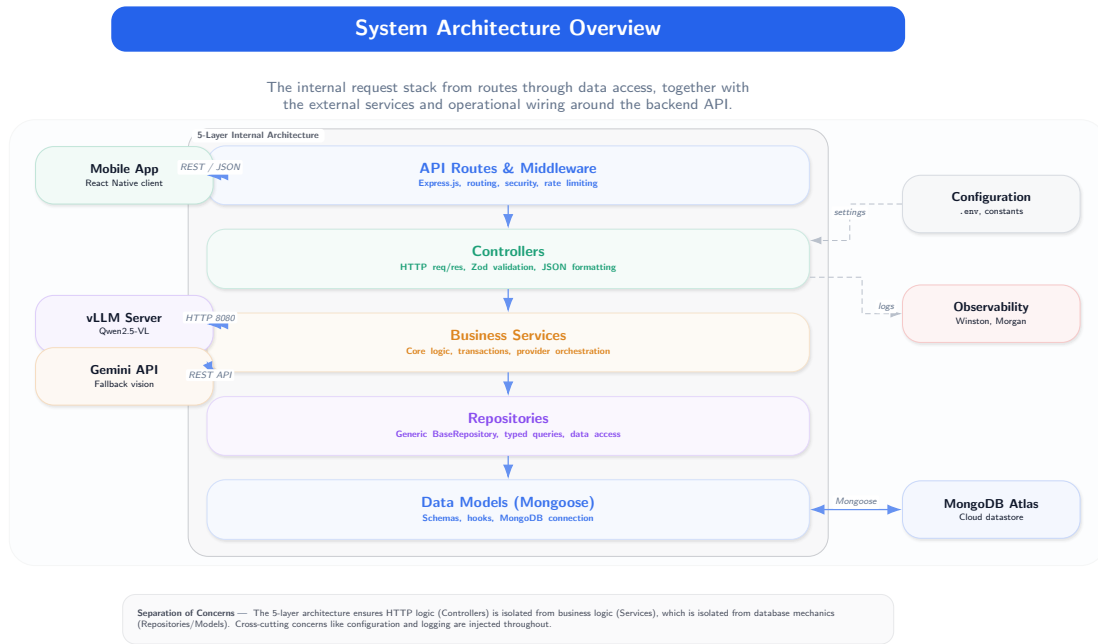


Figure 10: Backend system architecture overview showing the request stack and external service boundaries.

3.5.1 Runtime Modes and Application Initialization

The Express application is created in `src/app.ts`. During initialization, the server sets trust-proxy support, applies security middleware, applies CORS, applies the general rate limiter, enables JSON and URL-encoded parsing with a 10 MB limit, and registers HTTP request logging. A health endpoint is exposed at `/api/v1/health`; this endpoint reports server state such as timestamp, uptime, environment, backend mode, server exposure mode, and the public base URL.

The backend supports three runtime modes through the validated `BACKEND_MODE` environment variable. In `diagnose` mode, only the diagnosis route is enabled. In `non_ai` mode, only database-backed routes are enabled. In `complete` mode, both the diagnosis route and database-backed routes are enabled. This design is useful for development and deployment because the AI diagnosis stack can be run separately from the full user-management and farm-management stack when required.

Table 4: Backend runtime modes and route availability

Mode	Diagnosis route	Database-backed routes
<code>diagnose</code>	Enabled	Disabled
<code>non_ai</code>	Disabled	Enabled
<code>complete</code>	Enabled	Enabled

Configuration is validated with Zod before the server starts. The environment schema defines server, MongoDB, Redis, JWT, Cloudinary, SMTP email, weather, ML-service, Google AI, and ngrok settings. Required configuration depends on runtime mode: database-backed routes require JWT, Cloudinary, and SMTP credentials, while the diagnosis stack requires the ML service URL and either a bearer token or JWT private-key configuration when ML inference is enabled. This validation prevents the server from silently starting in an incomplete state.

3.5.2 Dependency Container

The backend constructs its dependencies in `src/container.ts`. Repositories are created first, followed by services, controllers, and route modules. The container wires the authentication service to the user, OTP, refresh-token, email, and user services; the scan service to scan, scan-resolution, and file-upload dependencies; and the diagnosis service to validation, VLM inference, and Google AI advisory clients. This explicit construction makes the dependency graph visible and avoids scattering object creation across route files.

3.6 Backend API Design

3.6.1 Route Groups

The backend exposes versioned API routes under `/api/v1`. The implemented route groups are authentication, users, scans, crops, tasks, notes, advisories, and diagnosis. The development environment also serves OpenAPI documentation through Swagger UI at `/api/docs` and the generated specification at `/api/docs.json`. Swagger comments in the route files document request bodies, security requirements, path parameters, query parameters, and expected response purposes.

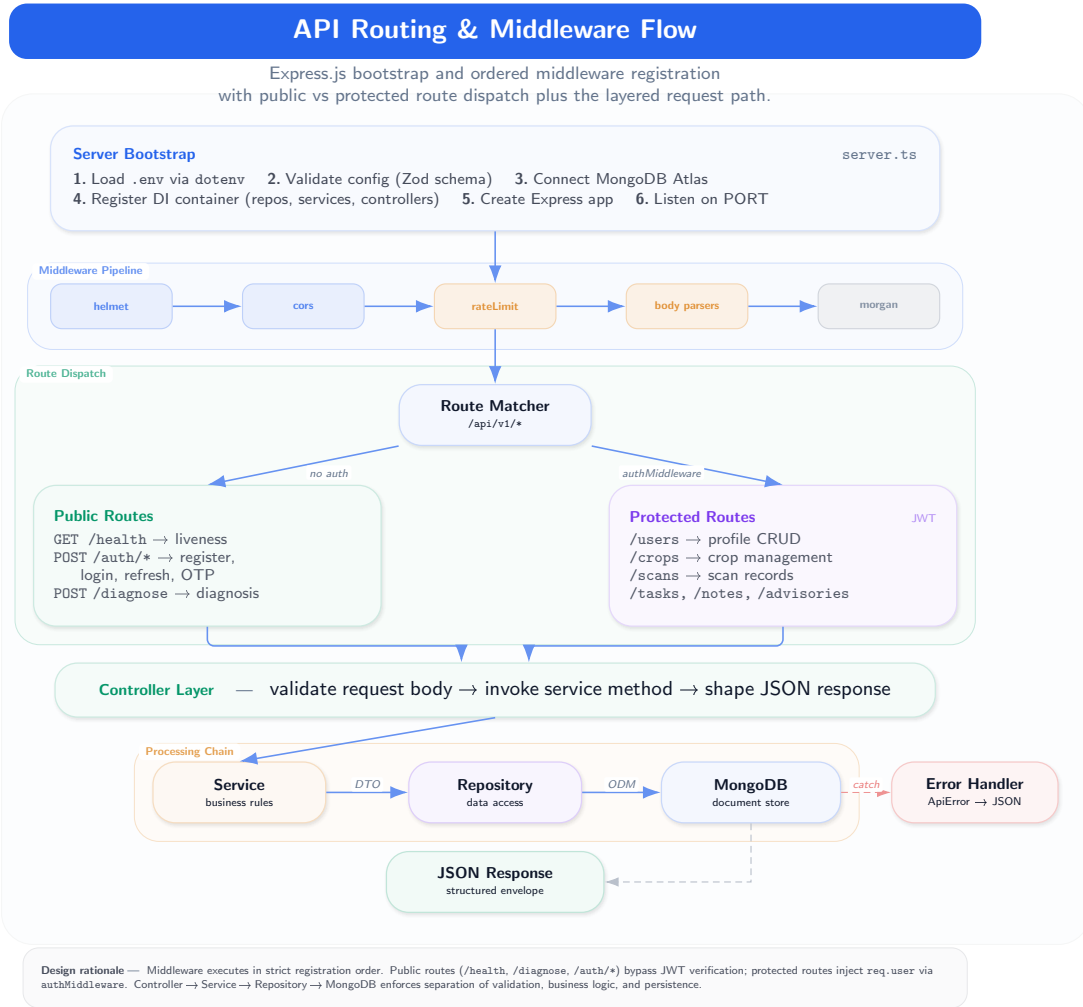


Figure 11: Backend API routing flow showing runtime mode selection, route-group registration, and request dispatch boundaries.

Table 5: Implemented backend route groups

Route group	Base path	Implemented responsibility
Health	/api/v1/health	Server status, runtime mode, uptime, and exposure information
Diagnosis	/api/v1/diagnose	Image validation, VLM inference request, advisory generation, diagnosis response
Authentication	/api/v1/auth	Registration, login, token refresh, logout, OTP, password reset
Users	/api/v1/users	Profile retrieval, profile update, profile image, password change, push tokens
Scans	/api/v1/scans	Scan upload, scan history, statistics, grouping, status, resolution, follow-up linkage
Crops	/api/v1/crops	Crop-plot creation, filtering, archive state, growth stage, health status
Tasks	/api/v1/tasks	Crop-task creation, batch creation, pending/overdue/upcoming views, completion state
Notes	/api/v1/notes	Crop notes, linked scan notes, note images, note-type counts
Advisories	/api/v1/advisories	Advisory creation, filtering, active advisories, dismiss/undismiss actions

3.6.2 Controller and Validation Pattern

Controllers receive typed service dependencies and expose Express request handlers. Validation is implemented with `express-validator` in the route/controller layer. Common

validation examples include MongoDB identifier checks for resource identifiers, bounded pagination values, enumerated values for task categories, advisory severities, scan statuses, and crop stages, and length limits for user text fields. The controllers format successful responses through a common response formatter and pass operational failures to a global error middleware.

Most database-backed route groups require authentication. The authentication middleware checks a Bearer access token, verifies the JWT with the configured secret, rejects expired or malformed tokens, rejects pending-verification and blocked users, and attaches the decoded user context to the Express request. This means service-layer methods can consistently receive the authenticated user identifier and enforce ownership through repository queries.

3.7 Backend Data Model

3.7.1 MongoDB Collections and Domain Entities

The backend uses Mongoose models for persistent state. The implemented domain entities include users, OTP records, refresh tokens, scans, scan resolutions, crop plots, crop tasks, crop notes, and advisories. The models include indexes for common access paths such as user ownership, status-based filtering, soft-delete filtering, date ordering, crop type, health status, task due date, advisory severity, and TTL cleanup for expiring documents.

Table 6: Implemented backend data models

Model	Primary purpose	Important implementation details
User	User account and preferences	Email, password hash, name, language, region, accessibility, location, status, push tokens
Otp	OTP verification state	Purpose-specific OTP hash, attempt count, expiry, unique email-purpose index, TTL expiry
RefreshToken	Session refresh state	Token hash, lookup ID, device context, revocation state, expiry, TTL cleanup
Scan	Disease scan history	Image URI, crop metadata, status, result object, client timestamp, offline flag, soft delete
ScanResolution	Outcome tracking for scans	Resolution type, treatment used, notes, follow-up scan references
CropPlot	Farmer crop plot record	Crop type, variety, region, district, area, planting date, stage, health status, archive state
CropTask	Farm task and reminder record	Plot link, category, due date, priority, completion state, auto-generated flag
CropNote	Crop observation and treatment note	Plot link, optional scan link, note type, title/body, image attachments, soft delete
Advisory	Crop advisory notification	Plot link, advisory type, severity, Nepali/English fields, dismiss state, expiry TTL

3.7.2 Repository Layer

The repository layer encapsulates database access. This is most visible in user-owned resources such as scans, crop plots, tasks, notes, and advisories. Repository methods accept user identifiers or plot identifiers and apply filters before returning results. This pattern reduces the risk of accidentally exposing one user's records to another user because ownership filtering is part of the data-access layer rather than being repeated manually in every

controller.

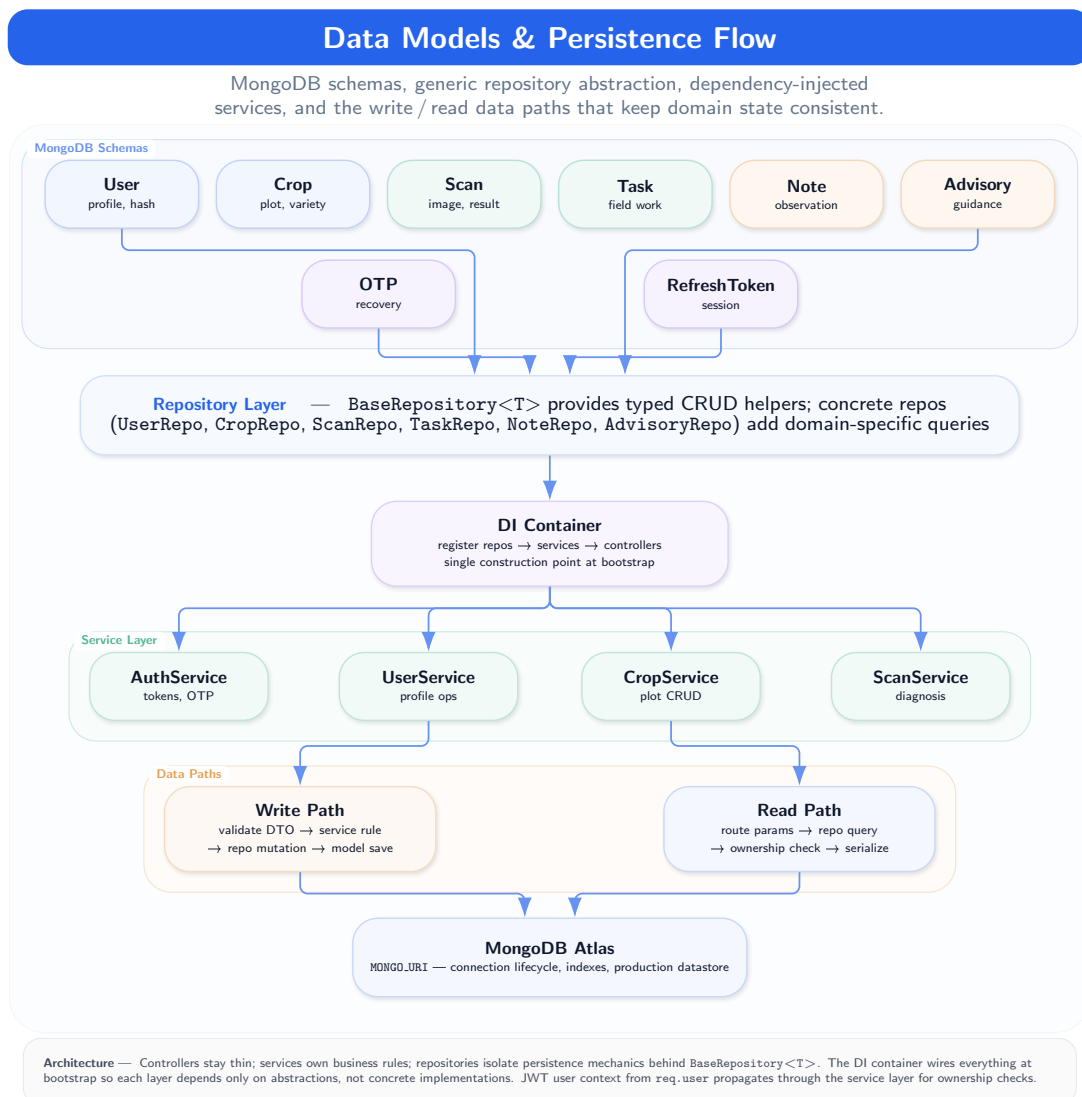


Figure 12: Backend data model and persistence flow showing the relationship between controllers, services, repositories, and MongoDB collections.

3.8 Backend Diagnosis and Advisory Pipeline

3.8.1 Image Validation

The diagnosis route accepts a single image through the multipart field image. The diagnosis validation service checks that the image exists, is non-empty, does not exceed the configured maximum upload size, and is a valid JPEG, PNG, or WEBP file. The validation does not trust the MIME type alone; it reads image headers and dimensions from the uploaded buffer. Images smaller than 64×64 pixels or larger than 4096×4096 pixels are rejected. The optional crop-name hint is also validated and limited to 80 characters.

3.8.2 VLM Inference Bridge

After validation, the diagnosis service generates a request identifier and calls the VLM inference client. Before inference, the client checks the VLM service readiness endpoint. The inference request is sent to `/v1/inference` with base64 image data, media type, a deterministic classification prompt, maximum token count of 320, temperature 0, and sampling disabled. The VLM client supports static bearer-token authentication and RS256 JWT-based bearer-token generation from configured private-key content or private-key path.

The VLM output is parsed as JSON when possible. If a JSON object is returned, the backend extracts fields such as crop, disease, scientific name, canonical label, confidence, confidence label, out-of-distribution flag, symptom descriptions, and alternatives. If the VLM output cannot be parsed into JSON, the raw text is still preserved and passed forward as part of the advisory generation context.

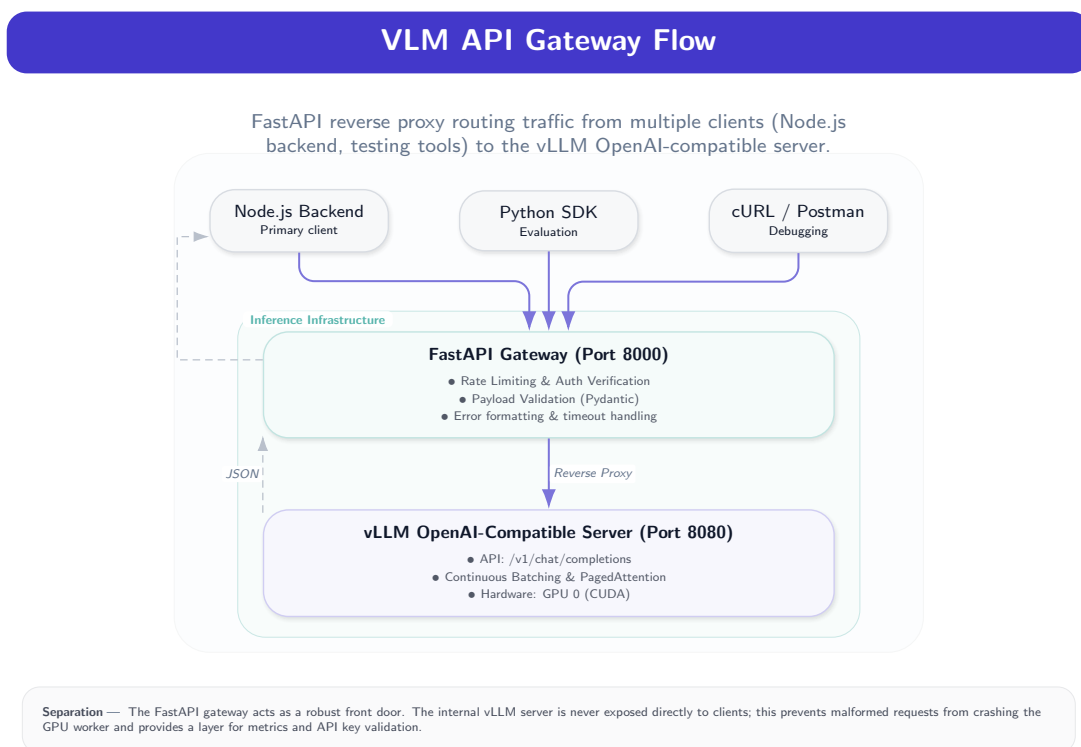


Figure 13: VLM API gateway flow showing the backend request path, readiness check, authentication, inference call, and response parsing boundary.

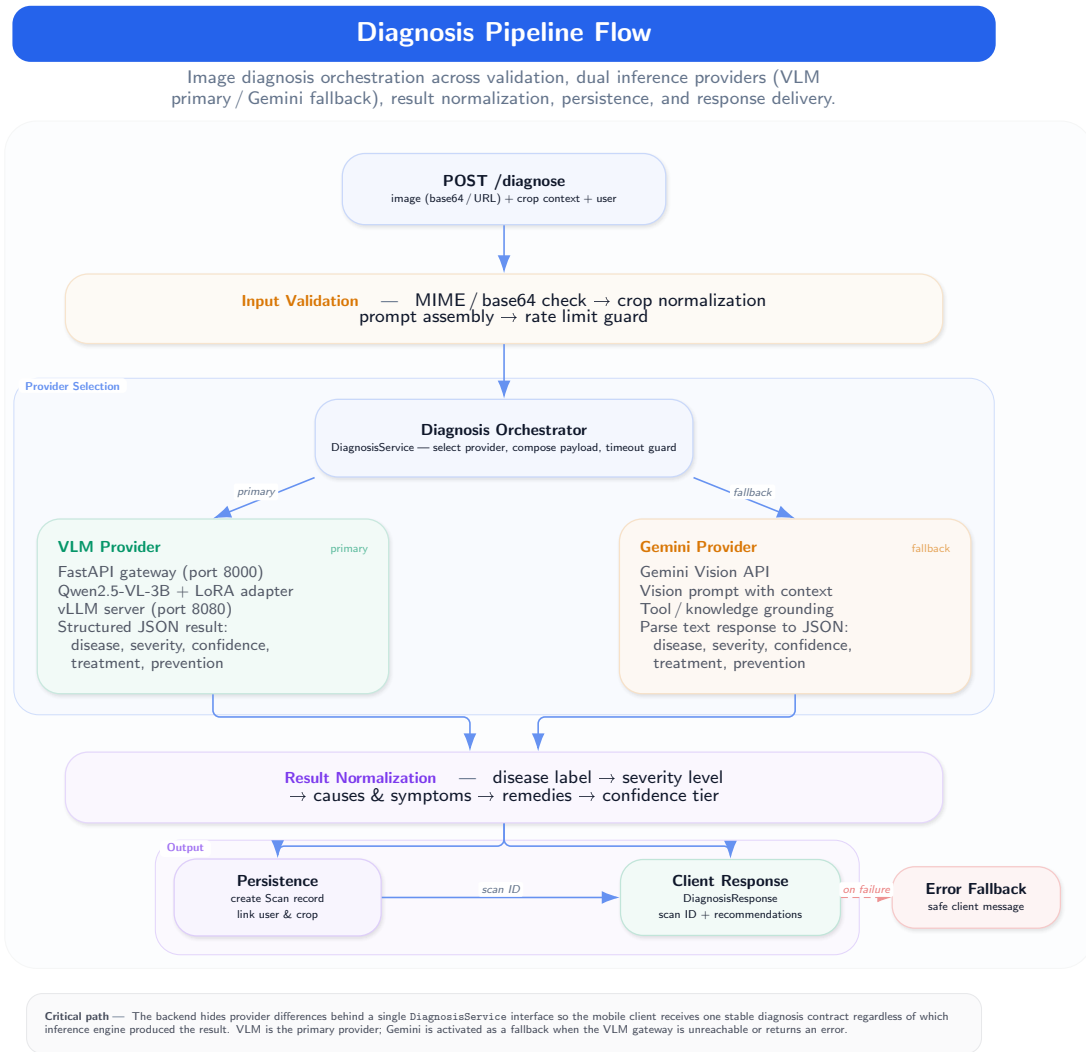


Figure 14: Backend diagnosis pipeline flow showing image validation, VLM inference bridging, advisory generation, and response assembly.

3.8.3 Google AI Advisory Bridge

The backend then calls the Google AI advisory client. This client sends the uploaded image and the parsed VLM prediction to Google AI Studio and requires the response to match a strict JSON schema. The schema includes disease name, Nepali disease name, scientific name, crop, confidence, confidence tier, out-of-distribution flag, symptom descriptions, treatment steps, prevention tips, safety warnings, organic alternatives, source URLs, and optional alternative diagnoses. The system prompt explicitly instructs the model to behave as an agricultural disease advisory expert for farmers in Nepal, to use Nepali and English fields, to avoid unsafe pesticide advice, and to avoid fabricated citations.

The advisory client can request Google Search grounding when enabled. If the selected model rejects the search tool, the client retries without grounding using the configured

fallback behavior. The client validates the returned JSON using Zod before the diagnosis service returns the final API response. The final response includes a request identifier, timestamp, disease fields, confidence tier, out-of-distribution flag, symptom descriptions, advisory object, processing time, and a model-version string composed from the VLM model and configured Google AI model.

3.9 Backend Security, Uploads, and Reliability Controls

3.9.1 Authentication and Session Handling

Authentication uses access tokens and refresh tokens. Registration creates a user account and the login flow returns an access token while storing the refresh token in an HTTP-only cookie. Refresh tokens are represented in the database as hashed token records with lookup identifiers, expiry, and revocation state. The backend also supports logout from the current session and logout from all devices.

OTP support is implemented for email verification, password reset, and phone verification purposes. OTP records store hashes rather than plain OTP values and include expiry and attempt tracking. The OTP collection has a unique email-purpose index and an expiry index, so each active OTP purpose for an email has a controlled lifecycle.

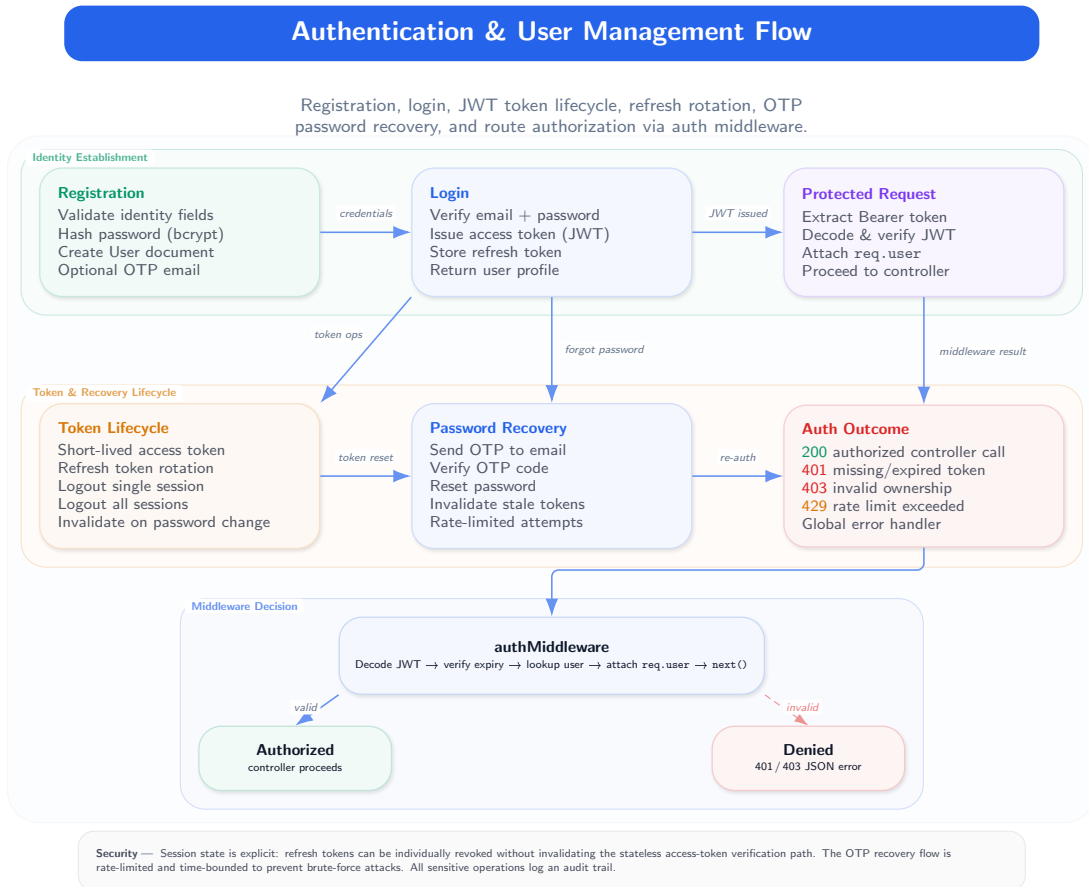


Figure 15: Backend authentication and user-management flow showing registration, login, token refresh, OTP handling, and session control.

3.9.2 Upload Handling and Cloudinary Storage

The backend uses Multer memory storage for uploaded files. File filtering checks both MIME type and extension against JPEG, PNG, and WEBP image formats. Scan images, profile pictures, and note images are uploaded to Cloudinary through the file-upload service. The Cloudinary configuration applies secure URLs and automatic quality/format transformations. The backend also supports deleting Cloudinary assets when profile pictures or note images are replaced or removed.

3.9.3 Request Safety and Error Handling

Request safety is handled through Helmet, CORS, rate limiting, validation, structured logging, and a global error handler. The general rate limiter allows 100 requests per 15 minutes, authentication routes allow 10 requests per 15 minutes, and OTP requests allow 5 requests per hour. The global error handler converts known application errors, Multer upload errors, Mongoose validation errors, duplicate-key errors, and JWT errors into con-

sistent JSON error responses. In production, unknown errors are hidden behind a generic internal-server-error message, while development mode exposes more diagnostic detail.

3.10 Mobile Application Architecture

3.10.1 Purpose and Implementation Scope

The mobile application is the farmer-facing interface of Krishi Vaidya. It is implemented as an Expo React Native application under `mobile_app/app`. Its role is broader than image capture alone: it provides onboarding, crop-plot management, scan capture, image review, diagnosis submission, result presentation, local scan history, reminders, notifications, settings, data export, and offline-aware operation. The application therefore acts as the field interface that connects the trained model work and the backend API to a practical user workflow. React Native and Expo support this cross-platform mobile implementation model (Expo, 2026; Meta Open Source, 2026).

The implemented mobile project uses Expo Router as its routing layer, React Native as the runtime, TanStack Query for server-state mutation handling, SQLite for local persistence, Zustand-style stores for UI-facing state, i18next for bilingual text, VisionCamera for native capture, and TFLite support for on-device inference assets. The application configuration declares Android package `com.krishivaidya.app`, application scheme `krishivaidya`, portrait orientation, camera and image-library permissions, typed routes, React Compiler support, and environment-driven configuration for API base URL, API key, Sentry DSN, weather API key, and feature flags.

Figure 16 shows the implemented architecture. The diagram was generated from the actual application structure rather than from an assumed design.

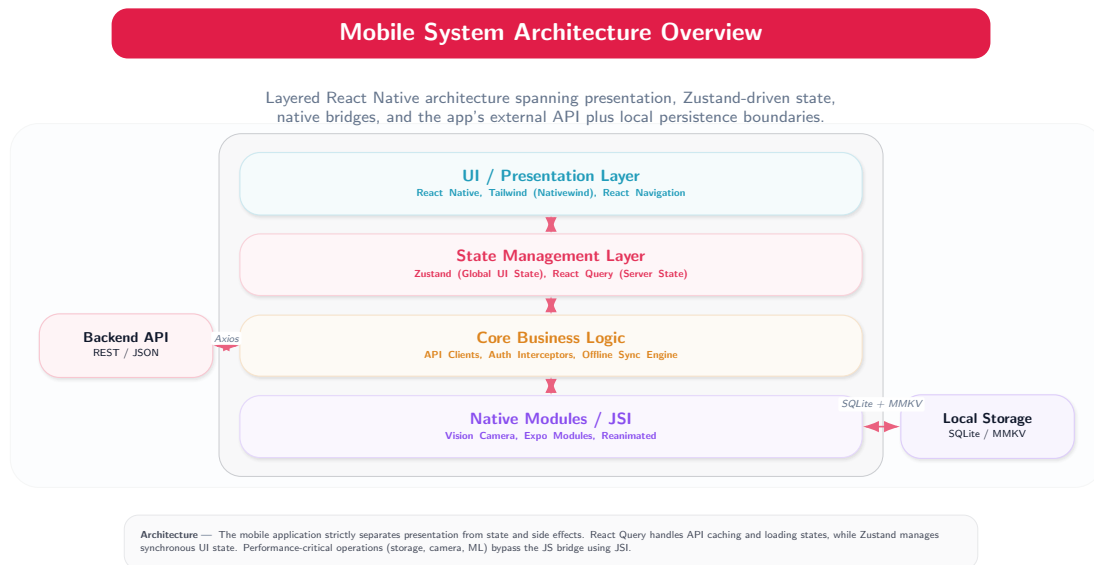


Figure 16: Mobile application architecture overview showing presentation, state, native modules, and local storage boundaries.

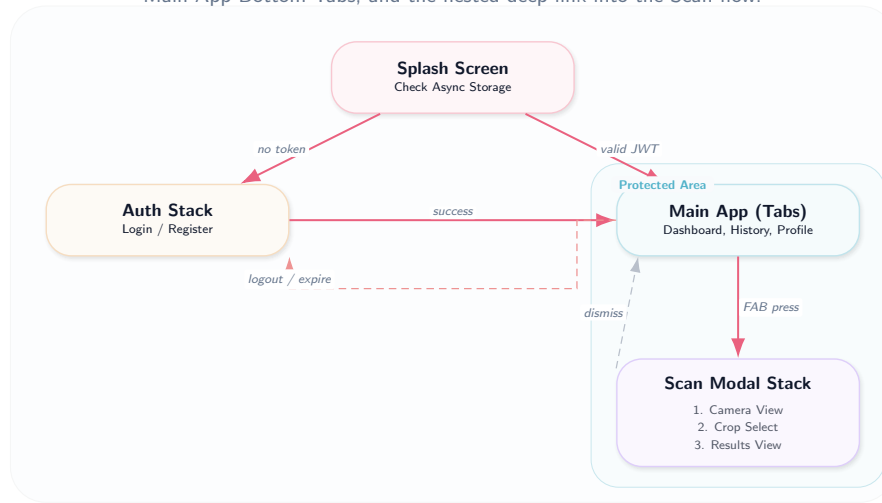
3.10.2 Application Shell and Navigation Structure

The root application layout initializes the global runtime before the user reaches feature screens. It loads fonts, controls the splash screen, creates a QueryClient with offline-tolerant defaults, initializes i18n, wraps the application in theme, error-boundary, sync-status, service, and repository providers, and bootstraps local repositories before rendering the route tree. It also refreshes local crop, app-setting, and notification stores from SQLite and preloads the TFLite service in the background. Route protection redirects users according to onboarding state, so first-time users enter the onboarding flow before the main tab interface.

The implemented route hierarchy separates the application into onboarding routes, bottom-tab routes, scan routes, crop-management routes, and settings routes. The bottom-tab navigator exposes Home, Crops, Scan, History, and Settings as the primary farmer workflows. The scan stack contains capture, preview, image-editor, analysis, and final-analysis screens. Crop-management routes include crop creation, crop detail, editing, task creation, note creation, notes, tasks, and weather. This structure keeps the scan workflow independent from longer-term farm-record management while still allowing results and reminders to connect back to crop records.

User Navigation & Routing Flow

React Navigation state machine: Switch from Splash/Auth stack to the Main App Bottom Tabs, and the nested deep link into the Scan flow.



Navigation Hierarchy — The Root Navigator uses conditional rendering to swap between the Auth Stack and the Main App Stack based on the JWT state, ensuring unauthenticated users cannot deep-link into protected tabs. The Scan Stack is presented modally over the bottom tabs to preserve context.

Figure 17: Mobile application navigation and routing flow showing onboarding, tab navigation, and feature-stack transitions.

Table 7: Implemented mobile route and workflow groups

Route group	Implemented screens	Primary purpose
Onboarding	Welcome, language, permission, crop, demo	First-run setup, language selection, permission preparation, crop context
Main tabs	Home, Crops, Scan, History, Settings	Primary application navigation for daily use
Scan stack	Capture, preview, image editor, analysis, final analysis	Image acquisition, review, detection, cloud diagnosis, report viewing
Crop routes	Add/edit crop, crop detail, crop weather, tasks, notes	Farm plot management and agronomic record keeping
Settings routes	Profile, option select, notifications, accessibility, export, model precision	Preference management, data export, accessibility, model deployment controls
Achievements	Achievements view and toast components	Lightweight engagement tracking based on completed app actions

3.10.3 Provider, Repository, and Dependency Structure

The mobile app avoids constructing all dependencies directly inside screens. The `ServiceProvider` exposes service abstractions such as camera, network, notification, haptics, voice, and location services. The `RepositoryProvider` exposes repositories for scans, crops, tasks, notes, and the data catalog. The repository bootstrap file is the central construction point for local data sources and repositories. It initializes SQLite, runs the V3 to V4 migration path, performs a one-time MMKV-to-SQLite migration, constructs local data sources, creates repository instances, and initializes the offline sync manager.

This structure is important because it separates interface code from persistence and integration details. Feature screens can call view models, view models can use repositories and services, and repositories can use SQLite or fixture-backed catalog data without forcing

the screens to know how those dependencies are built. The project also keeps read-only agricultural catalog data separate from mutable user records. Varieties, growth stages, fertilizer recommendations, disease catalog entries, and seasonal tips are stored as fixtures, while user-created scans, plots, tasks, notes, settings, achievements, and notifications are stored in SQLite.

3.10.4 Local Persistence and Offline-Aware Design

The mobile data layer is built around a SQLite schema in which user-facing records are retained locally. The implemented schema contains tables for scans, offline scan queue entries, reminders, sync queue entries, crop plots, crop tasks, crop notes, crop advisories, custom task categories, user settings, user achievements, achievement statistics, and notifications. Most mutable tables include sync-related fields such as `sync_status`, `updated_at`, and `deleted_at`. This design supports local-first behavior because user actions can be stored immediately on the device even when backend synchronization is delayed.

Table 8: Implemented mobile local persistence groups

Persistence group	Tables or stores	Supported manuscript interpretation
Scan records	<code>scans</code> , <code>offline_queue</code>	Local scan history, queued uploads, offline diagnosis fallback
Synchronization	<code>sync_queue</code> , sync columns on mutable tables	Retryable operation queue and change tracking mechanism
Crop management	<code>crop_plots</code> , <code>crop_tasks</code> , <code>crop_notes</code> , <code>crop_advisories</code>	Farmer plot, task, note, and advisory records
Personalization	<code>user_settings</code>	Language, theme, accessibility, onboarding, model precision, selected crops
Engagement and alerts	<code>user_achievements</code> , <code>user_achievement_stats</code> , <code>notifications</code> , <code>reminders</code>	App engagement state, reminder scheduling, notification history
Catalog data	JSON fixture files through catalog repository	Read-only crop, growth-stage, fertilizer, disease, and seasonal reference data

The offline sync manager registers operation handlers, monitors connectivity, resets previously interrupted processing items, listens for app-state changes, and starts a periodic sync timer with a five-minute interval. Sync queue processing uses batches of ten items and stops when connectivity is lost. The implementation already provides the synchronization mechanism, but the source code also makes clear that some generic table-push behavior is currently represented by marking local rows as synced until all matching backend endpoints are ready. Therefore, the mobile manuscript should describe the offline sync system as implemented infrastructure with partial backend push integration, not as fully proven two-way production synchronization.

Mobile Offline Sync Data Flow

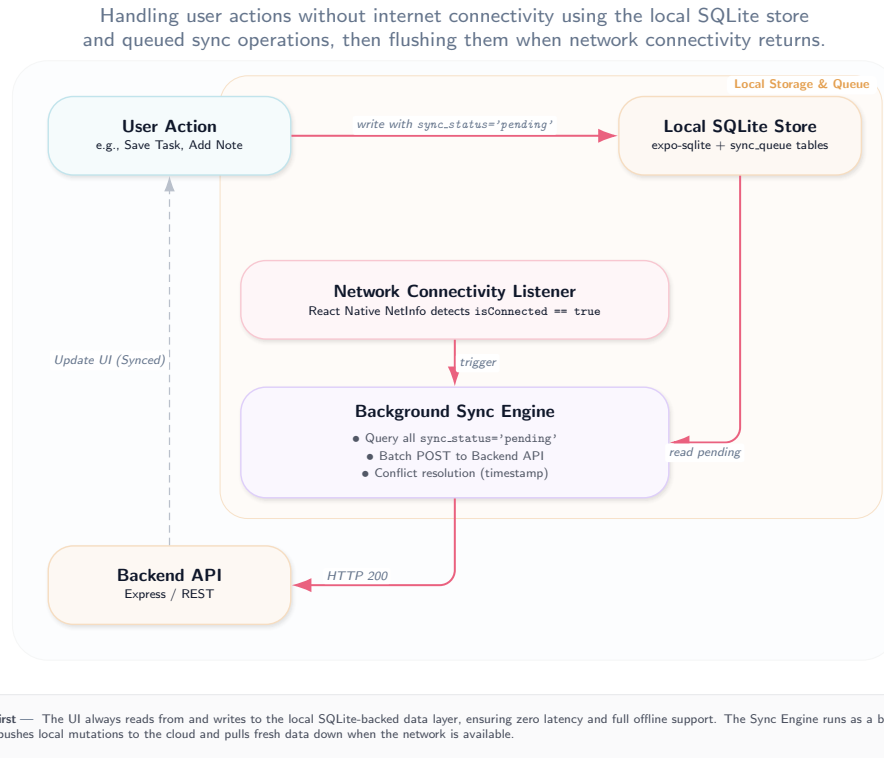


Figure 18: Mobile offline synchronization data flow showing local writes, queue management, connectivity checks, and retry behavior.

3.10.5 Scan Workflow and Diagnosis Flow

The scan workflow is implemented as a deliberate multi-stage process rather than as a single button action. The native capture screen uses VisionCamera and supports model overlay state, selected crop context, flash or torch control, focus and exposure behavior, zoom, camera switching, photo capture, and gallery selection. A web-compatible capture view exists as a fallback path. The preview stage reads image metadata and uses image-quality analysis to identify quality issues before the user proceeds. The image-editor stage allows the captured image to be refined before analysis.

The analysis view model implements an explicit state machine with phases ready, scanning, reviewed, sending, success, failed, and queued. The scanning phase uses a pre-detection passed from capture when available, a development mock only on web when enabled, or the inference service for image detection. The disease-diagnosis phase calls the network-aware diagnosis hook. When online, the hook attempts cloud diagnosis through the backend `/api/v1/diagnose` endpoint. When the connection is slow, it races the cloud request against a five-second timeout and falls back if necessary.

When offline, it queues the scan and attempts to return cached advisory content if available.

The API client posts the selected image as multipart form data to `/api/v1/diagnose` with an optional crop-name field. The client reads the base URL and optional API key from Expo configuration, applies a bearer authorization header when configured, uses a 60-second diagnosis timeout, logs requests and responses during development, and retries transient failures such as timeouts, rate-limit responses, and server errors up to three times with exponential backoff.

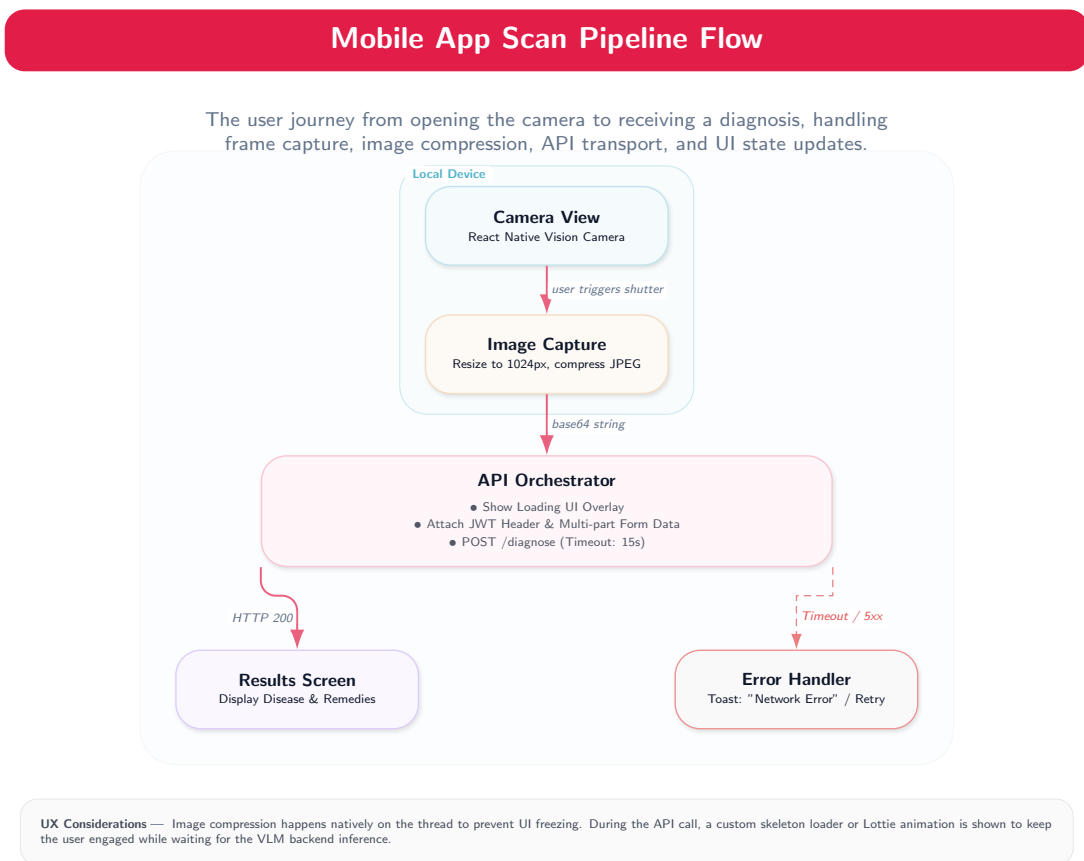


Figure 19: Mobile scan pipeline flow showing capture, preview, editing, diagnosis submission, and retry-aware request handling.

3.10.6 On-Device Model Asset Support

The mobile application includes native TFLite model support through a singleton TFLite service. The implemented model asset map includes three precision variants: `krishi_bouncer_int8.tflite`, `krishi_bouncer_fp16.tflite`, and `krishi_bouncer_fp32.tflite`. The default input configuration is 640×640 , with confidence threshold 0.35 and IoU threshold 0.45. The service reads the selected model precision from app settings, attempts to load the model with an Android GPU delegate,

and falls back to CPU loading if the GPU delegate fails. It also exposes runtime precision switching and a benchmark method that can load a chosen precision, run inference, and return load time and inference time.

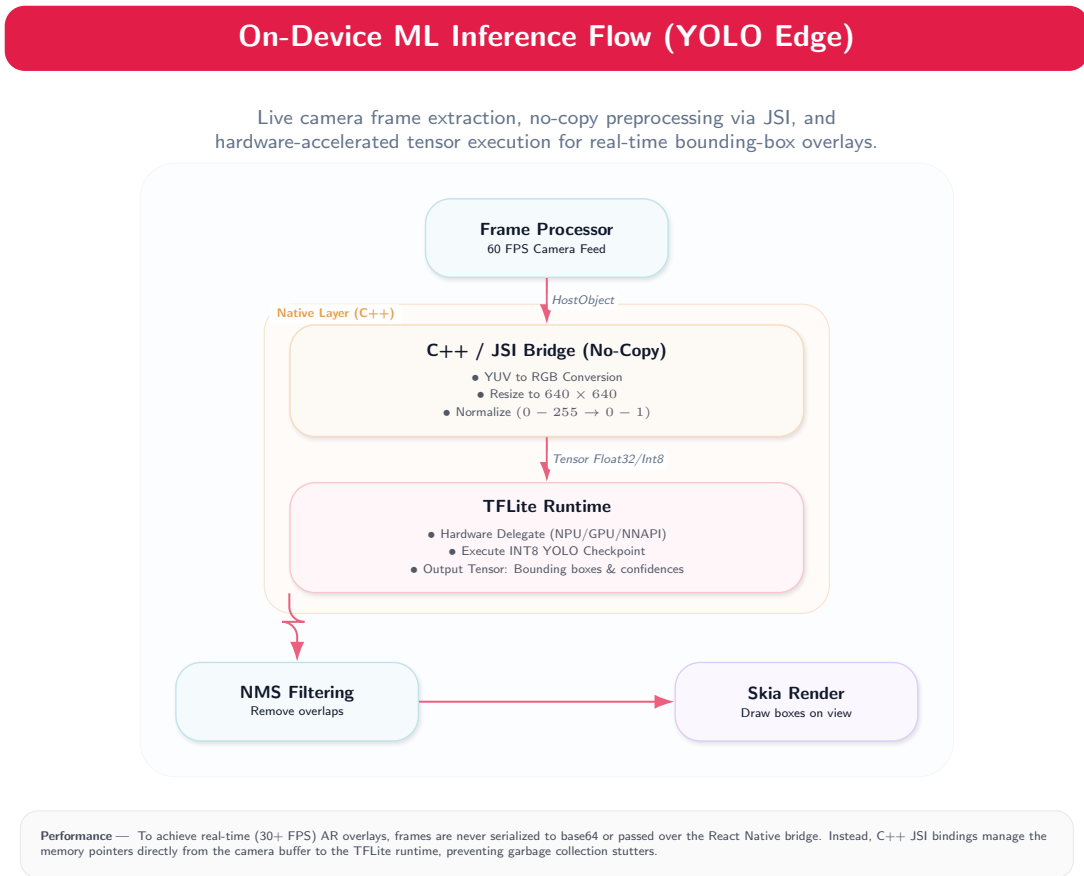


Figure 20: Mobile on-device ML inference flow showing model selection, load fallback, runtime precision switching, and benchmark support.

One integration detail must be documented carefully. The YOLO manuscript section describes the trained object detector and its class taxonomy from the YOLO training assets, while the mobile TFLite service currently defines a disease-label list in source code. The manuscript should not claim that this mobile TFLite asset has already been fully reconciled with the YOLO detector taxonomy unless that alignment is separately verified. For this reason, the mobile section treats TFLite support as implemented mobile inference infrastructure and identifies model-taxonomy alignment as an integration verification item.

3.10.7 Localization, Accessibility, and Farmer-Facing Output

The mobile application is designed as a Nepali-first interface with English support. The i18n configuration first attempts to use the persisted SQLite language preference, then device locale, and finally falls back to Nepali. The settings system includes language,

region, theme, Wi-Fi-only upload preference, haptics, reduced motion, voice output, high-contrast mode, large-text mode, color-blind mode, user profile fields, selected crops, and model precision. Result presentation includes treatment timeline support, organic alternative toggling, reminder creation, share-card generation, PDF export, voice output, feedback submission, and navigation back to report history.

This design is significant for the final-year project because it positions Krishi Vaidya as an applied agricultural decision-support system rather than only a model demonstration. The mobile app contains the workflow required to move from image capture to diagnosis, advisory presentation, follow-up actions, and longer-term crop records.

3.11 Integrated Diagnosis Workflow

3.11.1 Integration Objective

The integration layer connects the four implemented subsystems into one usable Krishi Vaidya workflow. The YOLO and mobile inference assets support image-focused pre-analysis and mobile deployment work. The VLM service provides crop-disease prediction from an image. The backend API acts as the controlled orchestration layer between the mobile client, VLM inference gateway, and advisory generator. The mobile app presents the result to the farmer and stores relevant local state. The integration method therefore focuses on how an image moves through the system, how each boundary validates or transforms the data, and how the final result becomes a farmer-facing diagnosis and advisory report.

This section is deliberately separated from the individual component sections. The component sections explain how each subsystem is built. The integration section explains how the subsystems cooperate. Figure 21 summarizes the verified end-to-end diagnosis flow from the implemented source files.

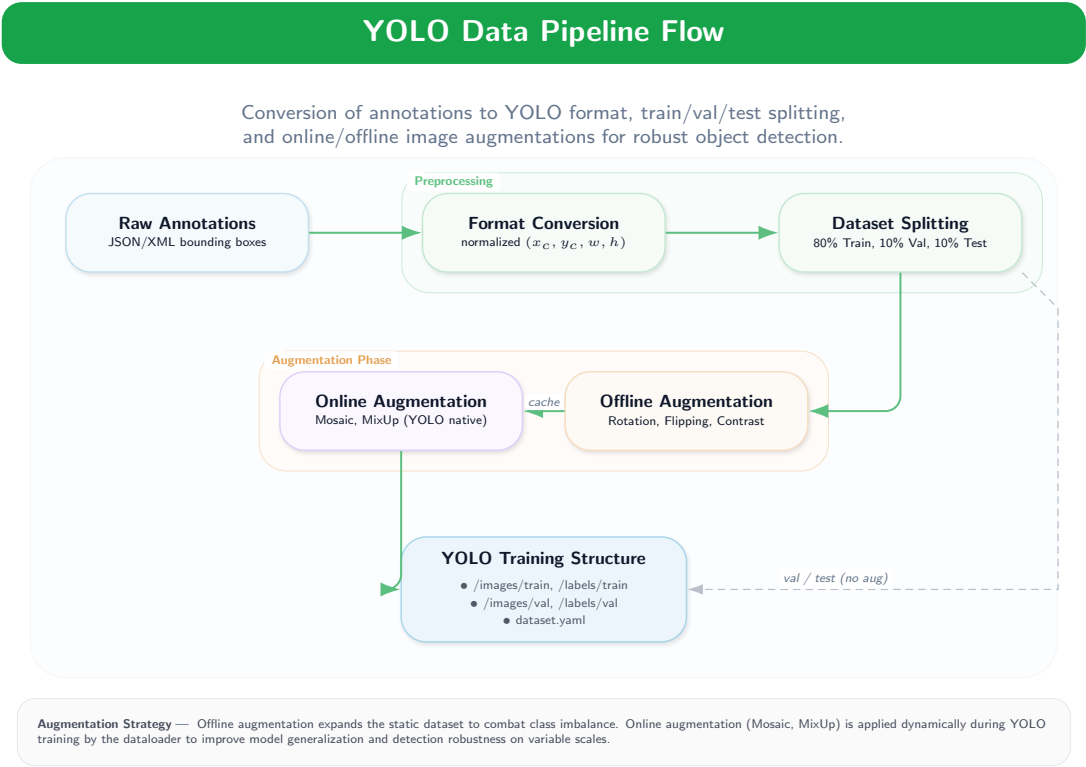


Figure 21: Combined flow-diagram overview covering the subsystem workflows used in the manuscript.

3.11.2 End-to-End Control Flow

The implemented online diagnosis path starts in the mobile scan workflow. A user captures an image with the native camera or selects an existing image from the gallery. The image passes through the preview and image-editor stages before the analysis view model enters its diagnosis phase. At this point, the network-aware diagnosis hook decides whether the request should be sent to the backend, served from cached advisory data, or queued for later synchronization.

When the device is online, the mobile API client sends a multipart request to `/api/v1/diagnose`. The image is sent in the multipart field `image`, and the app can include an optional `crop_name` field. The API client reads the backend base URL and optional API key from Expo configuration, attaches a bearer authorization header when configured, uses a 60-second timeout for diagnosis requests, and retries transient failures with exponential backoff for status codes such as 408, 429, 500, 502, 503, and 504.

The backend route receives this request through the diagnosis route group. The route is exposed as `POST /api/v1/diagnose` after the application-level route prefix is applied. The route uses single-image upload middleware and delegates request processing to the

diagnosis controller. The controller does not implement diagnosis logic itself; it passes the uploaded file and optional crop-name field into the diagnosis service and returns the service response as JSON.

Table 9: Integrated online diagnosis control flow

Step	Source component	Implementation evidence	Output
1	Mobile scan workflow	Capture, preview, editor, analysis view models	Image URI and optional crop context
2	Mobile diagnosis hook	Network-aware mutation with online, slow, offline branches	Cloud request, cached result, or queued scan
3	Mobile API client	Multipart <code>/api/v1/diagnose</code> request with retry and timeout	Backend diagnosis request
4	Backend diagnosis route	Single-image upload middleware and diagnosis controller	Validated service input
5	Backend validation	MIME, header, size, dimension, and crop-name checks	Validated image object
6	VLM client	<code>/ready</code> then <code>/v1/inference</code> with prompt and bearer/JWT auth	Raw VLM prediction text and metadata
7	Backend parser	JSON extraction with raw text preservation	Parsed VLM prediction context
8	Advisory client	Google AI request with strict schema validation	Structured diagnosis and advisory payload
9	Backend response assembly	Request ID, timing, confidence tier, model version	Application-facing JSON response
10	Mobile result workflow	Result view model, history, reminders, feedback, PDF export	Farmer-facing report and local record

3.11.3 Backend Orchestration Boundary

The backend is the central integration boundary because it protects the model and advisory services from direct mobile exposure. The diagnosis service creates a request identifier, records the processing start time, validates the optional crop name, and validates the uploaded image. The image validator rejects missing images, empty images, files larger than the configured upload limit, corrupt files, unsupported formats, MIME mismatches, images smaller than 64×64 pixels, and images larger than 4096×4096 pixels. It supports JPEG, PNG, and WEBP image formats.

After validation, the backend calls the VLM inference client. The client first checks the VLM gateway readiness endpoint `/ready`. If the VLM service is not configured, not ready, times out, or is unreachable, the backend raises a service-unavailable error rather than sending an uncontrolled partial result to the mobile app. If the VLM service is ready, the backend posts to `/v1/inference` with base64 image data, media type, deterministic generation settings, maximum token count of 320, request ID, and a compact crop-disease JSON prompt. The prompt treats the mobile crop name as a hint, not as a fact, and asks for fields such as crop, disease, Nepali disease name, scientific name, confidence, confidence label, out-of-distribution state, symptom descriptions, and alternatives.

Authentication between the backend and VLM service supports two modes. If a static bearer token is configured, the client sends that token. Otherwise, when private-key material is configured, the client generates an RS256 JWT with issuer, audience, subject, scope, expiry, and token identifier. The generated token is cached until near expiry. This matters for the integration design because it avoids placing VLM service credentials in the mobile application and keeps inference access behind the backend boundary.

3.11.4 VLM Inference Gateway Boundary

The VLM inference gateway is implemented as a FastAPI service, using a Python API framework based on standard type hints (FastAPI, 2026). It exposes a liveness endpoint `/health`, a readiness endpoint `/ready`, and the inference endpoint `/v1/inference`. Readiness succeeds only when the gateway has a vLLM client and the vLLM backend reports the expected model as reachable. Inference requests require JWT authentication and the correct inference scope. The request schema requires an image source and supports either base64 image data or a public image URL, but not both in the same request.

The gateway validates image data before forwarding it to vLLM. For the backend integration path, the image arrives as base64. The VLM gateway validates the image according to configured size, dimension, allowed-format, and visual-token limits. It then forwards validated image bytes and the prompt to the vLLM client. The response returned to the backend contains the request identifier, prediction text, latency in milliseconds, input-token count, output-token count, model identifier, and timestamp. The prediction text is normally expected to be JSON, but the backend does not trust that assumption blindly.

3.11.5 Prediction Parsing and Advisory Generation

The backend parses the VLM prediction in a defensive way. It first tries to parse the complete prediction as JSON. If that fails, it attempts to extract the first JSON object embedded inside the text. If no valid JSON object can be extracted, the backend preserves the raw VLM text. When JSON is available, the backend extracts fields such as crop, disease, disease Nepali name, scientific name, canonical label, confidence, confidence label, out-of-distribution flag, symptom descriptions, and alternatives. Numeric confidence values are clamped between 0 and 1, and alternatives are limited to at most three items.

The parsed or raw VLM output is then passed to the Google AI advisory client along with the validated image. The advisory client instructs the model to behave as an agricultural disease advisory expert for farmers in Nepal, to use Nepali and English fields, to avoid fabricated citations, and to avoid unsafe pesticide advice. The request can include Google Search grounding when enabled. If the selected model rejects the grounding tool, the client retries without grounding according to the configured fallback behavior. The response must match a Zod-validated schema before it is accepted.

Table 10: Integrated data contract across diagnosis boundaries

Boundary	Input	Validation or transformation	Output
Mobile to backend	Multipart image, optional crop name	URI check on client; file upload middleware on server	Express file and crop-name value
Backend validation	Uploaded image buffer	MIME/header/dimension/size checks; crop-name validation	Validated image and optional crop hint
Backend to VLM gateway	Base64 image, media type, prompt, request ID	Readiness check, timeout, bearer/JWT auth	VLM inference request
VLM gateway to vLLM	Validated image bytes and prompt	JWT scope, image validation, rate limit, vLLM health	Prediction text and inference metadata
Backend parser	VLM prediction text	JSON parse, embedded JSON extraction, raw-text fallback	Parsed VLM context
Backend to advisory model	Image and parsed VLM context	Strict system prompt, schema request, retry/fallback	Structured advisory payload
Backend to mobile	Diagnosis response JSON	Confidence-tier calculation, alternative filtering, model-version assembly	Farmer-facing diagnosis response
Mobile result view	Structured response JSON	Bilingual field selection, timeline mapping, local history save	Report, reminders, feedback, export paths

3.11.6 Mobile Offline and Slow-Network Integration

The mobile app includes separate behavior for online, slow, and offline states. When network status is online, it attempts the cloud diagnosis request. If that request fails, it falls back to the offline queue path. When network status is slow, it races the cloud diagnosis request against a five-second timeout and falls back if the request does not complete in time. When the device is offline, it does not attempt the backend call and instead queues the scan directly.

Queued scans are written to the SQLite-backed offline queue and also inserted into the main scan history as pending offline scans. The queue manager also enqueues a high-priority `scan_diagnosis` item in the unified sync queue. If a matching cached advisory is available, the diagnosis hook can return cached advisory content immediately while still keeping the scan queued for future cloud diagnosis. This behavior is important for field use because it prevents the app from becoming useless when network connectivity is unavailable.

3.11.7 Structured Response and Farmer-Facing Output

The backend returns a structured diagnosis response rather than exposing raw VLM text. The response contains request ID, timestamp, disease, Nepali disease name, scientific name, crop, numeric confidence, confidence tier, out-of-distribution flag, symptom descriptions, advisory details, processing time, and model version. Alternatives are included only for medium- or low-confidence responses. The confidence-tier calculation marks a response as out-of-distribution if the advisory model marks it out of distribution or if confidence is below 0.30. Confidence values of at least 0.85 are high, values from 0.60 to below 0.85 are medium, and lower non-OOD values are low.

The mobile result view consumes this structured response. It selects Nepali or English disease names according to current language, maps treatment steps into a timeline, supports organic alternative toggling, allows feedback submission, opens reminder creation, supports share-card and PDF export behavior, triggers voice output, and saves completed

scans to local history. Therefore, the integration output is not only a class label. It is a complete advisory object that can be displayed, stored, revisited, exported, and used for follow-up actions.

3.11.8 Integration Verification Boundary

The integration workflow is source-verified, but not yet fully runtime-verified in the manuscript. Source inspection confirms the route contracts, validation checks, inference gateway contract, advisory schema validation, mobile network branches, offline queueing, and result-display mapping. The available VLM API tests also cover health readiness, authentication requirement, bad-scope rejection, invalid-image rejection, successful inference response mapping, vLLM error mapping, and rate-limit behavior. However, the final thesis should still add live mobile-to-backend request examples, emulator or physical-device screenshots, measured latency, and an offline-to-online sync demonstration before making production-readiness claims.

3.12 Testing and Validation

3.12.1 Validation Strategy

Testing and validation for Krishi Vaidya were organized around the actual structure of the system. The project is not validated by one metric alone because it contains trained models, an inference gateway, a backend API, a mobile application, and an integrated diagnosis workflow. Therefore, the validation strategy separates five kinds of evidence: model-performance validation, dataset and pipeline integrity checks, automated backend/API tests, mobile source and inference checks, and integration-boundary verification.

This separation is important for accuracy. A high YOLO score does not prove that the backend is correct. A passing backend scan test does not prove that the mobile app works on a physical device. A valid VLM inference API test does not prove that the model predictions are accurate. The manuscript therefore treats each validation result according to what it actually verifies. The remaining evidence gaps are described in the subsections that follow.

3.12.2 YOLO Validation

The YOLO subsystem has the strongest quantitative detection evidence in the project. Its validation includes training dynamics, best-epoch metrics, robust validation on the valida-

tion split, threshold analysis, quantized model comparison, per-class metrics, latency measurements, confusion matrix outputs, and error-category analysis. The validation artifacts are stored under the YOLO result directories and were converted into manuscript-ready figures and tables in the YOLO manuscript data folder.

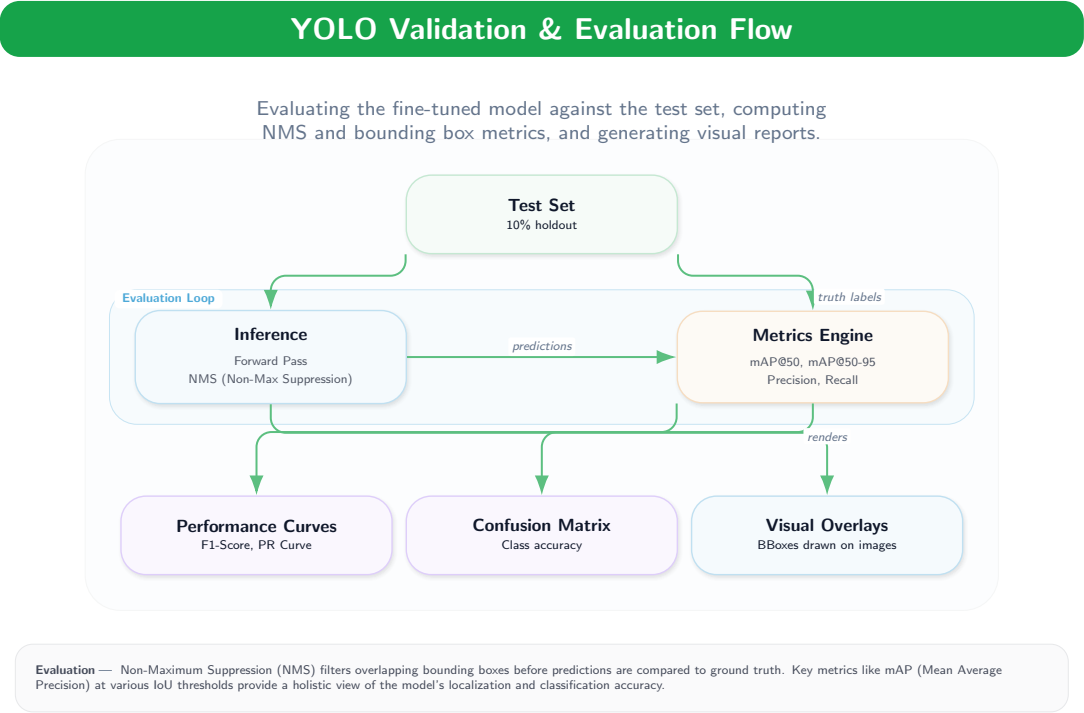


Figure 22: YOLO validation and evaluation flow showing metric computation, threshold analysis, and validation outputs.

The YOLO training metrics show the training trajectory across 100 epochs. The best recorded training summary used in the manuscript reports precision 0.71717, recall 0.74868, mAP50 0.76163, and mAP50-95 0.60191 at epoch 100. Robust validation provides an additional evaluation view using the validation images and labels. In the baseline robust-validation summary, 3,527 images and 20,443 ground-truth objects were evaluated, producing 3,701 true positives, 171 false positives, and 16,742 false negatives at confidence threshold 0.25 and IoU threshold 0.5. This corresponds to precision 0.69286, recall 0.79522, F1 0.74016, and mean inference time 12.609 ms in that validation run.

The YOLO validation scripts also have automated tests for the validation pipeline itself. These tests verify data-YAML loading, image collection, normalized-label conversion, IoU computation, prediction-ground-truth matching, class-aware matching, argument/config resolution, validation state-machine step selection, validation command execution wiring, dataset provisioning, training manifest creation, quantization manifest behavior, model-validation manifest behavior, packaging manifest behavior, launch smoke

behavior, and resume behavior. These tests support the reliability of the validation tooling, while the quantitative result files support the detector-performance claims.

3.12.3 VLM Validation

The VLM subsystem was validated at two levels: dataset/pipeline validation and prediction validation. Dataset validation checked the prepared Qwen-format records and image references. The recorded dataset-validation summary reports 13,294 images found, 6,647 Qwen records, zero decode failures, zero missing Qwen images, and zero recorded issues. This supports the claim that the exported VLM dataset was structurally usable for training and evaluation.

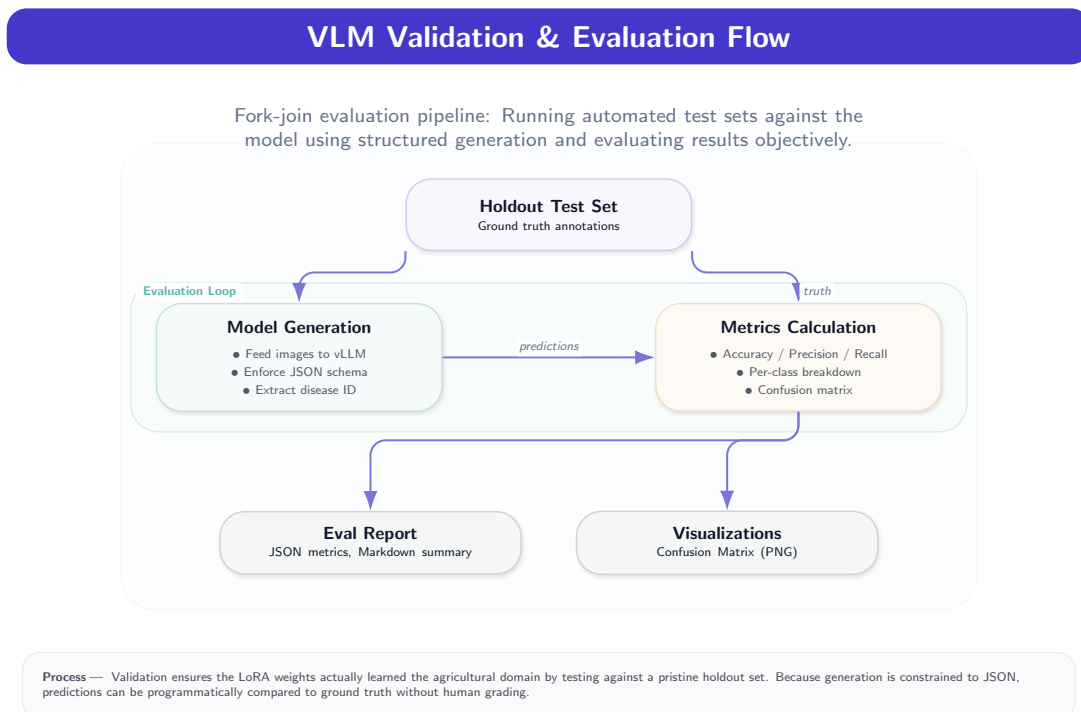


Figure 23: VLM validation and evaluation flow showing dataset checks, prediction validation, and parse-error handling.

Prediction validation was more severe. The independent test split contained 798 examples across 13 classes. The recorded prediction metrics report accuracy 0.13033, macro precision 0.52317, macro recall 0.18342, macro F1 0.24655, and 540 parse-error predictions. These values are already discussed in the VLM results section, but they are repeated here as validation evidence because they define the current reliability boundary of the VLM component. The validation does not support a claim that the current fine-tuned VLM is already diagnostically strong. It supports a more careful claim: the VLM training and evaluation pipeline exists, the dataset export is structurally validated, and the current model behavior

exposes a serious parseability and prediction-quality limitation that the final system must acknowledge.

The VLM repository also contains automated tests for dataset normalization, deduplication, source adapters, split creation, manifest behavior, training data loading, training runtime compatibility, runtime accelerator configuration, inference runtime configuration, inference batching, performance batch-size resolution, validation metric calculation, prediction parsing, and the FastAPI inference gateway. Across the visible VLM test files, there are 46 Python test functions, including 11 tests dedicated to the inference API route behavior.

3.12.4 VLM Inference API Validation

The VLM inference gateway has a focused API test suite. These tests use a stubbed vLLM client and FastAPI test client rather than loading the full model. This is appropriate for route-contract validation because the purpose is to verify HTTP behavior, authentication handling, request validation, and error mapping without requiring GPU inference in the test process.

The inference API tests cover liveness, readiness failure, readiness success, missing authentication, insufficient JWT scope, invalid base64 image rejection, successful inference response mapping, timeout/unreachable/backend-failure mapping, environment setting loading, settings-cache overrides, and configured rate-limit behavior. This gives the manuscript concrete evidence that the inference gateway is not only a conceptual endpoint. Its API contract and failure behavior are represented in automated tests.

3.12.5 Backend Validation

The backend validation evidence is strongest for the scan domain. The available unit tests focus on the scan service, and the integration tests focus on scan routes. The test setup uses MongoDB Memory Server, clears database collections between tests, and configures test environment variables for JWT, Cloudinary, Redis, email, MongoDB, and server port. This allows scan behavior to be tested without depending on a persistent external MongoDB instance.

The scan service unit tests cover scan creation, offline scan creation, owner-scoped retrieval, non-existent scan handling, cross-user access protection, pagination, status filtering, crop-type filtering, update behavior, soft deletion, statistics, resolution creation, duplicate resolution rejection, resolution retrieval, follow-up linkage, related scans, and grouping. The scan route integration tests cover authentication requirement, paginated re-

trieval, filtering, pagination, statistics, grouping, invalid group-by validation, scan lookup, missing-scan behavior, status retrieval, resolution creation, duplicate-resolution protection, follow-up linkage, and related-scan lookup.

There are 21 visible scan-service unit test cases and 26 visible scan-route integration test cases. This is meaningful evidence for the scan domain, but it is not complete backend coverage. Equivalent test evidence is not yet visible for every authentication, crop, task, note, advisory, diagnosis, VLM-client, Google-advisory-client, and upload-validation path. Therefore, the backend should be described as partially automated-tested, with scan behavior well represented and broader route-domain tests still required before claiming full backend verification.

3.12.6 Mobile Validation

The mobile application currently has source-level and harness-level evidence, but it does not yet have complete automated workflow evidence. Source inspection verifies the route hierarchy, provider setup, repository bootstrap, SQLite schema, offline queue, sync manager, diagnosis hook, API client, result view model, settings, localization, and model-asset loading paths. This is enough to document the implemented architecture, but it does not prove physical-device behavior.

The visible mobile test evidence is an ONNX YOLO engine test harness. It provides utilities for model loading, warmup, dummy-image inference, detection validation, inference-result validation, and full test-suite execution. However, it is a harness rather than evidence of a complete executed mobile app test suite. The mobile manuscript should therefore avoid claiming full app-level validation until an emulator or physical-device run is recorded. The remaining evidence should include screen screenshots, camera behavior, TFLite loading behavior, mobile-to-backend request timing, result-screen rendering, offline queue behavior, and recovery after network restoration.

3.12.7 Integration Validation

Integration validation is currently source-verified and contract-described rather than fully runtime-demonstrated. The source confirms the mobile-to-backend route contract, backend upload validation, backend-to-VLM readiness and inference calls, VLM gateway authentication and image validation, backend VLM prediction parsing, advisory schema validation, structured response assembly, mobile result mapping, and offline queueing. The generated integration diagram and integration tables document those boundaries.

The final missing step is live end-to-end validation. A complete final validation pass

should run the mobile app against a configured backend and VLM gateway, capture a real diagnosis request, record the backend response JSON, collect backend and VLM logs, measure latency, verify the result screen, and test offline-to-online recovery. Until that evidence is added, the manuscript should state that the integration workflow is implemented and source-verified, not that it has been fully production-validated.

Table 11: Testing and validation evidence summary

Area	Evidence available	What it supports	Remaining gap
YOLO	Training, robust validation, threshold, quantization, latency, error outputs; validation-tool tests	Detector-performance and deployment-tradeoff claims	Direct mobile-device detector profiling
VLM	Dataset validation, prediction metrics, confusion matrix, parse diagnostics; 46 visible Python tests	Dataset integrity and honest VLM limitation claims	Model improvement and stricter output-format reliability
VLM API	11 visible inference API tests	Health/readiness/auth/validation/error/limit contract claims	Live GPU-backed endpoint logs and latency
Backend	Scan service and scan route tests with MongoDB memory server	Scan-domain service and route behavior claims	Auth/crop/task/note/advisory/diagnosis test expansion
Mobile app	Source verification and ONNX inference harness	Architecture and planned inference-check claims	Executed emulator/device workflow tests and screenshots
Integration	Source-verified route and schema contracts	Implemented workflow boundary claims	Live end-to-end request, response, latency, and offline recovery evidence

CHAPTER 4

4. RESULTS AND DISCUSSION

4.1 YOLO Training Results

4.1.1 Training Dynamics

The YOLO detector was trained for 100 epochs. The final recorded training time was 25,646.7 seconds, equivalent to 7.12 hours. The training run reached its best recorded epoch at epoch 100, which indicates that the selected run did not terminate early before completing the configured training schedule.

Figure 24 presents the training dynamics. The curve shows the relationship between loss reduction and detection metrics across the training process. The model improves substantially from the early epochs and stabilizes toward the later part of training. The final epochs are important because the best mAP@50-95 value is recorded at the end of the run, not in an earlier intermediate checkpoint.

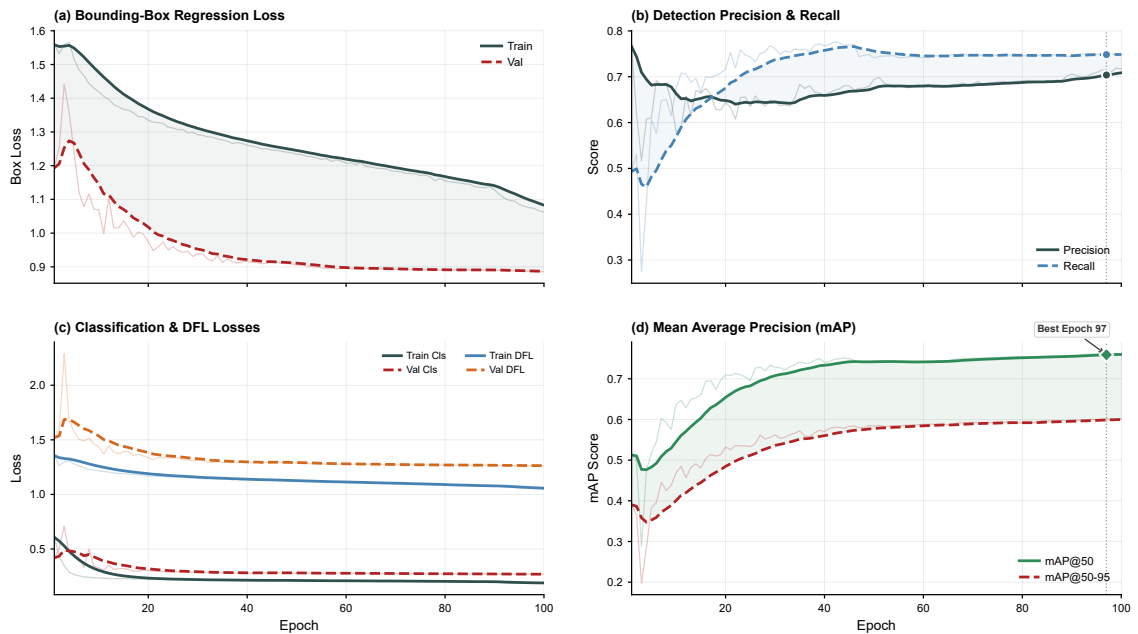


Figure 24: Training dynamics of the Krishi Vaidya YOLO detector

Table 12: Summary of training dynamics and best-epoch performance for the YOLO detector

Metric / Parameter	Value
Total epochs trained	100
Best training epoch	100
Input resolution (pixels)	640×640
Total training time	7.12 hours
<i>Loss residuals at best epoch</i>	
Train box loss	1.0627
Train class loss	0.1871
Train DFL loss	1.0493
Validation box loss	0.8855
Validation class loss	0.2681
Validation DFL loss	1.2635
<i>Detection accuracy at best epoch</i>	
Precision	0.7172
Recall	0.7487
F1-score	0.7326
mAP@50	0.7616
mAP@50-95	0.6019

4.1.2 Best-Epoch Performance

Table 12 summarizes the final and best-epoch values. The model achieved precision of 0.7172, recall of 0.7487, F1-score of 0.7326, mAP@50 of 0.7616, and mAP@50-95 of 0.6019. The recall is slightly higher than precision, which means the detector is more inclined toward finding annotated objects than toward filtering out every uncertain prediction. For the Krishi Vaidya workflow, this is a reasonable property for a first-stage visual detector because missed plant regions can prevent later analysis, while uncertain detections can still be filtered by thresholding, downstream validation, or user interaction.

The difference between mAP@50 and mAP@50-95 also matters. mAP@50 measures whether detected objects overlap the ground truth at a moderate threshold, while mAP@50-95 requires stronger localization across stricter IoU thresholds. The lower mAP@50-95 value indicates that localization quality remains more difficult than basic object discovery, especially in a dataset with strongly uneven class frequency and object scales.

4.2 YOLO Deployment and Threshold Results

4.2.1 Threshold Selection Behavior

The confidence-threshold sweep shows the expected precision-recall tradeoff. At the lowest tested threshold of 0.05, recall reaches 0.7848, but precision is lower at 0.6748. At moderate thresholds, precision rises while recall falls. The highest F1-score in the sweep is 0.7362 at a confidence threshold of 0.10. This threshold gives precision of 0.7463 and recall of 0.7264.

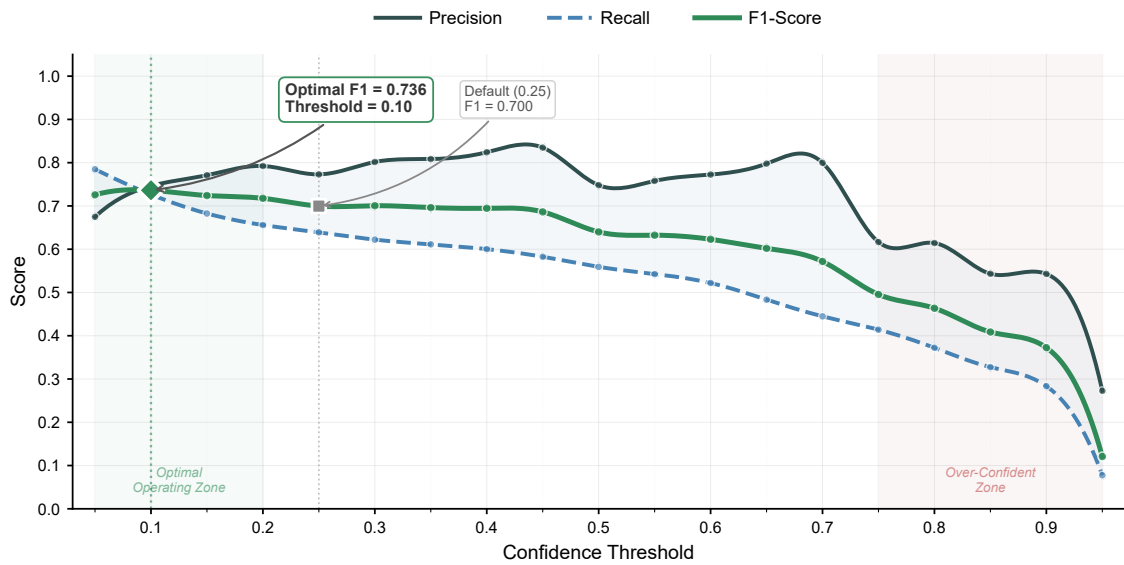


Figure 25: Confidence-threshold sensitivity of the YOLO detector

Table 13: Sensitivity sweep of confidence thresholds on the YOLO baseline model

Confidence threshold	Precision	Recall	F1-score	mAP@50	mAP@50-95	Fitness
0.05	0.6748	0.7848	0.7256	0.7225	0.5799	0.5799
0.10	0.7463	0.7264	0.7362	0.6879	0.5613	0.5613
0.15	0.7710	0.6825	0.7241	0.6573	0.5441	0.5441
0.20	0.7923	0.6559	0.7177	0.6352	0.5314	0.5314
0.25	0.7729	0.6390	0.6996	0.6219	0.5227	0.5227
0.30	0.8016	0.6221	0.7006	0.6082	0.5138	0.5138
0.35	0.8084	0.6113	0.6962	0.5983	0.5065	0.5065
0.40	0.8241	0.6003	0.6946	0.5885	0.4986	0.4986
0.45	0.8348	0.5826	0.6863	0.5730	0.4860	0.4860
0.50	0.7480	0.5591	0.6399	0.5510	0.4712	0.4712
0.55	0.7579	0.5426	0.6324	0.5344	0.4587	0.4587
0.60	0.7725	0.5220	0.6230	0.5164	0.4466	0.4466
0.65	0.7978	0.4834	0.6020	0.4790	0.4221	0.4221
0.70	0.7997	0.4448	0.5717	0.4412	0.3962	0.3962
0.75	0.6166	0.4140	0.4954	0.4109	0.3728	0.3728
0.80	0.6143	0.3722	0.4635	0.3712	0.3412	0.3412
0.85	0.5433	0.3274	0.4086	0.3263	0.3044	0.3044
0.90	0.5429	0.2835	0.3725	0.2818	0.2643	0.2643
0.95	0.2727	0.0776	0.1208	0.0777	0.0729	0.0729

The threshold sweep also shows why a high confidence threshold is unsuitable for this detector. At 0.95, recall falls to 0.0776 and F1-score falls to 0.1208. This would remove most detections and make the detector ineffective as a first-stage model. Conversely, the 0.05 threshold has higher recall but lower precision than the selected threshold. The 0.10 operating point therefore provides a practical balance between keeping enough detections and controlling false positives.

4.2.2 Quantization Tradeoffs

The deployment comparison evaluates the baseline PyTorch model and three TensorFlow Lite variants. The baseline PyTorch model has the lowest latency in the recorded environment, with 12.18 ms per image and 82.1 FPS. The TFLite variants are slower in this benchmark, but they represent the model formats needed for lightweight deployment workflows.

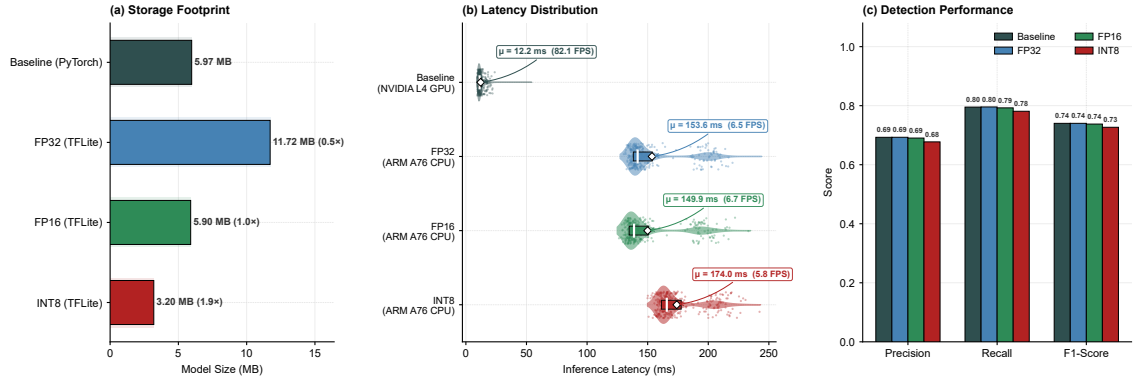


Figure 26: Deployment comparison across baseline and quantized YOLO variants

Table 14: Deployment efficiency comparison across YOLO model variants

Variant	Size (MB)	Compression	Device	Latency (ms)	FPS	Precision	Recall	F1
Baseline (PyTorch)	5.97	1.00×	cuda:0	12.18	82.1	0.6929	0.7952	0.7402
FP32 (TFLite)	11.72	0.51×	cuda:0	153.59	6.5	0.6930	0.7957	0.7402
FP16 (TFLite)	5.90	1.01×	cuda:0	149.94	6.7	0.6903	0.7926	0.7377
INT8 (TFLite)	3.20	1.87×	cuda:0	174.01	5.8	0.6774	0.7811	0.7268

The INT8 model is the smallest variant at 3.20 MB and gives the strongest compression ratio, $1.87\times$ relative to the baseline size. This reduction has a measurable accuracy cost: precision decreases from 0.6929 to 0.6774, recall decreases from 0.7952 to 0.7811, and F1-score decreases from 0.7402 to 0.7268. FP16 preserves performance more closely, with F1-score of 0.7377 and a model size of 5.90 MB. The FP32 TFLite model matches the baseline F1-score but is larger than the baseline model in this export.

These results show that quantization should not be selected only by size. For a mobile-first system, INT8 is attractive because of its smaller footprint, but FP16 may be preferable when a small accuracy loss is more important than maximum compression. Final selection should also consider actual Android-device profiling, because the current latency values were measured on `cuda:0`.

4.2.3 Per-Class Behavior

Aggregate deployment metrics hide important class-level differences. Table 15 shows that the best variant changes by class. The baseline PyTorch model gives the best F1-score for rice leaf, rice panicle, tomato leaf, brassica leaf, and cabbage head. FP32 performs best for rice grain cluster, maize ear, and tomato fruit. FP16 performs best for potato leaf. INT8 performs best for maize leaf and cauliflower head.

Table 15: Per-class F1-score comparison across YOLO model variants

Class	Baseline (PyTorch)	FP32 (TFLite)	FP16 (TFLite)	INT8 (TFLite)
Rice leaf	0.7750	0.7716	0.7207	0.7124
Rice panicle	0.7799	0.7295	0.7277	0.7426
Rice grain cluster	0.6911	0.7563	0.6980	0.7093
Maize leaf	0.7139	0.7162	0.7137	0.7655
Maize ear	0.6934	0.7736	0.7115	0.7363
Potato leaf	0.7446	0.7219	0.7857	0.6977
Tomato leaf	0.7793	0.7089	0.7336	0.6885
Tomato fruit	0.7788	0.7852	0.7085	0.7083
Brassica leaf	0.7876	0.7327	0.7240	0.7118
Cauliflower head	0.7164	0.7058	0.7255	0.7361
Cabbage head	0.7833	0.7252	0.7740	0.7245

This result suggests that quantization affects classes unevenly. The effect is not simply a uniform decline from baseline to INT8. Some classes improve under a converted model, while others lose performance. Because Krishi Vaidya targets multiple crop structures, deployment decisions should therefore consider the classes most important to the intended use case, not only the aggregate F1-score.

4.3 YOLO Error Analysis

4.3.1 Confusion Patterns

Figure 27 presents the confusion matrix for the YOLO detector. The confusion matrix is necessary because the dataset has severe class imbalance and because aggregate metrics do not show whether failures are concentrated in a few classes.

Figure 4: Baseline Model Confusion Matrix

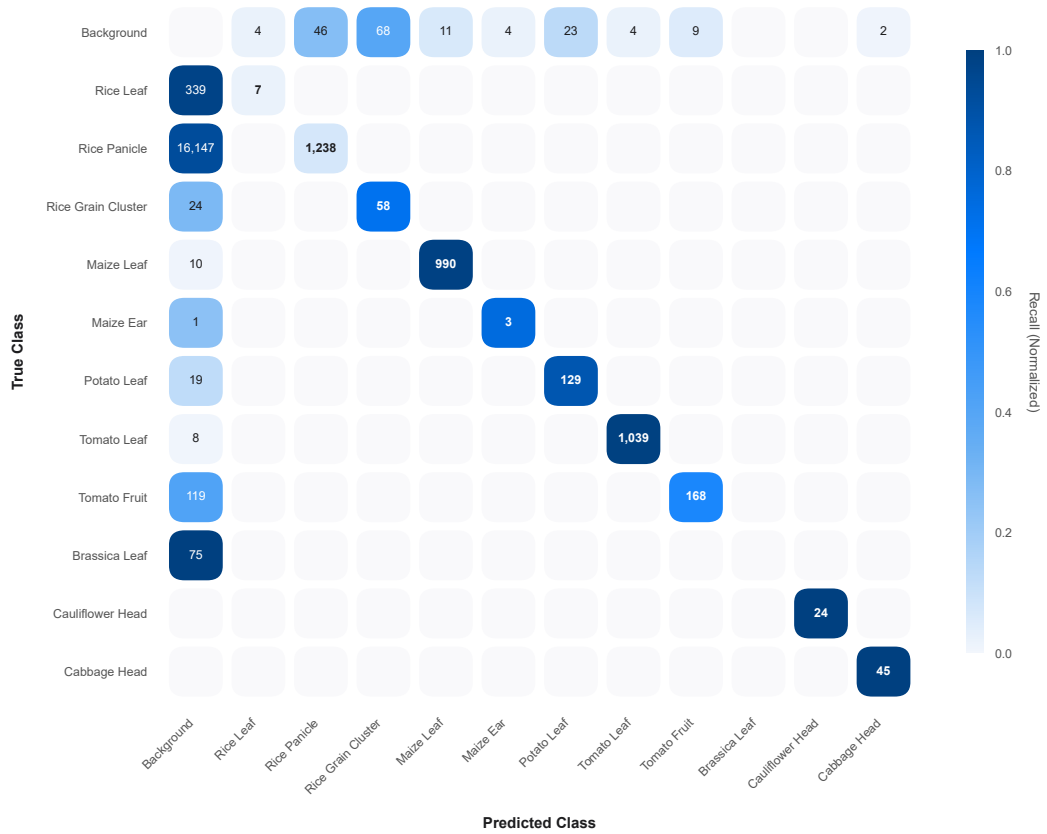


Figure 27: Confusion matrix of the YOLO detector

4.3.2 Error Categories

Table 16 separates true positives, missed detections, and ghost detections by class. The most significant failure is the rice panicle class, where 16,147 objects are missed, corresponding to 92.9% false negatives. Rice leaf also has a very high missed rate, with 339 missed objects and only 7 true positives. Brassica leaf has 75 missed objects and no true positives in the reported breakdown.

Table 16: Error analysis breakdown by YOLO class

Class	True positives	False negatives (missed)	False positives (ghost)
Rice panicle	1,238	16,147 (92.9%)	46 (3.6%)
Rice leaf	7	339 (98.0%)	4 (36.4%)
Tomato fruit	168	119 (41.5%)	9 (5.1%)
Rice grain cluster	58	24 (29.3%)	68 (54.0%)
Brassica leaf	0	75 (100.0%)	0 (0.0%)
Potato leaf	129	19 (12.8%)	23 (15.1%)
Maize leaf	990	10 (1.0%)	11 (1.1%)
Tomato leaf	1,039	8 (0.8%)	4 (0.4%)
Maize ear	3	1 (25.0%)	4 (57.1%)
Cabbage head	45	0 (0.0%)	2 (4.3%)
Cauliflower head	24	0 (0.0%)	0 (0.0%)

The error pattern shows that the detector is not equally reliable across classes. Maize leaf and tomato leaf show strong behavior in this breakdown, with low missed-detection and ghost-detection rates. Cabbage head and cauliflower head show no missed detections in the table, although their absolute true-positive counts are small. In contrast, rice panicle and rice leaf have high missed-detection rates, and rice grain cluster has a high ghost-detection rate. These results indicate that class-specific improvements are more important than simply extending training time.

The rice panicle result deserves careful interpretation. The dataset contains 115,046 rice panicle annotations, far more than any other class. Many panicle objects are likely small, clustered, or visually dense, and the curation report also shows that many dropped objects were related to out-of-bounds, truncated, or very small annotations. The high false-negative count therefore indicates a concrete limitation of the current detector: it can learn useful aggregate detection behavior, but dense or small structures remain difficult.

4.3.3 Summary of YOLO Findings

The YOLO component provides a complete, evidence-backed object-detection subsystem for Krishi Vaidya. The trained detector achieved F1-score of 0.7326, mAP@50 of 0.7616, and mAP@50-95 of 0.6019 at the best recorded epoch. The threshold sweep identifies 0.10 as the best F1 operating point among the tested confidence thresholds. The deployment comparison shows that TFLite variants preserve useful detection performance, with INT8 providing the smallest model size and FP16 providing a closer accuracy-size compromise.

The main limitation is class-specific reliability. The detector performs much better for some classes than others, and dense or underrepresented structures require further improvement. For the larger Krishi Vaidya system, this means the YOLO model is suitable as a visual detection component, but its outputs should be treated as supporting evidence rather than as the sole basis for disease diagnosis.

4.4 VLM Training and Resource Results

4.4.1 Fine-Tuning Performance

The VLM training run completed 3 epochs and 969 training steps using Qwen2.5-VL-3B-Instruct as the base model. Table 17 summarizes the training outcome. The final reported evaluation loss is 0.0146, and the mean token accuracy is 0.9967. The trainer-reported train loss is 0.1048, while the last logged train loss is 0.0097. These values show that the supervised fine-tuning process fitted the training objective strongly.

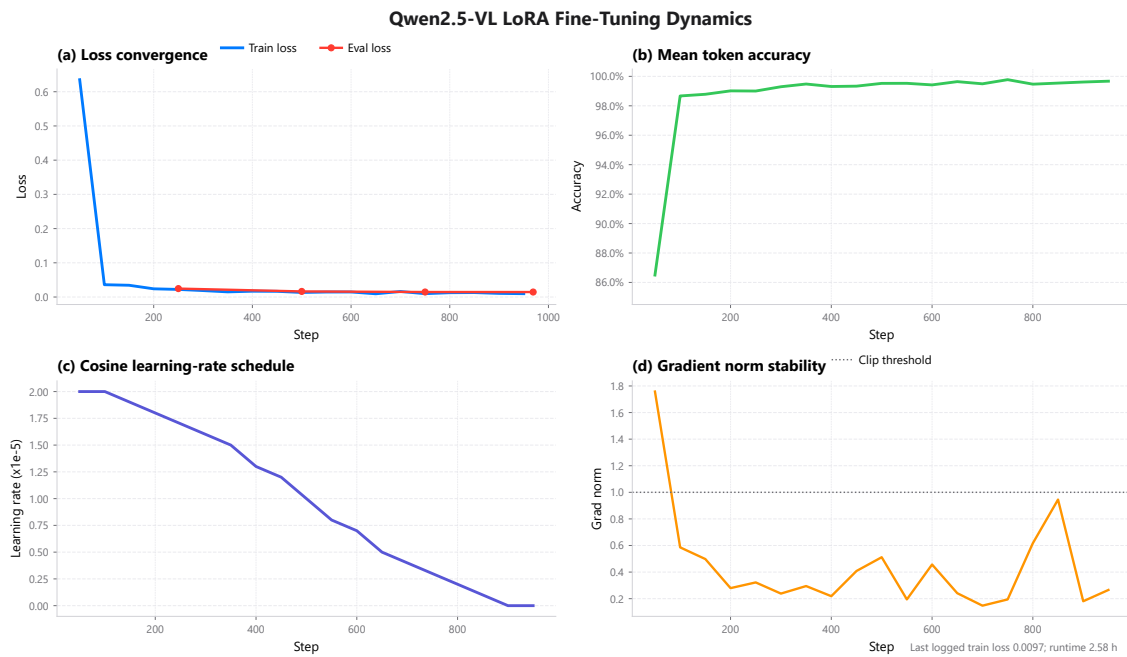


Figure 28: Training dynamics of the Qwen2.5-VL LoRA fine-tuning run

Table 17: Training summary for Qwen2.5-VL LoRA fine-tuning

Metric / setting	Value
Base model	Qwen/Qwen2.5-VL-3B-Instruct
Epochs completed	3.00
Train steps	969
Last logged train loss	0.0097
Trainer-reported train loss	0.1048
Final eval loss	0.0146
Mean token accuracy	0.9967
Runtime	2.58 h
Samples/sec	1.664
Learning rate	2.00e-05
Effective batch	4 x 4
LoRA rank / alpha	16 / 32
Quantization	4-bit nf4

The training result must be interpreted carefully. The low evaluation loss and high token accuracy indicate that the model learned the supervised answer format on the validation objective. They do not, by themselves, prove that the model will produce parseable and correct disease labels during free generation on the test split. For that reason, the VLM evaluation includes a separate prediction-metrics study, discussed in Section 4.5.

4.4.2 Runtime and Memory Behavior

The resolved runtime used an NVIDIA L4 GPU with CUDA 12.8 and compute capability 8.9. The run used `flash_attention_2`, Liger kernel support, bfloat16, and TF32. The memory telemetry shows peak allocated GPU memory of 12.56 GB and peak reserved memory of 13.08 GB, within the available 22.03 GB GPU memory.

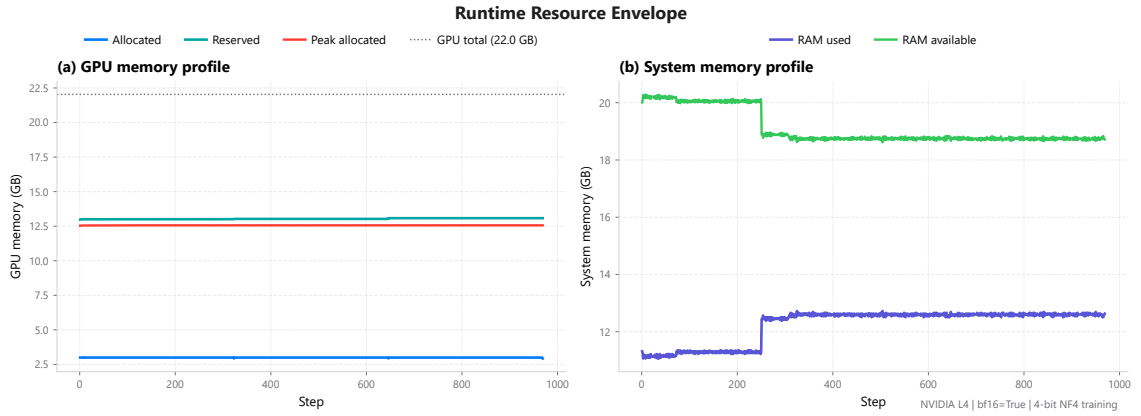


Figure 29: Runtime and memory behavior of the VLM training run

Table 18: Runtime and resource envelope for VLM fine-tuning

Metric / setting	Value
GPU	NVIDIA L4
CUDA / capability	12.8 / 8.9
GPU memory	22.03 GB
Peak allocated	12.56 GB
Peak reserved	13.08 GB
Peak RAM used	40.6%
Attention backend	flash_attention_2
Liger kernel	True
torch.compile	False
bf16 / tf32	True / True

The resource profile supports the technical feasibility of adapter fine-tuning a 3B-parameter VLM under a single-GPU setup. The use of 4-bit NF4 quantization, LoRA adapters, bfloat16 computation, and efficient attention was important for keeping the run inside the available GPU memory.

4.5 VLM Prediction Results

4.5.1 Dataset Split and Class Distribution

The validation study confirms that the prepared Qwen export contains 6,647 records and 13,294 image references. The train split contains 5,147 Qwen records, the validation split contains 702 records, and the test split contains 798 records. The image count is exactly twice the record count in each split, as reported by the validation output.

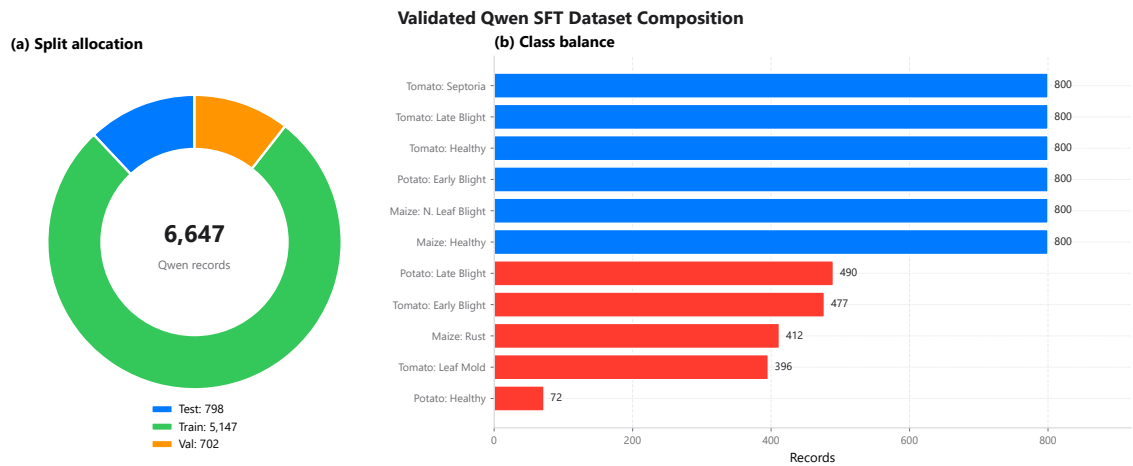


Figure 30: Dataset composition for the VLM Qwen export

Table 19: Dataset split validation summary for VLM records

Split	Qwen records	Images found
Train	5,147	10,294
Validation	702	1,404
Test	798	1,596
Total	6,647	13,294

Table 20: Qwen record distribution by VLM class

Class	Qwen records
Maize: Common Rust	412
Maize: Healthy	800
Maize: Northern Leaf Blight	800
Potato: Early Blight	800
Potato: Healthy	72
Potato: Late Blight	490
Tomato: Early Blight	477
Tomato: Healthy	800
Tomato: Late Blight	800
Tomato: Leaf Mold	396
Tomato: Septoria Leaf Spot	800

The class distribution is partially capped but still uneven. Several classes contain 800 records, while potato healthy contains only 72 records. This imbalance affects interpretation of macro metrics and per-class recall. The test split is also uneven: maize common

rust alone contributes 412 of the 798 test examples, which makes the model’s failure on that class especially important for the aggregate accuracy.

4.5.2 Prediction Summary

The test-set prediction metrics reveal a clear gap between supervised fine-tuning loss and usable structured prediction. Table 21 reports accuracy of 0.1303, macro precision of 0.5232, macro recall of 0.1834, and macro F1-score of 0.2465 over 798 test examples. The most important result is that 540 predictions, or 67.7%, are parse-error predictions.

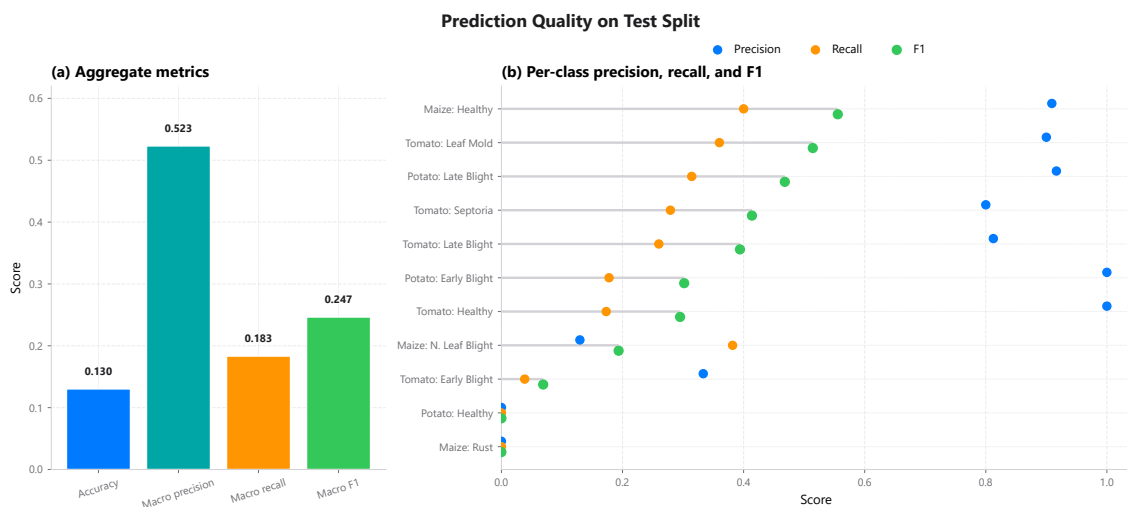


Figure 31: Prediction performance of the VLM on the test split

Table 21: Prediction summary on the VLM test split

Metric	Value
Split	test
Examples	798
Accuracy	0.1303
Macro precision	0.5232
Macro recall	0.1834
Macro F1	0.2465
Parse-error predictions	540 (67.7%)

This result is a central limitation of the current VLM implementation. The model was not judged only by whether the generated text looked plausible; it was judged by whether the generated output could be parsed into a canonical label for downstream use. A model response that cannot be parsed into `canonical_label` cannot reliably drive the backend

diagnosis or mobile-app result flow. Therefore, the parse-error rate directly reduces the practical diagnostic usability of the current VLM.

4.5.3 Per-Class Prediction Metrics

The per-class metrics in Table 22 show that the VLM behaves unevenly across classes. Maize healthy obtains the highest F1-score at 0.5556. Tomato leaf mold reaches 0.5143, potato late blight reaches 0.4681, tomato septoria leaf spot reaches 0.4138, and tomato late blight reaches 0.3939. Several classes perform poorly: maize common rust has 412 false negatives and no true positives, potato healthy has no true positives, and tomato early blight has only one true positive.

Table 22: Per-class prediction metrics for the VLM test split

Class	Support	Precision	Recall	F1	TP	FN	FP
Maize: Common Rust	412	0.0000	0.0000	0.0000	0	412	0
Maize: Healthy	50	0.9091	0.4000	0.5556	20	30	2
Maize: Northern Leaf Blight	55	0.1296	0.3818	0.1935	21	34	141
Potato: Early Blight	45	1.0000	0.1778	0.3019	8	37	0
Potato: Healthy	5	0.0000	0.0000	0.0000	0	5	0
Potato: Late Blight	35	0.9167	0.3143	0.4681	11	24	1
Tomato: Early Blight	26	0.3333	0.0385	0.0690	1	25	2
Tomato: Healthy	52	1.0000	0.1731	0.2951	9	43	0
Tomato: Late Blight	50	0.8125	0.2600	0.3939	13	37	3
Tomato: Leaf Mold	25	0.9000	0.3600	0.5143	9	16	1
Tomato: Septoria Leaf Spot	43	0.8000	0.2791	0.4138	12	31	3
Parse Error	0	0.0000	0.0000	0.0000	0	0	540
Maize: Early Leaf Spot	0	0.0000	0.0000	0.0000	0	0	1

The high false-positive count for maize northern leaf blight is also significant. The model predicts maize northern leaf blight 162 times, but only 21 of those predictions are correct. This creates 141 false positives for that label. The model also emits one prediction for maize early leaf spot, which is not a supported class in the reported test distribution. This indicates that the VLM can generate labels outside the expected class set unless output constraints are enforced.

4.5.4 Confusion Matrix and Prediction Distribution

The confusion matrix in Figure 32 visualizes how predictions are distributed across expected labels. The large parse-error column dominates the evaluation, which is consistent

with the prediction summary. This confirms that the main failure mode is not merely confusion between two similar disease classes; it is failure to produce a parseable canonical answer in many cases.

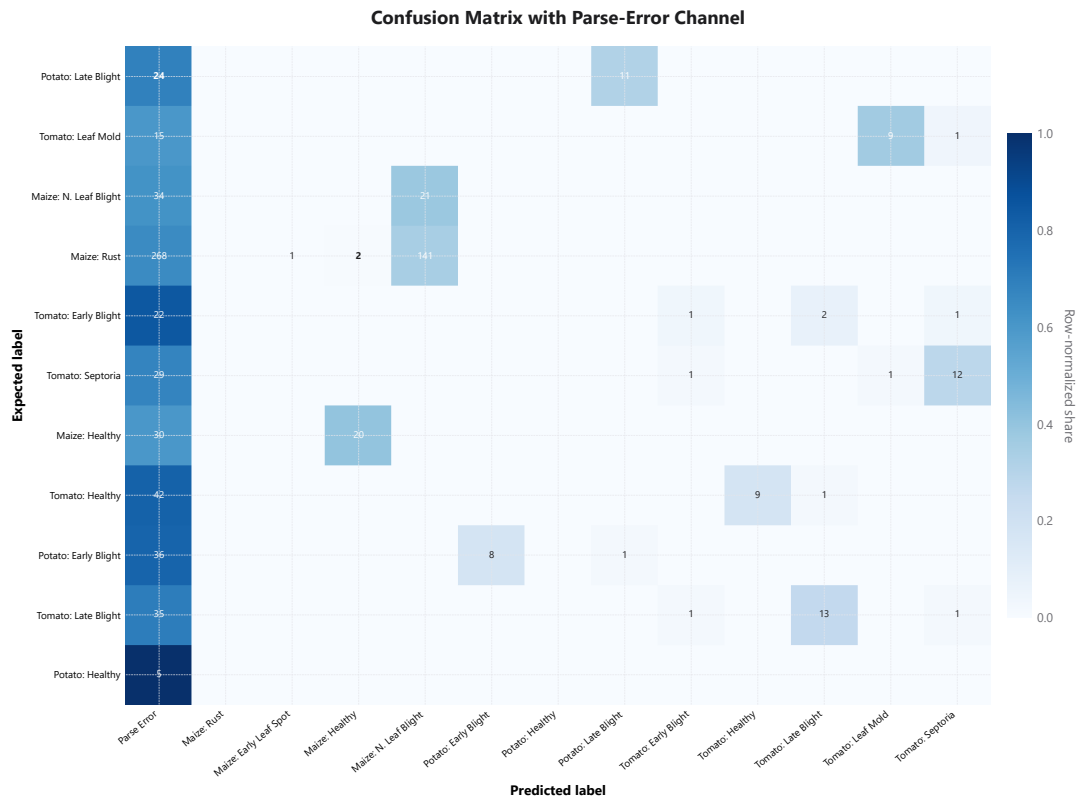


Figure 32: Confusion matrix for VLM prediction results

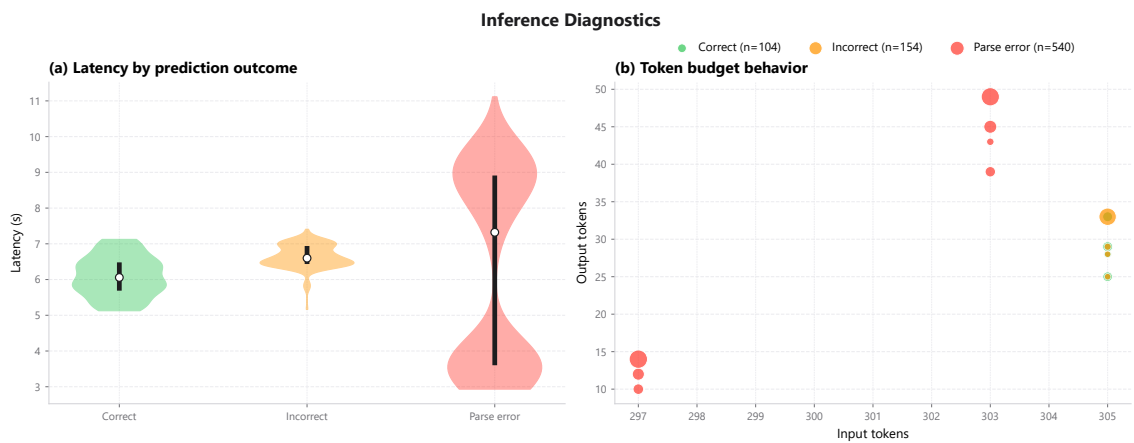


Figure 33: Inference diagnostics for VLM predictions

Table 23: Prediction distribution and median latency for the VLM test split

Predicted label	Count	Accuracy	Median latency
Parse Error	540	0.0000	7.32 s
Maize: Northern Leaf Blight	162	0.1296	6.67 s
Maize: Healthy	22	0.9091	5.32 s
Tomato: Late Blight	16	0.8125	5.92 s
Tomato: Septoria Leaf Spot	15	0.8000	6.50 s
Potato: Late Blight	12	0.9167	6.15 s
Tomato: Leaf Mold	10	0.9000	5.71 s
Tomato: Healthy	9	1.0000	5.32 s
Potato: Early Blight	8	1.0000	6.23 s
Tomato: Early Blight	3	0.3333	5.90 s
Maize: Early Leaf Spot	1	0.0000	5.88 s

The prediction distribution shows that parse errors are not only frequent but also slower than most successful label categories, with a median latency of 7.32 seconds. Non-parse-error labels generally have median latencies between 5.32 and 6.67 seconds. This suggests that the current generation path is expensive enough to require careful backend integration and that output formatting failures are costly in both accuracy and time.

4.5.5 Summary of VLM Findings

The VLM work demonstrates that Krishi Vaidya includes a real fine-tuned vision-language component, not only a detector or mobile interface. The model was fine-tuned with LoRA adapters on Qwen2.5-VL-3B-Instruct using 4-bit NF4 quantization and completed the training run within the available NVIDIA L4 resource envelope. The training objective was fitted successfully, as shown by the low final evaluation loss and high token accuracy.

However, the prediction results show that the current VLM is not yet reliable as an autonomous structured disease classifier. The major limitation is parseability: 540 of 798 test predictions could not be parsed into a canonical label. The model also shows class-level weaknesses, especially for maize common rust, potato healthy, and tomato early blight, and it can emit labels outside the expected test label set. For the larger Krishi Vaidya system, the VLM should therefore be described as a fine-tuned diagnostic component with clear current limitations. Its next improvement step is not simply more training;

it requires stricter output control, improved prompt consistency, possibly constrained decoding or post-processing, and targeted correction of weak classes.

4.6 Backend Implementation Results

4.6.1 Implemented API Surface

The backend implementation provides the server-side application layer required by Krishi Vaidya. The verified codebase contains eight main API domains: diagnosis, authentication, users, scans, crops, tasks, notes, and advisories. These domains cover the system’s core workflow: a farmer can authenticate, maintain profile information, upload crop images, preserve scan history, manage crop plots, attach notes, track tasks, and receive or manage advisories. The backend also exposes a health endpoint and development-time Swagger documentation.

Table 24: Backend implementation coverage by domain

Domain	HTTP coverage	Persistence	Important behavior
Diagnosis	POST	External VLM and Google AI calls	Validates image, calls VLM, generates advisory JSON
Authentication	POST	User, OTP, refresh-token models	Register, login, refresh, logout, OTP, password reset
Users	GET, PATCH, PUT, DELETE, POST	User model	Profile, password, profile picture, push token management
Scans	GET, POST, PATCH, DELETE	Scan and scan-resolution models	Upload, filter, group, resolve, follow-up, status tracking
Crops	GET, POST, PATCH, DELETE	Crop-plot model	Plot lifecycle, archive state, stage update, health update
Tasks	GET, POST, PATCH, DELETE	Crop-task model	Pending, overdue, upcoming, completion, batch creation
Notes	GET, POST, PATCH, DELETE	Crop-note model	Plot notes, scan-linked notes, image attachments
Advisories	GET, POST, PATCH, DELETE	Advisory model	Active advisories, dismiss, undismiss, batch creation

The implemented API surface is broader than a diagnosis-only prototype. It supports both disease analysis and crop-management records. This matters for the final system because the mobile application needs more than a single prediction endpoint: it must keep user-specific farm context, historical scans, and advisory state. The backend provides this context through authenticated, user-scoped resources rather than through anonymous image processing alone.

4.6.2 Layered Implementation Outcome

The final backend structure follows a consistent request path. Incoming requests enter the Express application, pass through middleware, reach a route group, execute controller validation, call a service method, and then use repository methods for persistence when required. The same pattern is used across scans, crops, tasks, notes, and advisories, which reduces implementation inconsistency between domains.

Table 25: Backend implementation layers and their realized responsibilities

Layer	Implemented files	Realized responsibility
Configuration	<code>src/config</code>	Environment validation, MongoDB, Redis, Cloudinary, Swagger, runtime options
Application shell	<code>src/app.ts</code>	Middleware registration, health endpoint, route mounting, service initialization
Container	<code>src/container.ts</code>	Explicit dependency construction for repositories, services, controllers, routes
Middleware	<code>src/middlewares</code>	Authentication, CORS, Helmet, rate limits, upload parsing, validation, logging, errors
Routes	<code>src/routes</code>	Versioned API grouping and Swagger-annotated endpoint definitions
Controllers	<code>src/controllers</code>	Request validation, parameter extraction, response formatting
Services	<code>src/services</code>	Domain rules, image upload orchestration, diagnosis pipeline, advisory generation
Repositories	<code>src/repositories</code>	Mongoose queries, pagination, ownership filters, soft-delete access patterns
Models	<code>src/models</code>	MongoDB schemas, indexes, validation rules, document methods

This separation is useful for thesis interpretation because it shows that the backend was designed as a maintainable application component, not as a collection of unrelated Express handlers. The service and repository layers also make testing more practical because domain behavior can be tested below the HTTP layer.

4.7 Backend Domain Discussion

4.7.1 Authentication and User Management

The authentication implementation supports registration, login, access-token refresh, logout, logout from all devices, OTP sending, OTP verification, forgot-password, and password reset. The password policy requires at least eight characters and at least one uppercase letter, one lowercase letter, and one number. Refresh tokens are not stored as plain tokens; they are represented through hashed records with lookup identifiers and revocation state. This is an appropriate design choice because refresh tokens are long-lived compared with access tokens and should not be recoverable from the database as usable secrets.

User management includes profile retrieval and update, profile-picture upload, password change, account deletion, and push-token add/remove operations. The profile schema includes bilingual and Nepal-specific fields such as Nepali name, preferred language, ecological region, province, district, municipality, ward, latitude, and longitude. It also includes theme and color-blind mode preferences, which shows that the backend stores application-level preference state rather than only authentication data.

4.7.2 Scan Lifecycle

The scan domain is the most complete persistence-backed workflow in the backend. A scan stores the uploaded image URI, crop metadata, status, result object, notes, tags, of-

fine flag, client-created timestamp, deletion state, and optional processing timestamps. The scan service uploads images to Cloudinary before creating scan records. It also supports scan retrieval, pagination, filtering by status and crop type, soft deletion, batch deletion, grouped views, related scans, status checks, scan resolution, resolution retrieval, follow-up linkage, pending-scan retrieval, offline-scan retrieval, offline scan synchronization, result updates, and failed-scan marking.

The scan-resolution model adds an important post-diagnosis layer. A completed scan can be marked as treated, resolved naturally, crop removed, or false positive. Treatment text, notes, and follow-up scan identifiers can be stored. This makes the backend suitable for tracking what happens after a disease prediction, which is important for an agricultural advisory system because diagnosis alone does not measure whether the farmer acted on the recommendation or whether the crop condition improved.

4.7.3 Crop, Task, Note, and Advisory Management

Crop plots represent the farmer's managed crop areas. Each plot stores crop type, variety, region, district, optional location, area value and unit, planting date, expected harvest date, growth stage, stage progress, health status, archive state, and soft-delete state. The crop routes support creation, retrieval, filtering, archive/unarchive actions, growth-stage updates, and health-status updates.

Tasks provide a scheduling layer around crop management. The task model supports categories such as watering, fertilizing, spraying, weeding, pruning, harvesting, soil preparation, scouting, and other tasks. Each task has a due date, completion state, priority, and optional auto-generated flag. The implemented routes expose pending, overdue, upcoming, plot-specific, batch, completion, and uncompletion operations.

Notes provide a record-keeping layer. A note is linked to a crop plot and can optionally be linked to a scan. It stores note type, title, body, image URIs, public image identifiers, tags, and soft-delete state. The implementation supports adding and removing images after note creation, which is important because farmers may add evidence over time rather than at the exact moment the note is first created.

Advisories provide a notification-like layer for crop decisions. The advisory model supports weather alerts, irrigation needs, fertilizer reminders, pest risk, disease risk, harvest windows, spray windows, frost warnings, and stage transitions. Advisories include severity, bilingual title and description fields, optional action labels, generated time, dismiss state, and expiry. The model uses an expiry index, which allows expired advisories to be cleaned up automatically.

4.8 Backend Diagnosis Integration Discussion

4.8.1 Structured Diagnosis Response

The backend diagnosis pipeline produces a structured response rather than returning raw model text to the mobile app. The response contains disease name, Nepali disease name, scientific name, crop, confidence, confidence tier, out-of-distribution flag, symptom descriptions, advisory details, processing time, request identifier, timestamp, and model version. The advisory object contains treatment steps, prevention tips, safety warnings, organic alternatives, and source URLs.

This response structure is important because the mobile application needs stable fields for display and decision support. It also reduces the risk of presenting unstructured model text directly to a farmer. The backend still uses the VLM output, but it places a second structured-generation and schema-validation step after the VLM to produce an application-facing advisory response.

4.8.2 Confidence and Out-of-Distribution Handling

The backend converts confidence into four confidence tiers: high, medium, low, and out-of-distribution. If the advisory generator marks a case as out of distribution, or if confidence is below 0.30, the backend assigns the ood tier. Confidence values from generated outputs are clamped between 0 and 1. Alternative diagnoses are included only when the confidence tier is medium or low. This design avoids overloading high-confidence responses with unnecessary alternatives and allows uncertain predictions to carry additional possibilities.

The design is sound, but it depends on the reliability of upstream VLM prediction and downstream advisory JSON generation. The VLM prediction results in this manuscript show high parse-error rates during independent evaluation. Therefore, the backend's structured advisory layer should be understood as an integration safeguard, not as proof that the current VLM model is already diagnostically reliable in all cases.

4.9 Backend Verification and Limitations

4.9.1 Automated Test Coverage

The available automated tests focus mainly on scan behavior. Unit tests cover scan creation, offline-scan creation, owner-scoped retrieval, filtering, updating, soft deletion,

statistics, resolution creation, duplicate-resolution prevention, resolution retrieval, follow-up linkage, and grouping by crop, disease, and month. Integration tests cover scan route authentication behavior, paginated retrieval, filtering, statistics, grouping validation, lookup by identifier, update, soft deletion, batch deletion, status retrieval, resolution creation, duplicate-resolution protection, resolution retrieval, follow-up linkage, and related-scan lookup.

Table 26: Implemented backend test focus

Test area	Evidence in repository	Covered behavior
Scan service unit tests	tests/unit/services/scanService.test.ts	Creation, ownership, filtering, update, delete, stats, resolution, follow-up, grouping
Scan route integration tests	tests/integration/routes/scanRoutes.test.ts	Auth requirement, pagination, filters, validation, status, resolution, related scans
Test database setup	tests/setup.ts	MongoDB memory server, database clearing, test environment variables

The test coverage is useful but not complete. It provides evidence that the scan workflow was treated seriously, but it does not yet provide equivalent test evidence for every route group. Authentication, crop, task, note, advisory, and diagnosis integration tests should be expanded before claiming complete backend verification.

4.9.2 Current Backend Limitations

Several limitations remain. First, the backend section currently has an architecture figure, but it still lacks captured API screenshots, Swagger screenshots, request-response examples from a running deployment, and deployment metrics. Second, the scan test suite is stronger than the test coverage for the other domains. Third, Redis is initialized with BullMQ-compatible options, but the visible implementation does not yet demonstrate a completed background-job pipeline for diagnosis processing. Fourth, the diagnosis route uses external VLM and Google AI services; therefore, final system reliability depends on those services being reachable, correctly configured, and returning valid schemas. Fifth, direct production-load testing and mobile-to-backend latency measurements have not yet been added to the manuscript evidence.

These limitations do not invalidate the backend implementation. They define what should be added before the final manuscript claims production readiness. The current backend can be described as a substantial implemented API layer with verified architecture, domain models, route groups, security controls, image handling, and diagnosis integration, while final deployment evidence and broader automated tests remain future documentation tasks.

4.10 Mobile App Implementation Results

4.10.1 Implemented Application Surface

The mobile application implementation shows that Krishi Vaidya has progressed beyond a model-only prototype. The app contains a complete navigation shell, onboarding flow, scan workflow, crop dashboard, history view, settings area, notification handling, export features, and local persistence. The implemented source also contains development fallbacks for web execution and native-specific paths for camera and TFLite functionality. This is important because the final system must support actual farmer workflows rather than only demonstrate that an image can be sent to a model.

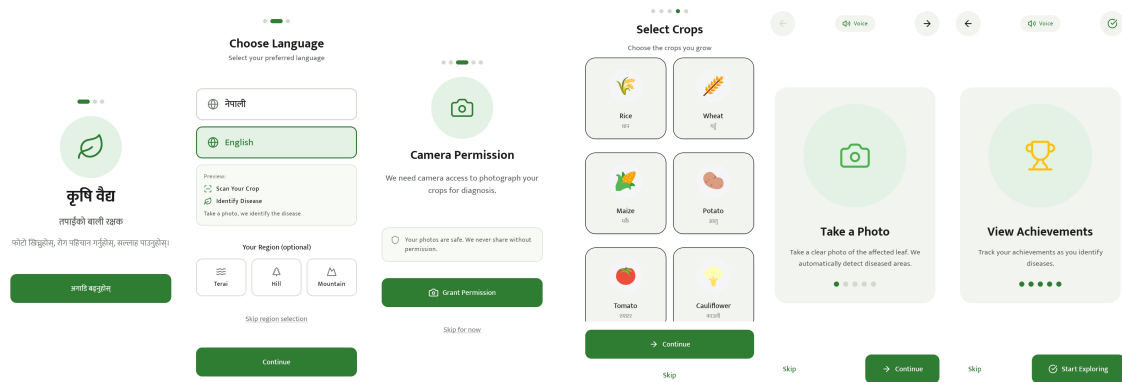


Figure 34: Representative first-run onboarding screenshots showing language selection, camera permission, crop selection, and onboarding completion.

Table 27: Mobile application implementation coverage

Area	Implemented evidence	Result for the manuscript
Application shell	Expo config, root layout, providers, typed routes	Production-shaped app skeleton and controlled startup sequence
Onboarding	Welcome, language, permission, crop, demo screens	First-run user preparation before main workflows
Scan workflow	Capture, preview, editor, analysis, final result screens	End-to-end image-to-report user path in the app
Crop management	Plot dashboard, add/edit crop, tasks, notes, crop weather	Long-term agricultural record keeping beyond diagnosis
History and export	Scan history, grouping/filtering, export and result report support	Retention and reuse of diagnosis outputs
Settings and accessibility	Language, region, model precision, notifications, accessibility, profile	User personalization and operational control
Offline support	SQLite schema, offline queue, sync queue, network monitor, sync manager	Local-first behavior with queued work when connectivity is weak

The application surface is therefore consistent with the thesis objective of building an integrated crop-disease support system. However, the evidence currently comes mainly from source inspection and generated architecture documentation. The manuscript now includes representative screenshots from a running build, but device-specific performance measurements and broader runtime-flow captures are still needed before making stronger deployment claims.

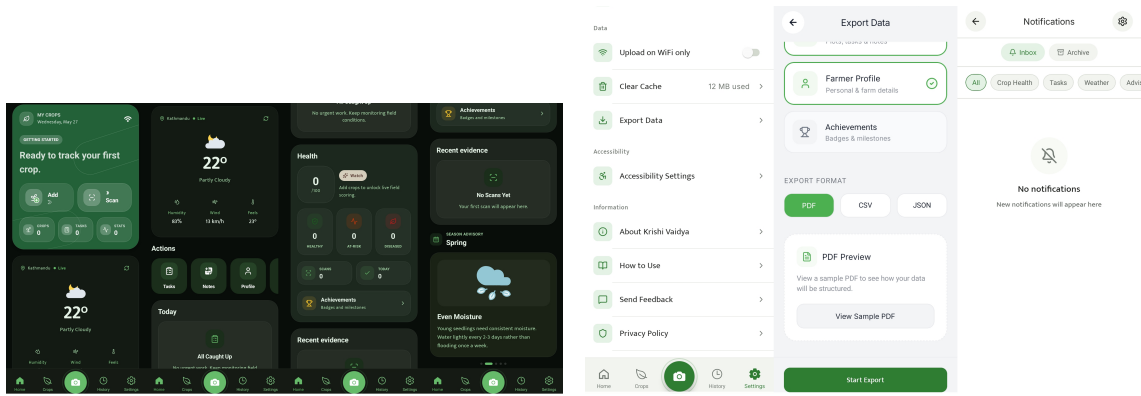


Figure 35: Representative dashboard and settings screenshots showing the home surface, crop summary, export controls, notifications, and accessibility-related configuration.

4.10.2 Scan and Diagnosis Workflow Result

The scan workflow is one of the strongest mobile implementation areas. The native capture view is not limited to opening the camera. It includes selected-crop context, camera permission handling, gallery selection, capture controls, model overlay state, and the ability to pass pre-detection information forward. The preview stage checks image quality before analysis, and the analysis stage uses a visible phase model. This gives the app a controlled interaction sequence: capture or select an image, review image quality, optionally refine the image, run detection, submit for diagnosis, and display a structured report.

The cloud diagnosis path is directly aligned with the backend diagnosis endpoint. The mobile API client posts multipart image data to `/api/v1/diagnose` and accepts an optional crop-name hint. The diagnosis hook then adapts behavior to network status. Online devices attempt cloud diagnosis. Slow connections attempt cloud diagnosis with a five-second fallback threshold. Offline devices queue the scan and attempt cached advisory output. This design is appropriate for agricultural use in Nepal because field connectivity can be unstable, and the application should not fail completely when the server is temporarily unreachable.

The result workflow is also broader than a plain prediction label. The result view model parses diagnosis output for disease fields, advisory content, treatment timeline steps, organic alternatives, voice output, feedback, reminder setup, share-card generation, and PDF export. This matters because advisory delivery and follow-up actions are the parts of the system that make the diagnosis usable to a farmer.

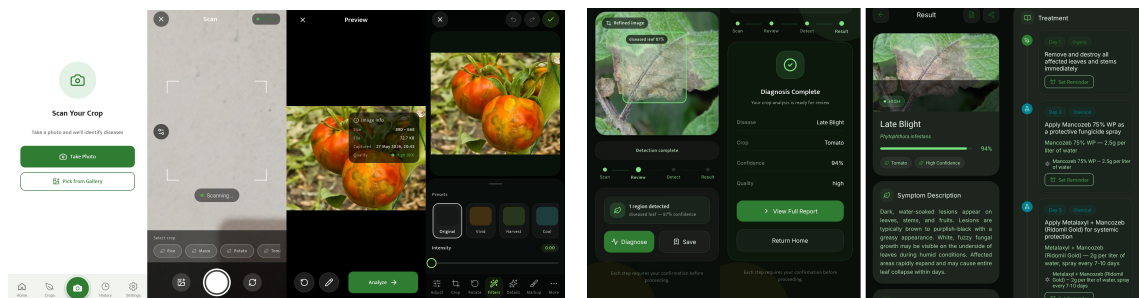


Figure 36: Representative scan and diagnosis screenshots showing capture, preview, editor support, diagnosis completion, structured result display, and treatment guidance.

4.10.3 Local Data and Offline Result

The local data layer provides a substantial offline foundation. Scan history, queued uploads, sync operations, crop plots, crop tasks, crop notes, crop advisories, custom categories, user settings, achievements, reminders, and notifications are represented in SQLite. The app also includes store refresh logic that loads these records into UI-facing state during startup. This allows the app to display prior records and user preferences even when network access is unavailable.

The sync implementation has two important outcomes. First, it provides concrete infrastructure for offline-to-online recovery: pending queue items, batch processing, retry state, network monitoring, app-resume triggers, and periodic synchronization. Second, it makes visible where final integration work remains. The sync manager source marks some generic pending database changes as synced until corresponding backend table-specific push handlers are implemented. Therefore, the correct manuscript interpretation is that offline queueing and sync infrastructure are implemented, while complete two-way synchronization for every local table remains a future integration task.

4.10.4 Crop Management and Farmer Record-Keeping Result

The mobile app includes a crop-management subsystem that complements the diagnosis workflow. The crop dashboard aggregates active plots, upcoming tasks, recent notes, active advisories, crop statistics, network status, and weather-related context. Crop repositories support creating, updating, archiving, filtering, and retrieving crop plots. Task and note repositories support record keeping around farm activities. The data-catalog repository provides read-only crop reference information from fixture files.

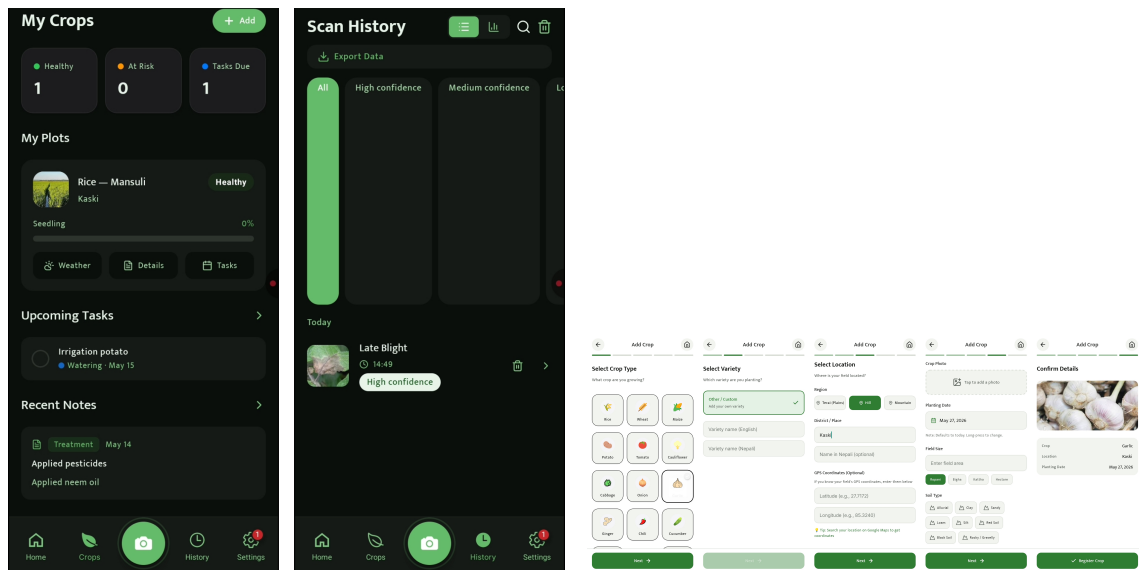


Figure 37: Representative crop-management screenshots showing the crop dashboard and crop-registration flow.

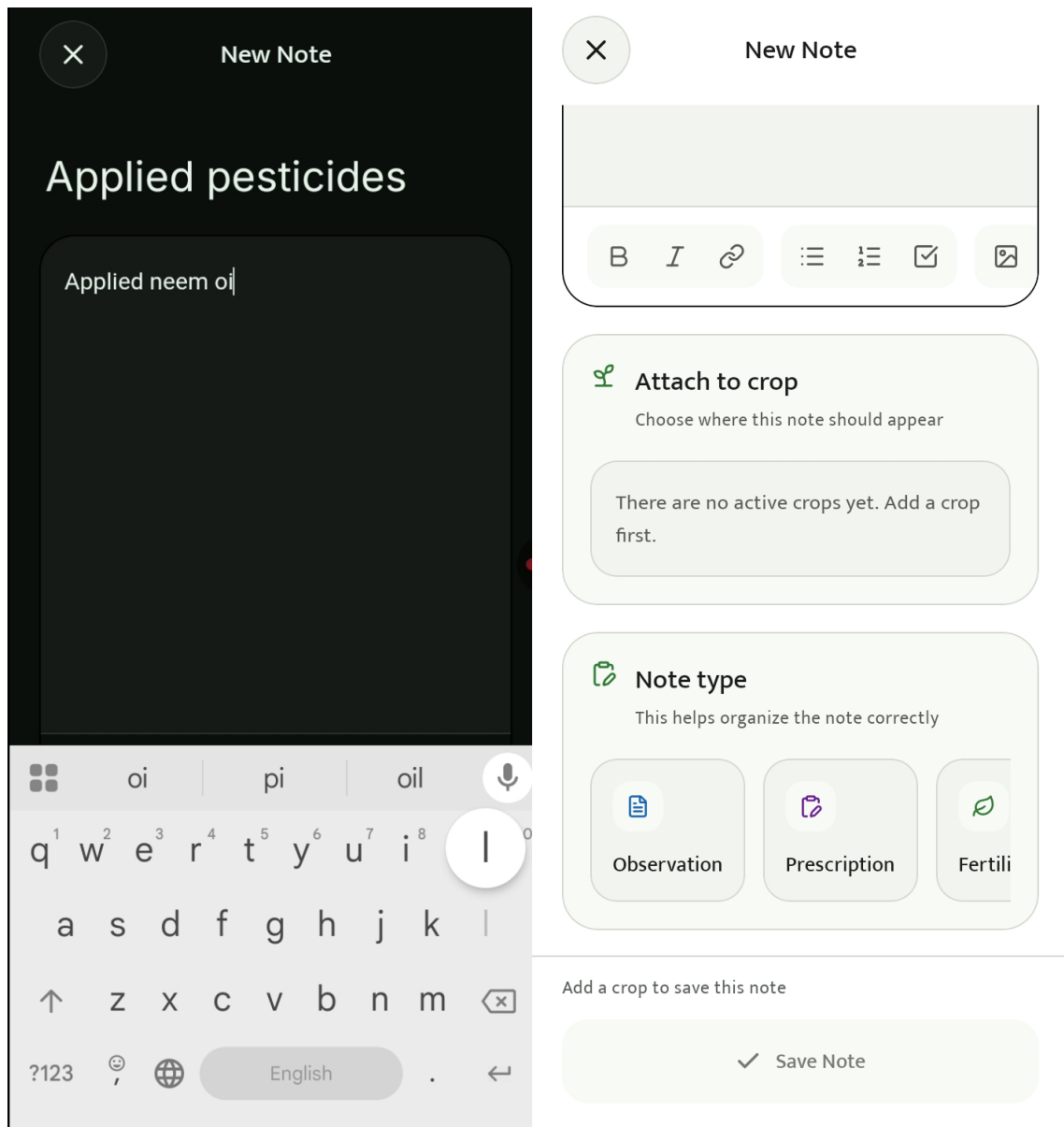


Figure 38: Representative note-creation screenshot showing crop-linked note entry and attachment flow.

This is a strong design decision because a farmer-facing disease application should not treat each scan as isolated. Crop condition, planting date, crop stage, task history, notes, weather context, and advisory reminders all provide context for later decisions. The implementation therefore supports a richer thesis claim: Krishi Vaidya is a mobile crop-health management system with an AI-assisted diagnosis workflow, not only a crop-image classifier.

4.10.5 Model Deployment and Mobile Integration Result

The mobile app includes TFLite model assets in int8, fp16, and fp32 precision variants and exposes model precision as a user-facing setting. The TFLite service attempts GPU

loading and falls back to CPU loading, which is a practical deployment pattern for Android devices with different hardware support. It also contains an inference benchmark method, so the app has a source-level path for measuring load and inference time by precision.

Table 28: Mobile model deployment evidence and integration notes

Item	Implemented evidence	Interpretation
TFLite assets	int8, fp16, and fp32 model files under mobile assets	Multiple mobile deployment variants are packaged
Runtime loader	Singleton TFLite service with selected precision	The app can choose a precision variant at runtime
Delegate strategy	Android GPU attempt with CPU fallback	Device heterogeneity is considered in the implementation
Benchmark hook	Load-time and inference-time return values in service method	Mobile profiling can be performed from existing code paths
Taxonomy alignment	Mobile labels differ from YOLO manuscript detector taxonomy	Final integration review is required before claiming full detector parity

The last row is important. The mobile source currently defines disease labels in the TFLite service, while the YOLO section of this manuscript documents the trained detector from its own training assets. This does not mean the mobile implementation is invalid, but it does mean the final manuscript should avoid claiming that the mobile TFLite package has already been validated as the same detector pipeline unless a separate model-export and label-alignment check is added.

4.10.6 Mobile Verification and Current Limitations

The mobile application has substantial implemented functionality, but the manuscript evidence is not yet complete. Source inspection confirms architecture, routes, repositories, persistence, offline sync infrastructure, diagnosis calls, localization, and model-asset loading paths. The manuscript now includes representative screenshots for onboarding, dashboard, scan, crop, and notes flows, but it still does not by itself prove real-device performance, UI correctness on all supported screen sizes, camera behavior on different Android devices, backend latency, or complete sync behavior across all domains. Those items still need device runs and integration tests.

Table 29: Mobile app verification status and remaining evidence

Area	Current status	Required final evidence
Architecture	Source-backed architecture figure generated	Keep figure and screenshot evidence together
Navigation	Route files and layouts inspected	Representative onboarding and dashboard screenshots included
Scan workflow	Capture, preview, editor, analysis, result code present	Representative scan and diagnosis screenshots included
Crop management	Crop dashboard, add-crop, and notes screenshots captured	Validate the workflows on a running device
Settings and export	Settings, export, and notifications screenshots captured	Record runtime behavior and accessibility checks
Offline support	SQLite, queue, sync manager implemented	Test offline-to-online recovery with real queued scans
Backend integration	Diagnosis client targets <code>/api/v1/diagnose</code>	Record request-response examples and latency measurements
TFLite deployment	Model assets and loader present	Verify exported model labels, precision behavior, and device inference time
Automated tests	Limited inference-engine test harness visible	Add app-level unit, integration, and workflow tests where feasible

Overall, the mobile app section can be written as an implemented and source-verified application component with clear remaining evidence requirements. It should not be reduced to a placeholder: the codebase contains a real application architecture and multiple

completed user-facing workflows. At the same time, the thesis should remain precise by distinguishing implemented source structure from runtime validation that still needs to be captured for the final submission.

4.11 Integrated System Discussion

4.11.1 End-to-End Integration Outcome

The integrated Krishi Vaidya system combines mobile image capture, backend orchestration, VLM prediction, structured advisory generation, and local farmer-facing result handling. The implementation is not a collection of disconnected demonstrations. The mobile app contains the scan workflow and result workflow; the backend exposes the diagnosis route; the VLM gateway exposes readiness and inference endpoints; the advisory client transforms model context into a schema-validated advisory; and the mobile result view consumes the returned structure for display, history, reminders, feedback, voice output, and PDF export.

The most important integration result is the separation of responsibilities. The mobile app does not directly call the VLM service or the Google AI service. The backend does not present raw VLM text to the farmer. The VLM gateway does not decide how treatment should be shown in the app. Each component has a defined boundary, and the data passed across those boundaries is validated or normalized before it moves to the next stage. This structure reduces coupling and makes the system easier to reason about during manuscript writing and future improvement.

Table 30: Integrated subsystem responsibilities

Subsystem	Integration responsibility	Evidence from implementation
Mobile app	Capture image, choose online/offline path, present result, store local state	Scan view models, diagnosis hook, API client, SQLite stores, result view model
Backend API	Validate upload, orchestrate VLM and advisory calls, return stable JSON	Diagnosis route, controller, validation service, diagnosis service, response types
VLM gateway	Validate inference request, protect inference with JWT, call vLLM, return prediction metadata	FastAPI <code>/ready</code> , <code>/v1/inference</code> , request schemas, API tests
VLM model	Generate crop-disease prediction text from image and prompt	Qwen2.5-VL inference gateway and validation outputs documented in VLM section
Advisory generator	Convert image plus VLM context into structured treatment guidance	Google AI advisory client, strict schema, grounding fallback, Zod validation
Local persistence	Preserve scan history, queued scans, settings, crop records, reminders, notifications	SQLite schema, repositories, offline queue, sync queue, stores

4.11.2 Data Contract Strengths

The backend-to-mobile response is the strongest formal contract in the integrated system. It contains request identifier, timestamp, disease fields, crop, confidence, confidence tier, out-of-distribution flag, symptom descriptions, advisory arrays, processing time, and model version. The advisory object separates treatment steps, prevention tips, safety warnings, organic alternatives, and source URLs. This contract gives the mobile app predictable fields for display and reduces the risk of placing unconstrained model

output directly in front of the user.

The backend-to-VLM boundary is also controlled. The backend checks readiness before inference, sends base64 image data and a deterministic prompt, limits generation length, sets temperature to zero, disables sampling, and includes the same request identifier used by the backend response. The VLM gateway validates authentication, image source shape, image validity, and rate limits. It returns latency and token counts, which can later be used for deployment reporting.

The advisory-generation boundary adds another safeguard. The backend does not accept arbitrary text as final treatment advice. The advisory client requires the Google AI response to match a schema before returning it. It also instructs the model to produce Nepali and English fields, to avoid fabricated citations, to include uncertainty behavior, and to avoid unsafe pesticide advice. This does not make the output clinically or agronomically perfect, but it provides an important engineering control around a generative step.

4.11.3 Offline and Degraded-Network Behavior

The integrated workflow is designed for unstable connectivity. The mobile diagnosis hook has separate behavior for online, slow, and offline network states. In a normal online state, it calls the backend. In a slow state, it attempts the backend request but falls back after five seconds. In an offline state, it queues the scan immediately. This design acknowledges a practical constraint: the intended users may be in agricultural field environments where network access is unreliable.

The offline path is not only a UI message. It writes the scan into the local queue, inserts a pending scan record into history, and creates a high-priority sync queue item. If cached advisory data exists for the crop, the app can return a cached result while retaining the queued scan for a future cloud diagnosis. The implementation therefore supports partial usefulness during connectivity loss while keeping the cloud path available when the network returns.

The remaining limitation is that full two-way synchronization across all local tables is not yet proven. The sync manager has the structure for queued operations, retry state, network monitoring, app-resume synchronization, and periodic synchronization, but some generic table-push behavior is still represented as local status updates until matching backend handlers are completed. This distinction should remain clear in the final manuscript.

4.11.4 Reliability Controls Across the Integrated Path

Reliability is handled through multiple layers rather than a single validation point. The mobile client checks that the image URI is usable before building the request. The backend upload route uses single-image upload middleware. The backend validation service checks file presence, file content, MIME consistency, image format, image dimensions, and maximum size. The VLM client checks service readiness and applies request timeouts. The VLM gateway validates JWT scope and image payloads. The advisory client retries configured failures and validates its final JSON schema. The mobile result view also normalizes advisory arrays to avoid downstream crashes when optional advisory fields are missing.

Table 31: Reliability controls in the integrated workflow

Layer	Control	Purpose
Mobile client	Image URI validation, retry interceptor, slow/offline fallback	Avoid invalid local requests and preserve workflow under poor connectivity
Backend upload	Multipart single-image route and upload middleware	Restrict diagnosis route to one uploaded image input
Backend validation	MIME/header match, file size, image dimensions, crop-name validation	Reject corrupt, unsupported, or unsafe request data before model calls
Backend VLM client	Readiness probe, timeout, bearer or RS256 JWT authentication	Protect inference service and avoid uncontrolled hanging requests
VLM gateway	JWT scope check, image validation, rate limiting, vLLM health check	Protect model endpoint and reject invalid inference inputs
Advisory client	Retry, optional grounding fallback, strict schema validation	Prevent invalid generated advisory JSON from reaching mobile clients
Mobile result layer	Safe advisory defaults, local history save, queued state handling	Keep result screens stable across cloud, cached, and queued outputs

These controls are valuable because the system depends on machine-learning and generative-model components. A model can return malformed text, an external service can be unavailable, or a mobile network can fail mid-request. The integration design does not remove those risks, but it limits how far an invalid or unavailable component can propagate through the user-facing workflow.

4.11.5 Integration Limitations and Evidence Gaps

The current integration evidence is source-backed and architecture-backed, but it is not yet a complete deployment evaluation. The manuscript now has the implemented route contracts, generated integration diagram, method description, and integration discussion. However, the final thesis should still include live evidence from the complete running system. The most important missing items are a real mobile-to-backend diagnosis request, captured response JSON, backend logs or screenshots, VLM gateway readiness and inference logs, mobile result screenshots, offline queue demonstration, and measured latency from image submission to final result.

There are also two technical alignment issues that must remain visible. First, VLM evaluation in this manuscript shows a high parse-error rate and low prediction accuracy on the independent test set. The backend advisory layer reduces the risk of exposing raw

unstructured VLM output, but it does not by itself prove that the current VLM is diagnostically reliable. Second, the mobile TFLite asset labels visible in the source differ from the YOLO manuscript detector taxonomy. This requires a final export-label alignment check before claiming that the packaged mobile model is the same detector described in the YOLO section.

Table 32: Integrated system evidence status

Evidence item	Current status	Required before final claim
Architecture and control flow	Generated and documented from source	Review figure placement and wording in final PDF
Mobile-to-backend route contract	Verified from mobile API client and backend route/controller	Capture real request and response from running services
Backend-to-VLM contract	Verified from backend VLM client and FastAPI gateway	Capture readiness and inference logs from running VLM gateway
Advisory schema contract	Verified from advisory client and schema validation	Capture at least one valid generated advisory response
Offline queue behavior	Verified from queue manager and sync infrastructure	Demonstrate offline queue and later recovery in app runtime
Runtime performance	Not yet measured end to end	Measure mobile request latency, backend processing time, and VLM latency
Model alignment	Partially documented with caveats	Reconcile mobile TFLite labels with YOLO training/export assets

4.11.6 Overall Integrated System Interpretation

The integrated system should be described as a substantial implemented prototype with a complete source-backed diagnosis pathway and clear remaining runtime-evidence requirements. Its strongest engineering qualities are modular boundaries, explicit validation, structured response contracts, offline-aware mobile behavior, and farmer-facing result handling. Its main weaknesses are not architectural absence but incomplete final verification: live end-to-end screenshots, latency measurements, mobile model alignment, and broader automated integration tests still need to be added.

This interpretation is important for manuscript accuracy. Overclaiming production readiness would be unsupported, but under-describing the integration would also be inaccurate. The source code shows that Krishi Vaidya is not merely a YOLO model, a VLM experiment, a backend, or a mobile app in isolation. It is an integrated agricultural advisory system whose final thesis narrative should present both the completed engineering structure and the remaining verification work with equal precision.

4.12 Cross-System Results Discussion and Limitations

4.12.1 Synthesis of Major Results

The main result of Krishi Vaidya is an implemented multi-component agricultural advisory system with substantial evidence at the model, backend, mobile, and integration layers. The YOLO subsystem provides the strongest quantitative model result. It has training curves, threshold analysis, deployment comparisons, per-class metrics, confusion-matrix outputs, and error-analysis artifacts. The VLM subsystem provides a complete fine-tuning

and evaluation pipeline, but its prediction results reveal a serious reliability limitation. The backend subsystem provides a structured API layer with domain models, upload validation, authentication/session controls, diagnosis orchestration, advisory generation, and scan-domain tests. The mobile app provides a farmer-facing interface with scan workflow, offline-aware diagnosis behavior, crop management, history, settings, localization, and result actions. The integration workflow connects these components through a source-verified diagnosis path.

The strongest overall contribution is therefore not a single model score. It is the construction of a full crop-health support pipeline that begins with field image capture and ends with a structured advisory report. This matters because many prototype systems stop at a classifier output. Krishi Vaidya contains the additional engineering needed to make the output usable: backend validation, structured response assembly, mobile presentation, scan history, reminders, offline queueing, and farmer-facing language support.

Table 33: Cross-system result synthesis

System area	Strongest result	Main caution
YOLO detector	Quantitative detection and deployment evidence is complete enough for manuscript use	Mobile package alignment and physical-device profiling still need final verification
VLM model	Dataset export, fine-tuning, and validation pipeline are complete and documented	Prediction accuracy and parseability are currently weak on the independent test set
Backend API	Structured diagnosis orchestration, domain models, validation, and scan tests exist	Broader route and diagnosis-integration tests remain incomplete
Mobile app	Complete source-backed farmer workflow and offline-aware structure exist	Final screenshots, device tests, and runtime measurements are missing
Integration	End-to-end source-verified diagnosis path exists across mobile, backend, VLM, and advisory layers	Live request-response and latency evidence still needs to be captured

4.12.2 Model-Level Interpretation

The YOLO model results indicate that the object-detection component is the most mature machine learning part of the project. Its metrics are not perfect, but they are supported by multiple forms of evidence: training dynamics, robust validation, threshold analysis, quantized deployment comparison, latency measurements, and error analysis. This allows the manuscript to discuss realistic deployment tradeoffs instead of only reporting a single final accuracy number.

The VLM results must be interpreted differently. The VLM work is substantial because it includes dataset construction, Qwen-format export, LoRA fine-tuning, quantized training, inference infrastructure, dataset validation, and prediction evaluation. However, the independent prediction metrics show that the current VLM is not yet reliable enough to be presented as a final diagnostic authority. The high parse-error count is especially important because the backend and mobile app depend on structured fields. A model that often fails to produce parseable output can still be useful as a research artifact, but it must not be overclaimed as a dependable diagnosis engine.

The correct interpretation is that the project has one relatively stronger model component and one exploratory but incomplete VLM component. The YOLO section supports confi-

dence in the object-detection work. The VLM section supports confidence in the pipeline and exposes the next research problem: improving structured multimodal disease prediction.

4.12.3 System-Level Interpretation

The backend and mobile results show that the project was engineered as a system rather than as an isolated notebook experiment. The backend provides request validation, route organization, repository-backed domain logic, image upload support, token/session handling, scan lifecycle management, crop/task/note/advisory records, VLM inference bridging, and advisory-schema validation. The mobile app provides the user-facing workflow required to apply those backend features in a field setting.

The important system-level finding is that the engineering structure is ahead of the final runtime evidence. The source code shows a serious architecture, but the final manuscript still needs runtime captures to prove how the system behaves when all services are running together. This is a normal distinction in software-engineering research: implemented structure and runtime validation are related, but they are not the same evidence. The manuscript should use both, and should never replace missing runtime evidence with assumptions.

4.12.4 Reliability and Safety Interpretation

Krishi Vaidya includes several reliability controls that are appropriate for an agricultural advisory system. Images are validated before model calls. Backend diagnosis uses request IDs and processing-time measurement. The VLM service is checked for readiness before inference. The VLM gateway uses JWT scope validation. The advisory generator is required to return schema-valid JSON. The mobile app handles cloud, cached, and queued outcomes. The result view uses structured fields instead of raw model text.

These controls improve engineering safety, but they do not solve every agronomic safety problem. Model uncertainty, class imbalance, field-image variability, pesticide recommendation risk, out-of-distribution images, and incorrect crop hints can still affect final advice. The system therefore should present confidence, uncertainty, retake guidance, safety warnings, and expert consultation guidance where appropriate. The backend already includes confidence-tier and out-of-distribution fields, which should remain central in the final app presentation and manuscript claims.

4.12.5 Current Limitations

The current limitations fall into four groups: model limitations, validation limitations, integration limitations, and manuscript-evidence limitations.

Model limitations are most visible in the VLM results. The VLM has low test accuracy and a high number of parse errors. This limits the strength of claims that can be made about automatic disease diagnosis. The YOLO model is stronger, but its mobile deployment path still requires final alignment between manuscript detector taxonomy and packaged mobile model labels.

Validation limitations are visible in the backend and mobile app. Backend scan tests are useful, but equivalent tests are not yet available for every route group and external diagnosis branch. The mobile app has source-backed architecture and an inference harness, but not a complete recorded emulator or physical-device validation suite. Integration validation is source-verified but not yet demonstrated through a captured live end-to-end request.

Integration limitations are tied to external services and runtime configuration. The backend diagnosis route depends on the VLM service and Google AI advisory service being reachable and properly configured. The advisory generator must return schema-valid JSON. The mobile app must be configured with the correct API base URL and must handle network failure correctly. Offline sync infrastructure exists, but complete two-way synchronization for every local table is not yet proven.

Manuscript-evidence limitations are practical but important. The final report still needs screenshots, live request-response examples, latency measurements, deployment notes, and runtime logs. Without those artifacts, the manuscript can accurately claim implemented architecture and source-verified workflows, but it should not claim full production deployment or field-tested performance.

Table 34: Cross-system limitations and required future evidence

Limitation group	Current limitation	Required action
VLM reliability	Low prediction accuracy and high parse-error count	Improve prompt format, constrained decoding, dataset balance, and evaluation feedback loop
YOLO deployment alignment	Mobile TFLite label list differs from manuscript detector taxonomy	Verify export source, label map, and mobile model package before final claim
Backend test coverage	Scan domain is tested more deeply than other route domains	Add auth, crop, task, note, advisory, diagnosis, and upload-validation tests
Mobile validation	No complete recorded emulator/physical-device workflow evidence yet	Capture screenshots, scan flow, result flow, offline queue, and TFLite behavior
Integration evidence	Source-verified but not yet live end-to-end demonstrated in manuscript	Capture real request-response logs and latency across mobile, backend, VLM, and advisory layers
Offline synchronization	Queue and sync infrastructure exists, but full table sync is incomplete	Add backend handlers and demonstrate offline-to-online recovery
Agronomic safety	Generated advice can still be uncertain or unsafe if not constrained and reviewed	Preserve confidence tiers, safety warnings, source links, and expert consultation guidance

4.12.6 Implications for Final Manuscript Claims

The final manuscript should make strong claims where the evidence is strong and cautious claims where the evidence is incomplete. It can strongly claim that the project implements a modular Krishi Vaidya system with YOLO, VLM, backend, mobile, and integration

components. It can strongly claim that YOLO and VLM datasets and outputs have been analyzed and documented using available figures and tables. It can strongly claim that the backend and mobile application contain substantial implemented architecture.

It should not claim that the current VLM model is already highly accurate. It should not claim that the mobile app has been field-tested unless those tests are actually performed. It should not claim full production readiness until deployment, latency, screenshots, runtime logs, and broader automated tests are added. This disciplined boundary will make the thesis stronger, not weaker, because it shows that the project team understands the difference between implementation, evaluation, and deployment.

CHAPTER 5

5. CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

Krishi Vaidya was developed as an integrated crop-health support system that combines computer vision, vision-language modelling, backend orchestration, and a mobile application into one project. The work demonstrates that a final-year B.Sc. CSIT project can move beyond a single model experiment and produce a connected software system with trained model components, application services, mobile workflows, structured data storage, and validation artifacts.

The YOLO component establishes the object-detection layer of the system. It was trained on a curated plant-part detection dataset containing 30,981 images and 147,467 object annotations across eleven agricultural object classes. The validation and result analysis show that the model achieved usable object-detection behavior for the intended plant-part localization task, with the best recorded training metrics reaching precision 0.71717, recall 0.74868, mAP50 0.76163, and mAP50-95 0.60191. The deployment study further examined threshold behavior and quantized variants, which is important because the target system includes mobile and edge-oriented constraints.

The VLM component establishes the disease-interpretation layer. The project prepared a structured vision-language dataset, validated the dataset pipeline, fine-tuned a Qwen-based VLM using LoRA and quantized training techniques, and evaluated prediction behavior through recorded validation outputs. The VLM work is technically significant because it supports diagnosis-oriented text and structured output rather than only fixed-label classification. At the same time, the current VLM results reveal an important limitation: the recorded prediction metrics and parse-error behavior show that the VLM pipeline is implemented and measurable, but not yet reliable enough to be treated as a final diagnostic authority without further improvement.

The backend component provides the application control layer. It implements authentication, domain routes, data models, repository abstraction, scan handling, diagnosis orchestration, advisory generation, validation, and test coverage for the scan path. The backend is not only a transport layer between the mobile app and the models; it is the component that turns model output into a structured application response and maintains system records.

The mobile application provides the user-facing layer. It implements routing, screen structure, provider and repository patterns, local SQLite persistence, offline and synchronization support, crop and task management, image-based scan workflow, backend diagnosis communication, model asset support, localization, and farmer-facing output. This makes the project usable as an application prototype rather than only a collection of scripts.

The integrated workflow connects these components into a diagnosis path: the mobile app captures or selects an image, the backend receives and validates the scan request, the VLM service handles model inference, the backend transforms the result into an advisory response, and the mobile app stores and displays the outcome. The integration has been verified from source structure and documented data contracts, while live end-to-end runtime evidence remains a required final validation step.

Overall, Krishi Vaidya satisfies the core objective of building an integrated agricultural diagnosis-support prototype. Its strongest current evidence lies in the completed YOLO artifacts, the complete VLM data and training pipeline, the implemented backend, the implemented mobile app, and the documented integration architecture. Its most important remaining work is not basic implementation, but reliability improvement, executed end-to-end testing, mobile runtime evidence, field evaluation, and expert validation of diagnosis and advisory content.

5.2 Major Contributions

The major contributions of this project are as follows:

1. A curated YOLO object-detection dataset and trained detector for agricultural plant-part localization, including validation metrics, threshold analysis, quantization comparison, and error analysis.
2. A structured VLM dataset and fine-tuning pipeline for crop disease interpretation using Qwen-based vision-language modelling, LoRA adaptation, quantized training, prediction parsing, and validation reporting.
3. A backend API that organizes authentication, crop records, scan records, diagnosis services, advisory logic, repository abstraction, validation middleware, and scan-related tests.
4. A React Native/Expo mobile application that supports farmer-facing scan workflow, local records, offline persistence, synchronization structure, crop and task management, localization, and backend diagnosis communication.

5. A documented integration workflow that connects the mobile app, backend, VLM inference service, and advisory response into a single diagnosis-support path.
6. A manuscript-ready planning and evidence system that maps model outputs, figures, tables, architecture diagrams, limitations, and remaining validation requirements.

5.3 Recommendations

Krishi Vaidya should be continued as an evidence-first system. The next work should not only add features, but should strengthen the reliability of the existing components. The recommendations are grouped into dataset and model improvements, deployment improvements, and mobile/user evaluation improvements.

5.3.1 Dataset and model improvements

The YOLO dataset should be expanded with more balanced class coverage. The current object distribution is useful for training, but some classes have much fewer annotations than others. Additional images for under-represented classes such as maize ear, rice grain cluster, cauliflower head, and cabbage head would improve the detector’s ability to generalize across plant structures. More field images should also be collected under varied lighting, occlusion, growth stages, camera quality, and background conditions.

The VLM dataset should be improved with stricter output supervision. The current validation shows that parse errors are a major limitation. Future dataset preparation should enforce consistent JSON-style response templates, include negative and uncertain cases, and add examples where the correct behavior is to decline overconfident diagnosis. The VLM should also be evaluated with expert-reviewed labels and advisory text rather than relying only on automated parsing metrics.

The diagnosis pipeline should separate disease classification confidence from advisory confidence. A model may identify a likely disease while still lacking enough context to recommend chemical or agronomic intervention. Future versions should include confidence calibration, uncertainty labels, and escalation guidance such as “consult an agricultural expert” when the visual evidence is weak.

5.3.2 System deployment improvements

The backend and VLM services should be tested under a live end-to-end deployment. The manuscript currently documents source-verified integration, but final deployment should

include request logs, response examples, latency measurements, failure-mode testing, authentication checks, and readiness behavior. These results should be archived and added to the appendix before final field deployment claims are made.

The backend test suite should be expanded beyond scan-focused tests. Additional tests should cover authentication, crops, notes, tasks, advisories, diagnosis history, validation failures, file upload limits, and authorization boundaries. This is necessary because the backend is responsible for data integrity and user-specific records.

The model-serving boundary should be hardened. The VLM service should expose documented health, readiness, timeout, and error responses. The backend should handle VLM unavailability without crashing the user workflow. Where possible, model outputs should be schema-validated before being stored or returned to the mobile app.

5.3.3 Mobile app and user evaluation improvements

The mobile app should be tested on real Android devices and, if available, different device classes. The final evidence should include screenshots, scan workflow recordings, local storage behavior, offline-to-online synchronization, response latency, error messages, and farmer-facing readability checks. These tests are necessary because mobile usability cannot be fully proven from source code inspection alone.

The application should also be evaluated with intended users. Farmer-facing language, disease names, severity explanations, and advisory instructions should be checked with agricultural experts and local users. If Nepali-language output is used, translation quality must be evaluated carefully because unclear advice can be more harmful than no advice.

Future versions should include stronger safety boundaries. The mobile app should avoid presenting model output as a guaranteed diagnosis. It should show uncertainty, recommend expert consultation for severe or unclear cases, and avoid unsupported pesticide prescriptions unless the advisory content has been reviewed by qualified agricultural personnel.

5.4 Future Enhancements

The following enhancements are recommended for future development:

1. Add a complete end-to-end test run from mobile image submission to backend response, VLM inference, advisory generation, local storage, and result display.

2. Improve VLM structured-output reliability by training with stricter response schemas and adding parser-aware validation during evaluation.
3. Add expert-reviewed disease and advisory labels for the VLM validation set.
4. Expand the YOLO dataset with more balanced class representation and more field-condition images.
5. Add final mobile screenshots, device-test evidence, and offline synchronization demonstrations to the appendix.
6. Add backend API documentation or Swagger/OpenAPI-style endpoint summaries for easier maintenance.
7. Add deployment documentation for environment variables, model-service startup, backend startup, mobile configuration, and production readiness checks.
8. Conduct a small controlled user evaluation with students, farmers, or agricultural technicians to assess usability and clarity of advisory output.
9. Add privacy and data-protection review for uploaded crop images and user records before real-world deployment.
10. Extend Nepali-language support for disease explanations and farmer-facing recommendations after expert review.

REFERENCE

- Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tan, S., et al. (2025). Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*. <https://arxiv.org/abs/2502.13923>
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). Qlora: Efficient fine-tuning of quantized llms. *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/2305.14314>
- Expo. (2026). *Expo documentation*. Retrieved May 27, 2026, from <https://docs.expo.dev/>
- Express.js. (2026a). *Express routing*. Retrieved May 27, 2026, from <https://expressjs.com/en/guide/routing.html>
- Express.js. (2026b). *Using express middleware*. Retrieved May 27, 2026, from <https://expressjs.com/en/guide/using-middleware.html>
- FastAPI. (2026). *Fastapi documentation*. Retrieved May 27, 2026, from <https://fastapi.tiangolo.com/>
- Food and Agriculture Organization of the United Nations. (2026a). *About plant production and protection*. Retrieved May 27, 2026, from <https://www.fao.org/plant-production-protection/about/en>
- Food and Agriculture Organization of the United Nations. (2026b). *Fao supports nepal in developing a digital agriculture strategy and action plan*. Retrieved May 27, 2026, from <https://www.fao.org/digital-villages-initiative/asia-pacific/news/fao-supports-nepal-in-developing-a-digital-agriculture-strategy-and-action-plan/en>

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*. <https://arxiv.org/abs/2106.09685>
- Liu, J., & Wang, X. (2021). Plant diseases and pests detection based on deep learning: A review. *Plant Methods*, 17(22). <https://doi.org/10.1186/s13007-021-00722-9>
- Meta Open Source. (2026). *React native: Introduction*. Retrieved May 27, 2026, from <https://reactnative.dev/docs/getting-started.html>
- Ministry of Agriculture and Livestock Development. (2024). *Statistical information on nepalese agriculture, 2080/81*. Government of Nepal. Retrieved May 27, 2026, from https://www.giwmscdnone.gov.np/media/pdf_upload/Statistical%20Information%20on%20Nepalese%20Agriculture%2C%202080_81_a7s8aj4.pdf
- Mohanty, S. P., Hughes, D. P., & Salathe, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science*, 7. <https://doi.org/10.3389/fpls.2016.01419>
- Mongoose. (2026). *Schemas*. Retrieved May 27, 2026, from <https://mongoosejs.com/docs/guide.html>
- National Statistics Office. (2024). *National sample census of agriculture 2021/22, nsca 2021/22*. Government of Nepal. Retrieved May 27, 2026, from <https://microdata.nsonnepal.gov.np/index.php/catalog/134/study-description>
- Ultralytics. (2026a). *Explore ultralytics yolov8*. Retrieved May 27, 2026, from <https://docs.ultralytics.com/models/yolov8/>
- Ultralytics. (2026b). *Model export with ultralytics yolo*. Retrieved May 27, 2026, from <https://docs.ultralytics.com/modes/export/>

Wang, P., Bai, S., Tan, S., Wang, S., Fan, Z., Bai, J., Chen, K., Liu, X., Wang, J., Ge, W., et al. (2024). Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*. <https://arxiv.org/abs/2409.12191>

APPENDIX

Appendix A: Supplementary Model Outputs

The manuscript-ready YOLO outputs are stored in `data/krishi_vaidya_yolo_data`. This folder contains the training dynamics figure, threshold sweep figure, quantization comparison figure, confusion matrix figure, training dynamics table, threshold sweep table, deployment summary table, per-class F1 table, and error analysis table used in Chapter 4.

The manuscript-ready VLM outputs are stored in `data/krishi_vaidya_vlm_data`. This folder contains the VLM training dynamics figure, memory/runtime figure, dataset composition figure, prediction performance figure, confusion matrix figure, inference diagnostics figure, training summary table, runtime resource table, dataset split table, class distribution table, prediction summary table, per-class metrics table, and prediction distribution table used in Chapter 4.

The generated architecture and validation figures are stored in the backend, mobile, integration, and validation data folders under `data/`. These include backend architecture, mobile app architecture, integrated diagnosis workflow, and validation coverage figures.

Appendix B: Backend API Details

The backend source is located in the project `backend/` directory. The implemented API structure includes route groups for authentication, crops, scans, diagnoses, advisories, tasks, notes, uploads, and user-related operations. The manuscript documents these route groups in Chapter 3 and Chapter 4 using source-backed tables.

Final submission evidence should add one of the following backend artifacts if available:

1. API request-response examples for the scan and diagnosis workflow.
2. Screenshots or exported documentation from an API client such as Postman or Swagger/OpenAPI.
3. Executed backend test logs for unit and integration tests.
4. Environment-variable documentation for local and deployment runs.

Appendix C: Mobile Screenshots

Mobile screenshots are still required for final submission. The source-backed manuscript sections already document the implemented mobile route structure, scan workflow, local persistence, offline-aware behavior, backend diagnosis client, and model asset support. However, final visual evidence should be captured from a running mobile build or emulator.

Recommended screenshots are:

1. Home or dashboard screen.
2. Crop record or crop detail screen.
3. Image capture or image selection screen.
4. Scan review screen before submission.
5. Diagnosis result screen.
6. Scan history screen.
7. Offline or synchronization status screen, if available.
8. Settings or language/localization screen.

Appendix D: Additional Setup and Build Notes

The manuscript is built from `manuscript/main.tex`. The PowerShell build script is located at `manuscript/scripts/build.ps1`. The latest verified build produced `manuscript/build/main.pdf`.

The project-level implementation evidence is distributed across the following directories:

1. `yolo1ml/`: YOLO training, validation, telemetry, and model-related scripts.
2. `vlm_model/`: VLM dataset preparation, fine-tuning, validation, inference gateway, and tests.
3. `backend/`: Express/Mongoose API, services, repositories, models, middleware, and tests.
4. `mobile_app/app/`: Expo React Native mobile application, feature modules, local storage, scan workflow, and API client logic.

5. `manuscript/data/`: manuscript-ready model figures, tables, generated architecture diagrams, workflow diagrams, and validation assets.

Before final submission, the following evidence should be added if time permits: complete executed-test logs, mobile screenshots, live end-to-end request-response logs, latency measurements, and expert-reviewed advisory examples.